REACT

CHEAT SHEETS

A Complete Reference for Modern React 19

JSX · Components & Props · State & Effects · Forms & Actions · Context · Hooks · Suspense & Transitions · Performance · TypeScript · Routing · Data Fetching · Server Components · Accessibility · Testing

98 REFERENCE SHEETS

CONTENTS

FOUNDATIONS

01–04	Introduction & Setup
05–09	JSX
10–15	Components & Props
16–22	State
23–25	Events
26–31	Forms

HOOKS & MODERN REACT

32–38	Refs & Effects
39–41	Context
42–45	Rules of Hooks & Custom Hooks
46–53	Concurrent & Modern Features
54–56	Error Handling & Debugging
57–62	Performance
63–66	TypeScript with React

ECOSYSTEM & SHIPPING

67–70	Styling
71–76	Routing with React Router
77–80	Data Fetching
81–84	State Management Libraries
85–88	Server Rendering & Server Components
89–90	Accessibility
91–94	Testing
95–98	Build, Deploy & Quick Reference

Table of Contents

INTRODUCTION & SETUP

01 Introduction & Mental Model
02 Creating a Project
03 Project Structure & Tooling
04 Rendering & the Root

JSX

05 JSX Syntax
06 JSX Expressions
07 Conditional Rendering
08 Rendering Lists
09 Fragments, Escaping & Raw HTML

COMPONENTS & PROPS

10 Function Components
11 Props
12 children & Composition
13 Composition Patterns
14 Keeping Components Pure
15 Class Components (Legacy)

STATE

16 useState
17 State Updates & Batching
18 Objects & Arrays in State
19 Structuring State
20 Sharing State Between Components
21 Preserving & Resetting State
22 useReducer

EVENTS

23 Event Handling
24 Event Propagation
25 Keyboard, Pointer & Focus Events

FORMS

26 Controlled Inputs
27 Uncontrolled Inputs
28 Form Actions
29 useActionState & useFormStatus
30 React Hook Form
31 Form Validation

REFS & EFFECTS

32 useRef
33 Refs to DOM Elements
34 useEffect
35 Effect Dependencies
36 You Might Not Need an Effect
37 Fetching Data in Effects
38 useLayoutEffect & useInsertionEffect

CONTEXT

CONCURRENT & MODERN FEATURES

46 Suspense
47 The use() API
48 useTransition & startTransition
49 useDeferredValue
50 useOptimistic
51 Activity
52 View Transitions
53 Document Metadata & Resources

ERROR HANDLING & DEBUGGING

54 Error Boundaries
55 Root Error Handling & StrictMode
56 React DevTools

PERFORMANCE

57 How Rendering Works
58 memo
59 useMemo & useCallback
60 React Compiler
61 Code Splitting & Lazy Loading
62 Profiling & Large Lists

TYPESCRIPT WITH REACT

63 TypeScript: Props & Components
64 TypeScript: Hooks
65 TypeScript: Events & Forms
66 TypeScript: Generic & Polymorphic Components

STYLING

67 Styling Basics
68 CSS Modules
69 Tailwind CSS in React
70 CSS-in-JS

ROUTING WITH REACT ROUTER

71 React Router: Setup & Modes
72 React Router: Navigation
73 React Router: Params & Search Params
74 React Router: Nested Routes & Layouts
75 React Router: Loaders & Actions
76 React Router: Framework Mode

DATA FETCHING

77 TanStack Query: Setup & Queries
78 TanStack Query: Mutations & Cache
79 TanStack Query: Advanced
80 SWR

STATE MANAGEMENT LIBRARIES

81 Choosing State Management
82 Redux Toolkit: Store & Slices
83 Redux Toolkit: Async & RTK Query

Sheet 01

Introduction & Mental Model

UI = f(state) · components · declarative rendering · one-way data flow · react vs react-dom · library vs framework

WHAT REACT IS

React is a JavaScript library for building user interfaces out of **components**: functions that take data (props and state) and return a description of the UI. You describe *what* the screen should look like for the current data, and React works out *how* to update the DOM when the data changes. Data flows one way, from parent to child, and changes travel back up through callbacks.

NOTES

- `react` holds components and hooks; `react-dom` renders them to the browser DOM
- React is a library, not a framework: routing, data loading and builds come from other tools
- Frameworks (Next.js, React Router framework mode) add routing, server rendering and bundling
- A re-render means calling your component again; React then commits only the DOM changes

♿ ACCESSIBILITY

- React outputs ordinary HTML, so accessibility depends on the elements and attributes you render
- Prefer native elements (`button` , `a` , `label`) over clickable `div` s; React doesn't add roles for you

VERSION NOTES

- Current release: React 19.3 (September 2026); no React 20 has been announced
- React 18 added concurrent rendering; React 19 added Actions, `use()` , ref as a prop and metadata tags
- Keep `react` and `react-dom` on the same version

CODE EXPLAINED

→ `Counter` is a component: a function whose name starts with a capital letter and returns JSX

→ `label` arrives as a **prop** from the parent; props are read-only

→ `useState` gives each `Counter` its own `count` ; the two counters are independent

→ Clicking calls `setCount` , which schedules a re-render with the new value

→ React compares the new output with the old and updates only the changed text node

UI AS A FUNCTION OF STATE

```
import { useState } from 'react';

// A component: data in, UI out
function Counter({ label }) {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      {label}: {count}
    </button>
  );
}

// Composing components into a tree
export default function App() {
  return (
    <main>
      <h1>Scoreboard</h1>
      <Counter label="Home" />
      <Counter label="Away" />
    </main>
  );
}

// Mental model:
//   UI = f(props, state)
//   event -> setState -> re-render -> commit DOM diff
```

CORE IDEAS

IDEA	MEANING
`Component`	Function returning UI; reusable and composable
`Props`	Inputs passed from parent to child
`State`	Data a component owns that changes over time
`Declarative`	Describe the result, not the DOM steps
`One-way data flow`	Data down via props, events up via callbacks
`Reconciliation`	Diffing old and new output to update the DOM

PACKAGES

PACKAGE	ROLE
`react`	Components, hooks, `Suspense`, `memo`
`react-dom`	Browser rendering: `createRoot`, `preload`, portals
`react-dom/server`	HTML streaming for server rendering
`react-dom/client`	`createRoot`, `hydrateRoot`
`Next.js / React Router`	Frameworks: routing, data, SSR, builds

⚠ COMMON MISTAKES

- Manipulating the DOM directly (`document.querySelector`) instead of changing state
- Treating React as a full framework and expecting built-in routing or data fetching
- Mixing major versions of `react` and `react-dom`

✓ TIPS & BEST PRACTICES

- Think in terms of data: what state exists, and what UI follows from it
- Start new apps with Vite or a framework, not a hand-rolled setup
- Read react.dev's *Thinking in React* before building your first app

QUICK SUMMARY

- Components are functions of props and state
- Data flows down; events flow up
- React diffs and commits DOM changes for you
- `react` + `react-dom` work together

Creating a Project

STARTING A NEW APP

Create a client-side React app with **Vite**: one command scaffolds a project with a dev server, hot module replacement and a production build. Pick the `react-ts` template to get TypeScript from the start. If you need routing, server rendering or data loading built in, start from a **framework** instead (Next.js or React Router framework mode). **Create React App** is deprecated and should not be used for new projects.

NOTES

- Vite needs a current Node.js LTS release; check `node -v` before scaffolding
- `npm create vite@latest` prompts for a framework and variant; flags skip the prompts
- Templates: `react`, `react-ts`, `react-swc`, `react-swc-ts` (SWC is a faster compiler)
- Use `npm`, `pnpm`, `yarn` or `bun`; the lockfile records which one the project uses

♿ ACCESSIBILITY

- Set `<html lang>` and a meaningful `<title>` in `index.html` right away
- Add `eslint-plugin-jsx-a11y` while the project is small

VERSION NOTES

- Create React App was deprecated in February 2025; migrate CRA apps to Vite or a framework
- React's docs recommend a framework for most new apps and Vite for client-only apps
- `npm create vite` installs React 19 by default

CODE EXPLAINED

→ `--` passes the `--template` flag through npm to the Vite scaffolder

→ `npm run dev` serves source files directly with instant hot updates

→ `npm run build` runs `tsc -b` then `vite build`, writing static files to `dist/`

→ `npm run preview` checks the real build before deploying

→ `-D` saves test tools as devDependencies so they stay out of the bundle

SCAFFOLD, RUN, BUILD

```
# create a TypeScript React app
npm create vite@latest my-app -- --template react-ts
cd my-app
npm install

# start the dev server (http://localhost:5173)
npm run dev

# type-check and bundle for production -> dist/
npm run build

# serve the production build locally
npm run preview

# add common libraries later
npm install react-router @tanstack/react-query
npm install -D vitest @testing-library/react

# framework alternatives
npx create-next-app@latest my-app
npx create-react-router@latest my-app
```

PACKAGE.JSON SCRIPTS

SCRIPT	RUNS
`dev`	`vite`: dev server with HMR
`build`	`tsc -b && vite build`
`preview`	`vite preview`: serve `dist/`
`lint`	`eslint .`
`test`	`vitest` (after you add it)

CHOOSING A STARTER

NEED	USE
`Client-only SPA, widget, dashboard`	Vite
`SEO, server rendering, file routes`	Next.js
`Data loaders, SSR, Remix-style app`	React Router framework
`Static docs site`	Astro / Docusaurus
`Learning or sandbox`	StackBlitz / CodeSandbox

⚠ COMMON MISTAKES

- Starting a new project with `create-react-app`
- Forgetting `--` so npm swallows `--template`
- Deploying the `dev` server instead of the `dist/` build

✓ TIPS & BEST PRACTICES

- Choose `react-ts` even for small apps; types catch prop mistakes early
- Commit the lockfile so every machine installs the same versions
- Run `npm run preview` before deploying to catch build-only bugs

QUICK SUMMARY

- `npm create vite@latest -- --template react-ts`
- `dev` · `build` · `preview`
- Framework when you need SSR or routing
- CRA is deprecated

Project Structure & Tooling

WHAT VITE GENERATES

A Vite React project is small: an `index.html` with a `<div id="root">`, a `src/main.tsx` entry that mounts the app, and an `App.tsx` root component. Static files that must keep their exact names go in `public/`; everything else is imported from `src/` so Vite can bundle and fingerprint it. **ESLint** with `eslint-plugin-react-hooks` catches hook mistakes; **Prettier** handles formatting.

NOTES

- `index.html` sits at the project root and is the real entry point for Vite
- Files in `public/` are served at `/` as-is; imported assets get hashed file names
- Group code by feature (`features/cart/`) once the app outgrows flat folders
- Path aliases (`@/components`) need both `vite.config.ts` and `tsconfig.json`

♿ ACCESSIBILITY

- `index.html` holds `<html lang="en">` ; change it for non-English apps
- Enable `eslint-plugin-jsx-a11y` rules alongside the hooks plugin

VERSION NOTES

- Recent `eslint-plugin-react-hooks` releases include the React Compiler lint rules
- Current Vite templates use flat ESLint config (`eslint.config.js`)

CODE EXPLAINED

→ `index.html` loads `/src/main.tsx` with `<script type="module">`

→ `@vitejs/plugin-react` handles JSX, Fast Refresh and the React Compiler hook-in

→ The `@` alias lets you write `import x from '@/hooks/x'`

→ `public/favicon.svg` is copied unchanged; `src/assets` files are fingerprinted

→ `features/` keeps components, hooks and tests for one feature together

PROJECT LAYOUT

```
my-app/
├── index.html          # <div id="root"> + script tag
├── package.json
├── vite.config.ts
├── tsconfig.json
├── eslint.config.js
├── public/
│   └── favicon.svg     # served as /favicon.svg
└── src/
    ├── main.tsx        # createRoot(...).render(<App/>)
    ├── App.tsx         # root component
    ├── index.css       # global styles
    ├── assets/         # imported images, fonts
    ├── components/     # shared UI
    ├── features/       # feature folders
    └── hooks/          # custom hooks

// vite.config.ts
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  resolve: { alias: { '@': '/src' } },
});
```

KEY FILES

FILE	PURPOSE
`index.html`	HTML shell, root div, entry script
`src/main.tsx`	Creates the React root
`src/App.tsx`	Top-level component
`vite.config.ts`	Plugins, aliases, dev server
`tsconfig.json`	TypeScript settings, path aliases
`eslint.config.js`	Lint rules (hooks, a11y)

TOOLING

TOOL	JOB
ESLint	Bugs and hook-rule violations
`eslint-plugin-react-hooks`	Rules of Hooks, deps, compiler rules
Prettier	Formatting only
TypeScript	Type checking (`tsc --noEmit`)
React DevTools	Inspect components in the browser

⚠ COMMON MISTAKES

- Putting imported images in `public/` and losing cache-busting hashes
- Adding an alias in Vite but not in `tsconfig.json` (or the reverse)
- Turning off `react-hooks/exhaustive-deps` instead of fixing the dependency

✓ TIPS & BEST PRACTICES

- Keep one component per file, named after the component
- Co-locate a component's styles and tests next to it
- Run ESLint and Prettier on save so reviews skip style debates

QUICK SUMMARY

- `index.html` → `main.tsx` → `App.tsx`
- `public/` = copied; `src/` = bundled
- Lint hooks with the official plugin
- Organize by feature as you grow

Rendering & the Root

createRoot · root.render · <StrictMode> · root.unmount · multiple roots · hydrateRoot · ReactDOM.render (removed)

MOUNTING THE APP

A React app starts by creating a **root** for a DOM node and rendering a component into it. `createRoot` comes from `react-dom/client`; `root.render(<App />)` draws the tree, and calling it again updates it. Wrap the app in `<StrictMode>` during development: it double-invokes renders and effects to expose impure code. For server-rendered HTML, use `hydrateRoot` instead so React attaches to existing markup.

NOTES

- Most apps call `createRoot` once; widgets embedded in other pages may create several roots
- `root.unmount()` removes the tree and runs every cleanup
- `StrictMode` checks run in development only and have no production cost
- Root options include `onCaughtError`, `onUncaughtError` and `identifierPrefix`

♿ ACCESSIBILITY

- Mount into an element inside `<body>` so landmarks render in document order
- Render a `<main>` landmark inside your root layout

VERSION NOTES

- `ReactDOM.render` and `ReactDOM.hydrate` were removed in React 19
- `createRoot` arrived in React 18 and enables concurrent features
- React 19 added root error callbacks (`onCaughtError` , `onUncaughtError`)

CODE EXPLAINED

- → `createRoot(container)` takes over the DOM node; existing children are replaced
- → The `!` tells TypeScript the element exists; check it in libraries
- → `onUncaughtError` reports errors no error boundary caught
- → `<StrictMode>` wraps the tree so every component gets development checks
- → `hydrateRoot` reuses server HTML instead of rebuilding it

MAIN.TSX

```tsx
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App';
import './index.css';

const container = document.getElementById('root')!;
const root = createRoot(container, {
  onUncaughtError(error, info) {
    reportError(error, info.componentStack);
  },
});

root.render(
  <StrictMode>
    <App />
  </StrictMode>,
);

// Later: tear everything down
// root.unmount();

// Server-rendered HTML instead:
// import { hydrateRoot } from 'react-dom/client';
// hydrateRoot(container, <App />);

// Legacy (removed in React 19):
// ReactDOM.render(<App />, container);
```

ROOT APIS

API	USE
`createRoot(el, opts?)`	Client-rendered app
`root.render(node)`	Render or update the tree
`root.unmount()`	Remove the tree, run cleanups
`hydrateRoot(el, node)`	Attach to server HTML
`createPortal(node, el)`	Render into another DOM node

STRICTMODE CHECKS

CHECK	CATCHES
`Double render`	Impure components
`Double effect run`	Missing cleanup
`Double ref callback`	Missing ref cleanup
`Deprecated API warnings`	Legacy context, string refs

⚠ COMMON MISTAKES

- Calling `createRoot` twice on the same element
- Removing `StrictMode` to silence double effects instead of adding cleanup
- Using `createRoot` on server HTML and causing a full re-render flash

✓ TIPS & BEST PRACTICES

- Keep `main.tsx` tiny: providers belong in `App` or a `Providers` component
- Wire root error callbacks to your error reporter early
- Use portals for modals that must escape `overflow: hidden` parents

QUICK SUMMARY

- `createRoot(el).render(<App />)`
- `hydrateRoot` for SSR markup
- `StrictMode` = dev-only checks
- `ReactDOM.render` is gone in 19

JSX Syntax

MARKUP IN JAVASCRIPT

JSX is a syntax extension that lets you write HTML-like markup inside JavaScript. The compiler turns each tag into a call to `jsx()` from `react/jsx-runtime`, which returns a plain object describing the element. JSX is stricter than HTML: every tag must close, a component returns **one root element**, and attributes use **camelCase** DOM property names such as `className`, `htmlFor` and `onClick`.

NOTES

- Lowercase tags (`div`) are DOM elements; capitalized tags (`Card`) are components
- `class` and `for` are reserved words in JS, so JSX uses `className` and `htmlFor`
- `aria-*` and `data-*` attributes keep their hyphens
- `style` takes an object with camelCase keys and number values in px

♿ ACCESSIBILITY

- Every `<img>` needs `alt` (use `alt=""` for decorative images)
- `htmlFor` links a `<label>` to an input by `id` ; screen readers announce the label

VERSION NOTES

- The automatic JSX runtime (React 17+) removed the need to `import React` in every file
- `React.createElement` still works but is rarely written by hand

CODE EXPLAINED

→ Parentheses around multi-line JSX stop automatic semicolon insertion after `return`

→ `width={48}` passes a number; `width="48"` would pass a string

→ `style={{ ... }}` : outer braces enter JS, inner braces are the object literal

→ Void elements such as `<br />` and `<img />` must self-close

→ `aria-label` and `data-id` keep HTML spelling

JSX AND WHAT IT COMPILES TO

```jsx
function Profile({ user }) {
  return (
    <section className="profile">
      <img
        src={user.avatar}
        alt=""
        width={48}
        style={{ borderRadius: 8, objectFit: 'cover' }}
      />
      <label htmlFor="bio">Bio</label>
      <textarea id="bio" defaultValue={user.bio} />
      <br />
      <button aria-label="Edit profile" data-id={user.id}>
        ✎
      </button>
    </section>
  );
}

// Compiled output (automatic runtime):
import { jsx, jsxs } from 'react/jsx-runtime';

jsxs('section', {
  className: 'profile',
  children: [jsx('img', { src: user.avatar, alt: '' })],
});
```

HTML VS JSX

HTML	JSX
`class`	`className`
`for`	`htmlFor`
`onclick`	`onClick` (function, not string)
`tabindex`	`tabIndex`
`style="color:red"`	`style={{ color: 'red' }}`
` `	` `
`<!-- -->`	`{/* */}`

RULES

RULE	EXAMPLE
One root per return	wrap in or <>…</>
Close every tag	<input />
camelCase props	autoComplete, readOnly
Capitalize components	<UserCard />
Booleans	disabled = disabled={true}

⚠ COMMON MISTAKES

- Writing `class=` and getting a console warning
- Passing a string to an event prop: `onClick="save()"`
- Naming a component in lowercase so React treats it as an HTML tag

✓ TIPS & BEST PRACTICES

- Let Prettier format JSX; it adds the wrapping parentheses for you
- Use the HTML-to-JSX converter in your editor when pasting markup
- Keep inline `style` for dynamic values; put the rest in CSS

QUICK SUMMARY

- JSX → `jsx()` calls → element objects
- One root, closed tags, camelCase
- `className` , `htmlFor` , `style={{}}`
- `aria-*` / `data-*` keep hyphens

JSX Expressions

{} · expressions vs statements · attribute expressions · template literals · {/* comments */} · string vs expression props

CURLY BRACES OPEN JAVASCRIPT

Inside JSX, **curly braces** switch back to JavaScript. Anything that produces a value works: variables, function calls, ternaries, template literals and array `map()`. Statements do not: `if`, `for` and variable declarations must happen above the `return`. In attributes, quotes pass a string and braces pass any value, including numbers, objects and functions.

NOTES

- Braces work in two places: as element children and as attribute values
- `null`, `undefined`, `true` and `false` render nothing
- Numbers and strings render as text; plain objects throw an error
- Text inside braces is escaped automatically, which prevents HTML injection

♿ ACCESSIBILITY

- Build `alt` and `aria-label` text from data so it stays accurate
- Don't put meaning only in an emoji; add visible or `aria-label` text

VERSION NOTES

- Unchanged since early React; JSX expressions are plain JavaScript

CODE EXPLAINED

→ `fullName` is computed above the `return`, where statements are allowed

→ `className` takes an expression, so a ternary picks the class

→ `{/* … */}` is the only way to comment inside JSX

→ Template literals build attribute strings from data

→ `{'{braces}'}` renders literal braces as text

EXPRESSIONS IN MARKUP

```jsx
const user = { first: 'Ada', last: 'Lovelace', unread: 3 };
const formatDate = (d) => d.toLocaleDateString('en-US');

function Header({ today }) {
  const fullName = `${user.first} ${user.last}`;
  const isBusy = user.unread > 0;

  return (
    <header className={isBusy ? 'header busy' : 'header'}>
      {/* comments need braces too */}
      <h1>Hello, {fullName}!</h1>
      <p>Today is {formatDate(today)}</p>
      <p>{user.unread} unread {user.unread === 1
        ? 'message' : 'messages'}</p>
      <img
        src={`/avatars/${user.first.toLowerCase()}.png`}
        alt={`${fullName}'s avatar`}
      />
      <span title="literal string">{'{braces}'}</span>
    </header>
  );
}
```

ALLOWED IN {}

WORKS	EXAMPLE
Variables	`{name}`
Calls	`{format(date)}`
Ternary	`{ok ? 'Yes' : 'No'}`
Logical &&	`{count > 0 && <Badge />}`
Templates	`{Hi ${name}}`
Array map	`{items.map(...)}`

NOT ALLOWED IN {}

FAILS	DO INSTEAD
`if / else`	ternary or early return
`for` loops	`array.map()`
`const x = …`	declare above `return`
Plain objects	render a property: `{obj.name}`
`switch`	lookup object or helper function

⚠ COMMON MISTAKES

- Rendering an object (`{user}`) and getting *Objects are not valid as a React child*
- Quoting an expression: `src="{url}"` passes the literal text
- Expecting `{false}` or `{null}` to print the word

✓ TIPS & BEST PRACTICES

- Move long expressions into named variables above the `return`
- Use `String(value)` when you want booleans to show as text
- Use `JSON.stringify(obj, null, 2)` inside `<pre>` for quick debugging

QUICK SUMMARY

- `{}` = JavaScript expression
- Quotes = string, braces = any value
- Statements go above `return`
- `null` / `false` render nothing

Conditional Rendering

SHOWING DIFFERENT UI

React has no special conditional syntax: you use JavaScript. Return early with `if`, choose between two pieces of JSX with a **ternary**, or show something only when a condition is true with **&&**. Return `null` to render nothing. Watch the left side of &&: if it is the number `0`, React renders `0`.

NOTES

- `cond && <X />` renders `<X />` when `cond` is truthy, otherwise renders `cond` itself
- `false`, `null` and `undefined` render nothing; `0` and `NaN` render as text
- Element variables (`let content = …`) keep complex branches readable
- Returning `null` still runs the component's hooks

♿ ACCESSIBILITY

- Hiding content by not rendering it also removes it from screen readers
- Announce status changes (loading, errors) with a live region, not just a swapped element

VERSION NOTES

- Unchanged across React versions

CODE EXPLAINED

→ Early returns handle the cases where nothing else should render

→ `body` holds JSX like any other value

→ The ternary chooses text for the heading

→ `messages.length > 0` is a boolean, so `&&` never renders `0`

→ The commented line shows the classic `0` bug

FOUR PATTERNS

```jsx
function Inbox({ user, messages, loading, error }) {
  // 1. Early return
  if (!user) return <SignIn />;
  if (error) return <p role="alert">{error.message}</p>;

  // 2. Element variable
  let body;
  if (loading) body = <Spinner />;
  else body = <MessageList items={messages} />;

  return (
    <section>
      {/* 3. Ternary */}
      <h2>{messages.length ? 'Inbox' : 'No mail'}</h2>

      {/* 4. && with a boolean on the left */}
      {messages.length > 0 && (
        <Badge count={messages.length} />
      )}

      {/* Bug: renders "0" when empty */}
      {/* {messages.length && <Badge />} */}

      {body}
    </section>
  );
}
```

PICK A PATTERN

SITUATION	USE
Whole component differs	early `return`
Two alternatives	`cond ? <A /> : <B />`
Show or nothing	`cond && <A />`
Many branches	element variable or lookup object
Render nothing	`return null`

WHAT RENDERS

VALUE	OUTPUT
`true / false`	nothing
`null / undefined`	nothing
`0`	`0`
`''`	nothing (empty text)
`NaN`	NaN

⚠ COMMON MISTAKES

- `{count && <X />}` printing `0`
- Nesting ternaries three deep instead of using early returns
- Calling hooks after an early `return`, which breaks the Rules of Hooks

✓ TIPS & BEST PRACTICES

- Coerce with `!!value` or a comparison before `&&`
- Put all hooks at the top, then the early returns
- Use a map object (`{ idle: <A/>, done: <B/> }[status]`) for status switches

QUICK SUMMARY

- `if` + early return
- Ternary for either/or
- `&&` with a boolean only
- `null` = render nothing

Rendering Lists

map() · key · stable IDs · index as key · filter then map · keyed Fragment

ARRAYS BECOME ELEMENTS

To render a list, transform an array of data into an array of elements with `map()`. Each item needs a **key**: a string or number that is unique among its siblings and stays the same for that item across renders. React uses keys to match items between renders, so a stable key keeps each item's state and DOM attached to the right data when the list is sorted, filtered or edited.

NOTES

- Keys only need to be unique among siblings, not across the whole app
- Use IDs from your data: database IDs, UUIDs, slugs
- `key` is not passed to the component as a prop; pass the ID separately if needed
- Array index keys are safe only for static lists that never reorder or change length

♿ ACCESSIBILITY

- Render lists with `<ul>` / `<ol>` and `<li>` so screen readers announce item counts
- When filtering, announce the new result count in a live region

VERSION NOTES

- Unchanged; React 19 still warns when list items lack keys

CODE EXPLAINED

→ `filter()` runs first, then `map()` turns each todo into an element

→ `key={todo.id}` goes on the outermost element returned from `map()`

→ `id` is passed again because components can't read `key`

→ `<Fragment key>` groups `<dt>` and `<dd>` without an extra wrapper

→ The short `<>` syntax can't take a key

LISTS WITH KEYS

```jsx
const todos = [
  { id: 'a1', text: 'Write docs', done: true },
  { id: 'b2', text: 'Ship release', done: false },
];

function TodoList({ items }) {
  const pending = items.filter((t) => !t.done);

  return (
    <ul>
      {pending.map((todo) => (
        <TodoItem key={todo.id} id={todo.id} todo={todo} />
      ))}
    </ul>
  );
}

function TodoItem({ id, todo }) {
  return <li data-id={id}>{todo.text}</li>;
}

// Several elements per item: keyed Fragment
import { Fragment } from 'react';

function Glossary({ terms }) {
  return (
    <dl>
      {terms.map((t) => (
        <Fragment key={t.id}>
          <dt>{t.word}</dt>
          <dd>{t.meaning}</dd>
        </Fragment>
      ))}
    </dl>
  );
}
```

KEY CHOICES

SOURCE	VERDICT
`Database ID`	best
`crypto.randomUUID() at creation`	good (store it)
`Slug / unique name`	good if stable
`Array index`	only for static lists
`Math.random() in render`	never: new key every render

INDEX-KEY BUGS

ACTION	SYMPTOM
`Insert at top`	inputs show the wrong values
`Sort`	checkbox state stays in place
`Delete`	next item inherits state
`Filter`	animations attach to wrong rows

⚠ COMMON MISTAKES

- Generating keys during render with `Math.random()` or `Date.now()`
- Putting `key` on the inner element instead of the one returned by `map()`
- Using index keys for editable or sortable lists

✓ TIPS & BEST PRACTICES

- Assign IDs when items are created, not when they are rendered
- Extract the item into its own component once it has more than a line of JSX
- Use a keyed `Fragment` when each item renders siblings

QUICK SUMMARY

- `items.map(i => <X key={i.id} />)`
- Keys: unique among siblings, stable
- Index keys only for static lists
- Keyed `Fragment` for multi-element items

Fragments, Escaping & Raw HTML

GROUPING, TEXT AND RAW HTML

A **Fragment** (<>…</>) groups children without adding a DOM node, which keeps tables, flex layouts and lists valid. JSX trims whitespace at line breaks, so adjacent elements on separate lines touch; insert `{' '}` where you want a space. All text in braces is **escaped**, so user input can't inject markup. To render trusted HTML strings, use `dangerouslySetInnerHTML` and sanitize first.

NOTES

- Use `<Fragment key>` when mapping; the `<>` shorthand takes no props
- Whitespace inside a single line is kept; line breaks between tags are removed
- HTML entities like ` ` work in JSX text, or write `{'\u00a0'}`
- `dangerouslySetInnerHTML` replaces all children; the element must have none

♿ ACCESSIBILITY

- Fragments keep `<tr>` directly inside `<tbody>` and `<li>` inside `<ul>`, preserving table and list semantics
- Sanitized HTML still needs headings and alt text checked

VERSION NOTES

- React 19.3: **Fragment refs** are stable; `<Fragment ref>` gives a `FragmentInstance` for focus, observers and event listeners on its children
- `<>` shorthand since React 16.2

CODE EXPLAINED

→ `Row` returns `<td>` s without a wrapper that would break the table

→ `{' '}` keeps the space between `</strong>` and *on*

→ `DOMPurify.sanitize` removes scripts and event attributes before injection

→ `__html` is deliberately awkward so raw HTML stands out in reviews

→ A Fragment ref can focus the first focusable child without a wrapper div

FRAGMENTS, SPACES AND HTML

```jsx
import { Fragment, useRef } from 'react';
import DOMPurify from 'dompurify';

function Row({ cells }) {
  return (
    <>
      {cells.map((c) => <td key={c.id}>{c.value}</td>)}
    </>
  );
}

function Byline({ author, date }) {
  return (
    <p>
      By <strong>{author}</strong>{' '}
      on <time>{date}</time>
    </p>
  );
}

function Article({ html }) {
  const clean = DOMPurify.sanitize(html);
  return <div dangerouslySetInnerHTML={{ __html: clean }} />;
}

function Toolbar({ children }) {
  const group = useRef(null);  // FragmentInstance
  // group.current.focus() -> first focusable child
  return <Fragment ref={group}>{children}</Fragment>;
}
```

FRAGMENT FORMS

FORM	USE
`<>…</>`	Group children, no props
`<Fragment key>`	Grouped list items
`<Fragment ref>`	Focus, observe or listen on children (19.3)
`Array return`	Works but needs keys

WHITESPACE

JSX	RENDERS
`<b>A</b> <b>B</b> (one line)`	A B
`Tags on separate lines`	AB
`{' '}`	single space
` `	non-breaking space
`{'\n'}`	newline (visible in <pre>)

⚠ COMMON MISTAKES

- Wrapping with a `<div>` inside `<tr>` or `<ul>` and creating invalid HTML
- Passing unsanitized user HTML to `dangerouslySetInnerHTML` (XSS)
- Losing a space between inline elements on separate lines

✓ TIPS & BEST PRACTICES

- Prefer Markdown renderers that output React elements over raw HTML
- Let your formatter insert `{' '}` where it is needed
- Reach for Fragment refs before adding wrapper divs just to hold a ref

QUICK SUMMARY

- `<>` groups without a DOM node
- `{' '}` for spaces across lines
- Text is escaped by default
- Sanitize before `dangerouslySetInnerHTML`

Function Components

capitalized names · function vs arrow · default vs named export · returning JSX · one component per file · no nested definitions

COMPONENTS ARE FUNCTIONS

A **function component** is a JavaScript function that returns JSX. Its name must start with a **capital letter** so JSX treats `<Avatar />` as a component and `<avatar />` as an HTML tag. Components are defined at the top level of a module and exported for use elsewhere. React calls the function on every render, so it should only compute and return UI.

NOTES

- Function declarations and arrow functions both work; pick one style per codebase
- A file can export one **default** and any number of **named** exports
- Return `null` to render nothing; returning `undefined` is allowed since React 18
- Components can be as small as one element; split when a piece has its own purpose

♿ ACCESSIBILITY

- Give reusable components props for `aria-label`, `id` and `aria-describedby` so callers can label them
- Forward unknown props (`...rest`) to the root element so ARIA attributes reach the DOM

VERSION NOTES

- Function components with hooks are the standard since React 16.8
- `React.FC` is optional; most codebases type props directly instead

CODE EXPLAINED

→ The default export is imported without braces and can be renamed

→ Named exports are imported in braces with their exact names

→ `...rest` collects props like `onClick` and spreads them onto `<button>`

→ Arrow components need parentheses to return JSX implicitly

→ A component defined inside another is recreated every render, so its state resets

DEFINING AND EXPORTING

```jsx
// Avatar.tsx: default export
export default function Avatar({ src, name }) {
  return <img className="avatar" src={src} alt={name} />;
}

// Buttons.tsx: named exports
export function PrimaryButton({ children, ...rest }) {
  return <button className="btn primary" {...rest}>
    {children}
  </button>;
}

export const GhostButton = ({ children, ...rest }) => (
  <button className="btn ghost" {...rest}>{children}</button>
);

// App.tsx
import Avatar from './Avatar';
import { PrimaryButton, GhostButton } from './Buttons';

export default function App() {
  return (
    <div>
      <Avatar src="/ada.png" name="Ada" />
      <PrimaryButton onClick={save}>Save</PrimaryButton>
      <GhostButton>Cancel</GhostButton>
    </div>
  );
}

// ✗ Never define components inside components
function Parent() {
  function Child() { return <p />; }  // new type each render
  return <Child />;
}
```

EXPORT STYLES

STYLE	IMPORT
`export default function X`	`import X from './X'`
`export function X`	`import { X } from './X'`
`export const X = () =>`	`import { X } from './X'`
`Re-export barrel`	`export { X } from './X'`

NAMING

RULE	EXAMPLE
`PascalCase component`	`UserCard`
`File matches component`	`UserCard.tsx`
`Handlers: handle*`	`handleSubmit`
`Callback props: on*`	`onSubmit`
`Boolean props: is/has`	`isOpen`, `hasError`

⚠ COMMON MISTAKES

- Lowercase component names (`<userCard />`) rendering as unknown HTML tags
- Defining a component inside another component's body
- Forgetting to return the JSX from a block-bodied arrow function

✓ TIPS & BEST PRACTICES

- Default-export page-level components, name-export shared ones
- Keep components small enough to read without scrolling
- Accept and spread `...rest` on design-system components

QUICK SUMMARY

- Capitalized function returning JSX
- Define at module top level
- Default or named exports
- Split by purpose, not by size

Props

passing props · destructuring · default values · read-only · {...spread} · boolean shorthand

PASSING DATA DOWN

Props are the arguments a parent passes to a child, written like HTML attributes. The child receives them as a single object, usually destructured in the parameter list. Any value can be a prop: strings, numbers, objects, arrays, functions and JSX. Props are **read-only**: a component must never change its own props; it asks the parent to change them instead.

NOTES

- Default values go in the destructuring pattern: `{ size = 'md' }`
- A prop with no value (`<Modal open />`) is `true`
- Spread an object to pass many props: `<Avatar {...user} />`
- When props change, React re-renders the child with the new values

♿ ACCESSIBILITY

- Pass accessible names (`label` , `alt`) as required props so callers can't forget them
- Use `aria-*` props for state (`aria-expanded={isOpen}`), not just visual classes

VERSION NOTES

- React 19 ignores `defaultProps` on function components; use default parameters
- `propTypes` checks were removed in React 19; use TypeScript

CODE EXPLAINED

→ Destructuring lists exactly which props `Button` accepts

→ `variant = 'primary'` applies only when the prop is `undefined`

→ `disabled` with no value passes `true`

→ `{...avatarProps}` spreads `src` and `size` ; `alt` is added after

→ Later props win: `<X {...a} b={1} />` overrides `a.b`

PROPS IN, UI OUT

```jsx
function Button({
  label,
  variant = 'primary',   // default value
  disabled = false,
  onClick,
}) {
  return (
    <button
      className={`btn btn-${variant}`}
      disabled={disabled}
      onClick={onClick}
    >
      {label}
    </button>
  );
}

function Toolbar({ user, onSave }) {
  const avatarProps = { src: user.photo, size: 32 };

  return (
    <div className="toolbar">
      <Avatar {...avatarProps} alt={user.name} />
      <Button label="Save" onClick={onSave} />
      <Button label="Delete" variant="danger" disabled />
    </div>
  );
}

// ✗ Mutating props
function Bad({ items }) {
  items.push('x');   // changes the parent's array
  return null;
}
```

PROP VALUE TYPES

PASS	SYNTAX
String	`title="Hi"`
Number	`size={32}`
Boolean true	`open` or `open={true}`
Object	`style={{ gap: 8 }}`
Function	`onSave={handleSave}`
JSX	`icon={<Star />}`

PROPS VS STATE

PROPS	STATE
Passed in by parent	Owned by the component
Read-only	Updated with a setter
Configures the component	Remembers changes over time
Change → parent re-renders	Change → this component re-renders

⚠ COMMON MISTAKES

- Mutating a prop object or array
- Copying a prop into state and wondering why it stops updating
- Spreading all props onto a DOM element and leaking unknown attributes

✓ TIPS & BEST PRACTICES

- Destructure in the signature so the prop list is visible
- Prefer several simple props over one big config object
- Name callback props `onSomething` to match DOM events

QUICK SUMMARY

- Props = function arguments
- Defaults via destructuring
- Read-only; ask the parent to change
- `{...obj}` spreads, later props win

children & Composition

NESTING COMPONENTS

Whatever you put between a component's opening and closing tags arrives as the **`children`** prop. That lets a component wrap content it knows nothing about: cards, layouts, dialogs. For more than one slot, pass JSX through named props such as `header` or `footer`. This **composition** is React's main tool for reuse; you rarely need inheritance.

NOTES

- `children` can be text, elements, arrays, functions or nothing
- Type it as `ReactNode` in TypeScript
- Named slot props take any JSX: `<Page sidebar={<Nav />} />`
- *Specialization*: build `WelcomeDialog` by rendering `Dialog` with fixed props

♿ ACCESSIBILITY

- Layout components should render landmarks (`<header>` , `<nav>` , `<main>`) around their slots
- A `Card` that wraps a heading should let callers choose the heading level

VERSION NOTES

- `children` behavior is unchanged in React 19

CODE EXPLAINED

- → `Card` places `children` without knowing what they are
- → `Layout` has three slots: two named props and `children`
- → JSX passed as a prop is just a value; nothing renders until the receiver places it
- → `DangerCard` reuses `Card` by composing it, not extending it
- → Content between tags becomes `children` , including multiple elements

CHILDREN AND SLOTS

```jsx
function Card({ title, children }) {
  return (
    <article className="card">
      <h2>{title}</h2>
      <div className="card-body">{children}</div>
    </article>
  );
}

function Layout({ header, sidebar, children }) {
  return (
    <div className="layout">
      <header>{header}</header>
      <nav>{sidebar}</nav>
      <main>{children}</main>
    </div>
  );
}

// Specialization
function DangerCard(props) {
  return <Card {...props} title={`⚠ ${props.title}`} />;
}

export default function App() {
  return (
    <Layout header={<Logo />} sidebar={<Menu />}>
      <Card title="Welcome">
        <p>Anything can go here.</p>
        <button>Get started</button>
      </Card>
      <DangerCard title="Billing">Card expired</DangerCard>
    </Layout>
  );
}
```

COMPOSITION TOOLS

TOOL	WHEN
`children`	One main content area
`Named JSX props`	Several slots
`Specialized wrapper`	Preset version of a generic component
`Render prop`	Child needs data from the parent
`Context`	Many levels need the same data

CHILDREN VALUES

WRITE	CHILDREN IS
`<X>Hi</X>`	`'Hi'`
`<X><A /></X>`	one element
`<X><A /><B /></X>`	array of two
`<X />`	`undefined`
`<X>{fn}</X>`	a function

⚠ COMMON MISTAKES

- Passing everything through props when `children` would read better
- Inspecting or rewriting `children` instead of using explicit props
- Using class inheritance to share UI

✓ TIPS & BEST PRACTICES

- Pass JSX down instead of data plus flags (`icon={<Star />}` over `icon="star"`)
- Composition often removes the need for context
- Keep layout components free of data fetching

QUICK SUMMARY

- Tag contents → `children`
- Named props for extra slots
- Compose, don't inherit
- Type as `ReactNode`

Composition Patterns

compound components · render props · higher-order components · components as props · Children API · cloneElement (legacy)

REUSABLE STRUCTURE

Beyond `children`, a few patterns share structure between components. **Compound components** split one widget into parts that coordinate through context (`<Tabs>`, `<Tabs.List>`, `<Tabs.Panel>`). **Render props** pass a function that returns JSX. **Higher-order components** wrap a component to add behavior. Custom hooks have replaced most render props and HOCs, but you will still meet all three in libraries.

NOTES

- Compound components keep their shared state private and expose a readable API
- A render prop can be `children` as a function: `<Mouse>{pos => …}</Mouse>`
- HOCs are named `withX` and return a new component
- `Children.map` and `cloneElement` inspect or modify children; both are fragile

♿ ACCESSIBILITY

- Compound widgets must wire ARIA between parts (`role="tablist"` , `aria-controls` , `aria-selected`)
- Headless libraries (React Aria, Radix) supply this wiring; prefer them for tabs, menus and listboxes

VERSION NOTES

- React docs list `Children` and `cloneElement` as legacy APIs
- Custom hooks (16.8+) are the default way to share logic

CODE EXPLAINED

→ `Tabs` owns the state; parts read it through context

→ `<TabsContext value>` is the React 19 provider syntax

→ Attaching parts as `Tabs.Tab` groups the API under one import

→ Each `Tab` sets `aria-selected` so assistive tech knows the active tab

→ Callers arrange the parts however they want

COMPOUND TABS

```jsx
import { createContext, use, useState } from 'react';

const TabsContext = createContext(null);

export function Tabs({ defaultValue, children }) {
  const [value, setValue] = useState(defaultValue);
  return (
    <TabsContext value={{ value, setValue }}>
      {children}
    </TabsContext>
  );
}

Tabs.Tab = function Tab({ value, children }) {
  const ctx = use(TabsContext);
  const selected = ctx.value === value;
  return (
    <button role="tab" aria-selected={selected}
      onClick={() => ctx.setValue(value)}>
      {children}
    </button>
  );
};

Tabs.Panel = function Panel({ value, children }) {
  const ctx = use(TabsContext);
  return ctx.value === value
    ? <div role="tabpanel">{children}</div> : null;
};

// Usage
<Tabs defaultValue="a">
  <div role="tablist">
    <Tabs.Tab value="a">Profile</Tabs.Tab>
    <Tabs.Tab value="b">Billing</Tabs.Tab>
  </div>
  <Tabs.Panel value="a">…</Tabs.Panel>
</Tabs>
```

PATTERN COMPARISON

PATTERN	TODAY
`Compound components`	Common in UI libraries
`Render props`	Rare; mostly replaced by hooks
`HOCs (withX)`	Legacy; still in some libraries
`Custom hooks`	Default for sharing logic
`cloneElement`	Avoid; pass props explicitly

RENDER PROP VS HOOK

RENDER PROP	HOOK
`<Mouse>{p => <Dot at={p} />}</Mouse>`	`const p = useMouse()`
`Adds nesting`	Flat
`Works in class components`	Function components only
`Shares logic + markup`	Shares logic only

⚠ COMMON MISTAKES

- Using `cloneElement` to inject props that break when children are wrapped
- Stacking several HOCs and losing track of where props come from
- Building tabs or menus without keyboard support

✓ TIPS & BEST PRACTICES

- Reach for a custom hook first, compound components second
- Throw a clear error when a part is used outside its parent
- Use a headless library for complex ARIA widgets

QUICK SUMMARY

- Compound = parts sharing context
- Render prop = function returning JSX
- HOC = `withX(Component)`
- Hooks replace most of these

Keeping Components Pure

pure render · side effects in handlers · local mutation · StrictMode double render · idempotence · props as snapshots

RENDER MUST BE PURE

React expects components to be **pure functions** of their props, state and context: given the same inputs, return the same JSX, and don't change anything outside the function while rendering. Side effects (network calls, timers, DOM changes, logging to services) belong in **event handlers** or, as a last resort, **effects**. `StrictMode` renders components twice in development to surface impurity.

NOTES

- Mutating variables or objects created *during* this render is fine (local mutation)
- Mutating props, state, context or module-level variables is not
- React may render a component several times before committing, or not commit at all
- Purity is what lets React pause, resume and skip renders safely

♿ ACCESSIBILITY

- Impure renders cause flicker and duplicated content that confuses screen readers

VERSION NOTES

- StrictMode double rendering exists since React 18; React 19 dims the second log in DevTools
- The React Compiler assumes purity; impure components can break when memoized

CODE EXPLAINED

→ `BadGuest` shows different numbers in StrictMode because it renders twice

→ `Guest` returns the same output for the same `number`

→ `guests` is created inside the render, so pushing to it is safe

→ `analytics.track` during render fires on every re-render, not once per visit

→ Work triggered by the user belongs in the event handler

PURE VS IMPURE

```jsx
let guestCount = 0;

// ✗ Impure: changes a module variable during render
function BadGuest() {
  guestCount = guestCount + 1;
  return <h2>Guest #{guestCount}</h2>;
}

// ✓ Pure: output depends only on props
function Guest({ number }) {
  return <h2>Guest #{number}</h2>;
}

// ✓ Local mutation is fine
function GuestList({ count }) {
  const guests = [];
  for (let i = 1; i <= count; i++) {
    guests.push(<Guest key={i} number={i} />);
  }
  return <>{guests}</>;
}

// ✗ Side effect in render
function Tracker({ page }) {
  analytics.track(page);        // runs on every render
  return <p>{page}</p>;
}

// ✓ Side effects go in handlers or effects
function SaveButton({ data }) {
  const save = () => api.save(data);   // on click
  return <button onClick={save}>Save</button>;
}
```

WHERE SIDE EFFECTS GO

EFFECT	PLACE
Save on click	event handler
Fetch when shown	effect or data library
Subscribe to a store	useSyncExternalStore
Measure DOM	useLayoutEffect
Log page view	effect keyed on the page

MUTATION RULES

TARGET	ALLOWED IN RENDER?
Local variables	yes
Objects created this render	yes
Props	no
State	no (use the setter)
Module/global variables	no
ref.current	no (except lazy init)

⚠ COMMON MISTAKES

- Calling `setState` during render without a guard (infinite loop)
- Sorting a prop array in place: `items.sort()` mutates the parent's data
- Disabling StrictMode to hide double-render symptoms

✓ TIPS & BEST PRACTICES

- Copy before sorting: `items.toSorted()` or `[...items].sort()`
- Read `Date.now()` and random values in handlers or state initializers
- Treat StrictMode warnings as bugs

QUICK SUMMARY

- Same inputs → same JSX
- No outside changes during render
- Local mutation is OK
- Effects go in handlers first

Class Components (Legacy)

extends Component · render() · this.state · setState · componentDidMount · componentWillUnmount · hook equivalents

CLASSES YOU'LL STILL MEET

Before hooks, components with state were **classes** extending `React.Component`. They define `render()`, keep state in `this.state`, update it with `this.setState()`, and react to mounting and updates with **lifecycle methods**. New code uses function components, but classes still appear in older codebases and are the only way to write an **error boundary** without a library.

NOTES

- `setState` merges the object you pass into existing state (hooks replace instead)
- `this` must be bound for handlers; class fields with arrow functions do it for you
- Classes can't use hooks; wrap them or convert them
- `PureComponent` skips re-renders when props and state are shallowly equal

♿ ACCESSIBILITY

- Focus management in classes used `componentDidUpdate` plus refs; the same rules apply with hooks

VERSION NOTES

- Classes are still supported in React 19 but receive no new features
- String refs and legacy context (`contextTypes`) were removed in React 19
- `UNSAFE_componentWillMount` and similar remain deprecated

CODE EXPLAINED

→ `state = {…}` is a class field that sets initial state

→ `componentDidMount` runs once after the first render is in the DOM

→ `componentWillUnmount` cleans up the interval

→ `tick` is an arrow class field, so `this` is bound

→ The hook version puts setup and cleanup together in one `useEffect`

A CLASS COMPONENT AND ITS HOOK VERSION

```jsx
import { Component, useEffect, useState } from 'react';

class Clock extends Component {
  state = { now: new Date() };

  componentDidMount() {
    this.id = setInterval(this.tick, 1000);
  }

  componentWillUnmount() {
    clearInterval(this.id);
  }

  tick = () => this.setState({ now: new Date() });

  render() {
    return <time>{this.state.now.toLocaleTimeString()}</time>;
  }
}

// Same component with hooks
function ClockFn() {
  const [now, setNow] = useState(new Date());

  useEffect(() => {
    const id = setInterval(() => setNow(new Date()), 1000);
    return () => clearInterval(id);
  }, []);

  return <time>{now.toLocaleTimeString()}</time>;
}
```

LIFECYCLE → HOOKS

CLASS	HOOK
`constructor / state =`	`useState`
`componentDidMount`	`useEffect(fn, [])`
`componentDidUpdate`	`useEffect(fn, [deps])`
`componentWillUnmount`	effect cleanup
`shouldComponentUpdate`	`memo`
`getDerivedStateFromError`	no hook (class only)

CLASS ESSENTIALS

MEMBER	PURPOSE
`render()`	Return JSX
`this.props`	Props
`this.setState(obj or fn)`	Merge update
`static contextType`	Read one context
`createRef()`	Refs

⚠ COMMON MISTAKES

- Mutating `this.state` directly instead of calling `setState`
- Reading `this.state` right after `setState` and expecting the new value
- Forgetting to bind handlers and getting `this` as `undefined`

✓ TIPS & BEST PRACTICES

- Convert classes to functions when you touch them for other work
- Use `setState(prev => …)` when the next state depends on the previous one
- Keep a class only for error boundaries, or use `react-error-boundary`

QUICK SUMMARY

- `extends Component` + `render()`
- `setState` merges
- Lifecycle methods ↔ `useEffect`
- Still needed for error boundaries

useState

[value, setValue] · initial value · lazy initializer · re-render trigger · multiple state variables · per-instance state

COMPONENT MEMORY

`useState` gives a component a value that survives re-renders. It returns a pair: the **current value** and a **setter**. Calling the setter stores the new value and asks React to re-render the component. Each instance of a component gets its own state, so two `<Counter />`s count independently. Pass a function as the initial value (a **lazy initializer**) when computing it is expensive.

NOTES

- Name the pair `[thing, setThing]` by convention
- The initial value is used only on the first render; later values come from React
- Regular variables reset on every render and don't trigger updates
- Use several `useState` calls for unrelated values

♿ ACCESSIBILITY

- When state toggles visibility, mirror it in ARIA: `aria-expanded`, `aria-pressed`, `aria-checked`

VERSION NOTES

- `useState` arrived in React 16.8 and is unchanged in React 19
- The React Compiler doesn't change how state works

CODE EXPLAINED

- → `useState(0)` sets `count` to `0` on the first render only
- → `setCount(count + 1)` stores a new value and schedules a re-render
- → `aria-expanded={open}` keeps assistive tech in sync with the toggle
- → `useState(() => …)` reads `localStorage` once instead of every render
- → `Broken` changes `n`, but nothing tells React to render again

USESTATE BASICS

```jsx
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}

function Disclosure({ title, children }) {
  const [open, setOpen] = useState(false);
  return (
    <div>
      <button aria-expanded={open}
        onClick={() => setOpen(!open)}>
        {title}
      </button>
      {open && <div>{children}</div>}
    </div>
  );
}

// Lazy initializer: runs once
function Editor() {
  const [draft, setDraft] = useState(
    () => localStorage.getItem('draft') ?? ''
  );
  return <textarea value={draft}
    onChange={(e) => setDraft(e.target.value)} />;
}

// ✗ Regular variable: never updates the UI
function Broken() {
  let n = 0;
  return <button onClick={() => n++}>{n}</button>;
}
```

USESTATE API

PART	MEANING
`useState(initial)`	Declare state
`useState(() => init())`	Lazy initial value
`state`	Value for this render
`setState(next)`	Replace value
`setState(prev => next)`	Update from previous value

STATE OR NOT?

DATA	STORE AS
Changes and shows on screen	state
Computable from props/state	plain variable
Changes but never shown	ref
Never changes	constant outside component
Shared by many components	lifted state or context

⚠ COMMON MISTAKES

- Calling `useState` inside a condition or loop
- Writing `useState(expensive())` so the work runs every render
- Storing values in state that can be computed from other state

✓ TIPS & BEST PRACTICES

- Keep state minimal; derive everything else during render
- Use the updater form when the next value depends on the current one
- Reach for `useReducer` when several values change together

QUICK SUMMARY

- `const [x, setX] = useState(init)`
- Setter → re-render
- Per-instance memory
- Lazy init with a function

State Updates & Batching

state as a snapshot · queued updates · updater function · automatic batching · Object.is bailout · flushSync

STATE IS A SNAPSHOT

Within one render, a state variable is a **fixed snapshot**: calling the setter doesn't change the variable you're reading, it queues a new value for the *next* render. React **batches** every update from the same event into one re-render. When the next value depends on the previous one, pass an **updater function** (`n => n + 1`) so each queued update sees the latest value.

NOTES

- Automatic batching covers handlers, timeouts, promises and native events
- Updater functions run during the next render, in the order they were queued
- If the new value is `Object.is` -equal to the old one, React may skip the re-render
- `flushSync` forces an update to apply immediately (rarely needed)

♿ ACCESSIBILITY

- Use `flushSync` before moving focus to an element that a state update just rendered, or focus it in an effect

VERSION NOTES

- React 18 extended automatic batching beyond React event handlers
- In React 17 and earlier, updates in timeouts and promises were not batched

CODE EXPLAINED

→ `n` is `0` for the whole handler, so three `setN(n + 1)` calls produce `1`

→ Each updater receives the result of the previous one, producing `3`

→ `console.log(n)` prints the snapshot, not the queued value

→ After `await`, the two setters still batch into one render

→ `flushSync` commits the update so the new DOM node exists on the next line

SNAPSHOTS AND UPDATERS

```
function Counter() {
  const [n, setN] = useState(0);

  function addThreeWrong() {
    setN(n + 1);   // 0 + 1
    setN(n + 1);   // 0 + 1
    setN(n + 1);   // 0 + 1  -> 1
  }

  function addThree() {
    setN((prev) => prev + 1);   // 0 -> 1
    setN((prev) => prev + 1);   // 1 -> 2
    setN((prev) => prev + 1);   // 2 -> 3
  }

  function logAfterSet() {
    setN(n + 5);
    console.log(n);    // still the old value
  }

  return <button onClick={addThree}>{n}</button>;
}

// One re-render for both updates (batched)
function Form() {
  const [name, setName] = useState('');
  const [saved, setSaved] = useState(false);
  async function save() {
    await api.save(name);
    setSaved(true);
    setName('');
  }
}

// Apply now, then measure the DOM
import { flushSync } from 'react-dom';
flushSync(() => setItems([...items, next]));
listRef.current.lastChild.scrollIntoView();
```

QUEUE EXAMPLES

CALLS (N = 0)	RESULT
`setN(n+1) ×3`	1
`setN(p => p+1) ×3`	3
`setN(5); setN(p => p+1)`	6
`setN(p => p+1); setN(42)`	42
`setN(0) when n is 0`	may skip render

BATCHING

SOURCE	BATCHED?
`React event handler`	yes
`setTimeout / promise`	yes (18+)
`Native addEventListener`	yes (18+)
`Inside flushSync`	no, applied at once
`Separate events`	separate renders

⚠ COMMON MISTAKES

- Expecting the state variable to change right after calling the setter
- Using `setN(n + 1)` in loops or repeated calls
- Reaching for `flushSync` to fix logic that should use updaters

✓ TIPS & BEST PRACTICES

- Name updater params after the state: `setCount(c => c + 1)`
- Compute the next value once, store it in a variable, then use it for both the setter and any logging
- Read the DOM after updates in an effect, not right after the setter

QUICK SUMMARY

- State = snapshot per render
- Setters queue, React batches
- Updater fn for previous-based updates
- `flushSync` = rare escape hatch

Objects & Arrays in State

immutability · spread copies · nested updates · add/remove/replace items · toSorted() / with() · Immer / use-immer

REPLACE, DON'T MUTATE

React decides whether to re-render by comparing the old and new state references. If you **mutate** an object or array in place, the reference stays the same and React may ignore the change. Always create a **new** object or array: spread for objects, and non-mutating array methods (`map`, `filter`, `toSorted`, `with`) for arrays. **Immer** lets you write mutating code that produces immutable updates.

NOTES

- Spread copies are shallow; nested objects need their own spread
- `[...arr, x]` adds, `filter` removes, `map` replaces
- `toSorted()`, `toReversed()`, `toSpliced()` and `with()` return new arrays (ES2023)
- Immer's `produce` or `useImmer` hook creates the copy for you

♿ ACCESSIBILITY

- Mutation bugs can leave the UI out of sync with `aria-checked` / `aria-selected`; immutable updates keep them accurate

VERSION NOTES

- ES2023 copying array methods are available in all current browsers
- React 19 doesn't change the immutability rule

CODE EXPLAINED

- → `{ ...user, name }` copies the top level and overrides one field
- → Nested data needs a spread at every level you change
- → `map` returns a new array with one item replaced by a new object
- → `toSpliced`, `toSorted` and `with` never touch the original
- → `useImmer` gives a draft you can push into; Immer builds the new state

IMMUTABLE UPDATES

```
const [user, setUser] = useState({
  name: 'Ada',
  address: { city: 'London', zip: 'N1' },
});

// Object field
setUser({ ...user, name: 'Grace' });

// Nested object
setUser({
  ...user,
  address: { ...user.address, city: 'Paris' },
});

const [todos, setTodos] = useState(initial);

// Add / remove / replace
setTodos([...todos, { id: crypto.randomUUID(), text }]);
setTodos(todos.filter((t) => t.id !== id));
setTodos(todos.map((t) =>
  t.id === id ? { ...t, done: !t.done } : t));

// Insert, sort, replace by index (new arrays)
setTodos(todos.toSpliced(1, 0, newTodo));
setTodos(todos.toSorted((a, b) => a.text.localeCompare(b.text)));
setTodos(todos.with(0, { ...todos[0], pinned: true }));

// Immer: write "mutations" on a draft
import { useImmer } from 'use-immer';
const [state, update] = useImmer({ todos: [] });
update((draft) => { draft.todos.push(newTodo); });

// ✗ Mutation: same reference, may not re-render
user.name = 'Grace'; setUser(user);
```

ARRAY OPERATIONS

GOAL	USE
Add	`[...arr, x]` / `[x, ...arr]`
Remove	`filter`
Replace	`map` or `with(i, x)`
Insert	`toSpliced(i, 0, x)`
Sort / reverse	`toSorted` / `toReversed`

AVOID IN STATE

MUTATING	COPYING INSTEAD
`push, unshift`	spread
`splice`	`toSpliced` / `filter`
`sort, reverse`	`toSorted`, `toReversed`
`arr[i] = x`	`with(i, x)`
`obj.key = v`	`{ ...obj, key: v }`

⚠ COMMON MISTAKES

- Sorting state in place with `sort()`
- Spreading only the top level and mutating a nested object
- Calling `setUser(user)` after mutating `user`

✓ TIPS & BEST PRACTICES

- Keep state shallow so updates need fewer spreads
- Use Immer when updates go three or more levels deep
- Store items in arrays with IDs, not by index

QUICK SUMMARY

- New reference = re-render
- Spread objects, copy arrays
- `toSorted` / `with` / `toSpliced`
- Immer for deep updates

Structuring State

derived values · redundant state · duplicated state · flattening nested state · grouping related state · impossible states

CHOOSING THE RIGHT SHAPE

Well-structured state is the smallest set of values from which everything else can be **derived**. Avoid storing data that can be computed during render, avoid **duplicating** the same object in two places, and avoid combinations of booleans that allow **impossible states**. Group values that always change together, and flatten deeply nested data so updates stay simple.

NOTES

- Derived data (counts, filtered lists, full names) is computed in the render body
- Store a selected item's ID, not a copy of the item
- A single `status` string beats `isLoading` + `isError` + `isSuccess`
- Normalize nested trees into a lookup object keyed by ID

♿ ACCESSIBILITY

- A single `status` makes it easy to render one accurate message for screen readers

VERSION NOTES

- These principles come from react.dev's *Choosing the State Structure* guide; unchanged in 19

CODE EXPLAINED

→ `fullName` updates automatically because it's computed each render

→ `status` makes *sending and sent at once* impossible

→ Storing `selectedId` means edits to `items` show up in `selected`

→ `pos` keeps `x` and `y` in one update

→ A flat map lets you update one place without copying the whole tree

BEFORE AND AFTER

```
// ✗ Redundant and contradictory
const [first, setFirst] = useState('');
const [last, setLast] = useState('');
const [fullName, setFullName] = useState('');   // derived
const [isSending, setIsSending] = useState(false);
const [isSent, setIsSent] = useState(false);    // both true?

// ✓ Minimal
const [first, setFirst] = useState('');
const [last, setLast] = useState('');
const fullName = `${first} ${last}`;            // derived
const [status, setStatus] = useState('typing'); // 'sending' | 'sent'

// ✗ Duplicated object
const [items, setItems] = useState(list);
const [selected, setSelected] = useState(list[0]);

// ✓ Store the ID, look it up
const [selectedId, setSelectedId] = useState(list[0].id);
const selected = items.find((i) => i.id === selectedId);

// ✓ Group values that change together
const [pos, setPos] = useState({ x: 0, y: 0 });

// ✓ Flat, normalized tree
const [places, setPlaces] = useState({
  root: { id: 'root', childIds: ['eu', 'as'] },
  eu: { id: 'eu', title: 'Europe', childIds: [] },
});
```

STATE SMELLS

SMELL	FIX
Value computable from others	derive it
Two booleans that conflict	one status string
Same object in two states	store an ID
Deep nesting	normalize by ID
Prop copied into state	use the prop

PRINCIPLES

PRINCIPLE	EXAMPLE
Group related	`{ x, y }`
Avoid contradictions	`status`
Avoid redundancy	`fullName` derived
Avoid duplication	`selectedId`
Avoid deep nesting	`byId` map

⚠ COMMON MISTAKES

- Syncing derived state with an effect
- Initializing state from a prop and expecting it to follow the prop
- Tracking `isOpen` for each item instead of one `openId`

✓ TIPS & BEST PRACTICES

- Ask: *can I compute this from what I already have?*
- Name the prop `initialX` when you really do want a one-time copy
- Draw the state transitions as a small diagram for complex forms

QUICK SUMMARY

- Minimal state, derive the rest
- One status, not many booleans
- IDs, not copies
- Flat beats nested

Sharing State Between Components

lifting state up · single source of truth · controlled vs uncontrolled components · prop drilling · inverse data flow

LIFTING STATE UP

When two components need the same changing data, move the state to their **closest common parent** and pass it down as props, along with callbacks to change it. The parent becomes the **single source of truth**. A component whose important values come from props is **controlled** by its parent; one that keeps them in local state is **uncontrolled**. Lifting too high causes **prop drilling** through layers that don't use the data.

NOTES

- Data flows down as props; changes flow up through callback props
- Name callbacks after what happened: `onChange`, `onSelect`, `onToggle`
- Components can support both modes: `value` + `onChange`, or `defaultValue`
- Fix deep prop drilling with composition first, then context

♿ ACCESSIBILITY

- Lifted state keeps related widgets (accordion panels, radio groups) consistent, so only one reports `aria-expanded="true"` at a time

VERSION NOTES

- The pattern is unchanged; React 19 adds no new API for sharing local state

CODE EXPLAINED

→ `openIndex` lives in `Accordion`, the common parent of both panels

→ Each `Panel` receives `isOpen` instead of owning it

→ `onShow` sends the request back up; the parent decides

→ Only one panel can be open because there is one source of truth

→ The heading-wrapped button follows the WAI-ARIA accordion pattern

ACCORDION WITH LIFTED STATE

```jsx
function Accordion() {
  const [openIndex, setOpenIndex] = useState(0);

  return (
    <>
      <Panel
        title="About"
        isOpen={openIndex === 0}
        onShow={() => setOpenIndex(0)}
      >
        Founded in 2010.
      </Panel>
      <Panel
        title="Contact"
        isOpen={openIndex === 1}
        onShow={() => setOpenIndex(1)}
      >
        hello@example.com
      </Panel>
    </>
  );
}

// Controlled: state lives in the parent
function Panel({ title, isOpen, onShow, children }) {
  return (
    <section>
      <h3>
        <button aria-expanded={isOpen} onClick={onShow}>
          {title}
        </button>
      </h3>
      {isOpen && <div>{children}</div>}
    </section>
  );
}
```

STEPS

STEP	DO
1	Remove state from the children
2	Add it to the common parent
3	Pass value down as a prop
4	Pass a callback to request changes
5	Children call the callback

CONTROLLED VS UNCONTROLLED

CONTROLLED	UNCONTROLLED
`value` + `onChange` props	`defaultValue` prop
Parent owns state	Component owns state
Easy to sync and reset	Less wiring
More re-renders in parent	Isolated updates

⚠ COMMON MISTAKES

- Keeping copies of the same state in sibling components
- Lifting state to the app root when only two siblings need it
- Passing the setter itself instead of an intent-named callback

✓ TIPS & BEST PRACTICES

- Lift only as high as the closest common parent
- Pass JSX as `children` to skip intermediate layers
- Switch to context when the same prop passes through three or more layers that don't use it

QUICK SUMMARY

- Shared state → common parent
- Value down, callback up
- Single source of truth
- Watch for prop drilling

Preserving & Resetting State

position in the tree · same type, same position · key to reset · nested definition bug · state across conditionals

STATE LIVES IN THE TREE

React ties state to a component's **position in the render tree**, not to the JSX variable or function call. If the same component type renders at the same position, its state is **preserved** across re-renders, even with different props. Change the type, or move it to a different position, and the state is **destroyed**. Give a component a different **key** to force a fresh instance with reset state.

NOTES

- Swapping `<Counter />` for `<p>` at a position destroys the counter's state
- `{cond ? <A /> : <A />}` keeps state: same type, same position
- `{cond && <A />}{!cond && <A />}` resets: two different positions
- Defining a component inside another creates a new type every render, resetting its state

♿ ACCESSIBILITY

- Resetting with `key` also resets focus; move focus deliberately after a reset

VERSION NOTES

- React 19.2's `<Activity>` can hide a subtree while preserving its state (see sheet 51)

CODE EXPLAINED

→ The first ternary keeps one `Counter` 's count when switching players

→ With different keys, React treats them as two components; each switch starts at zero

→ `key={contact.id}` clears the draft message when you switch contacts

→ Keys aren't just for lists: any element can take one

→ `Input` is a new function each render, so React remounts it

PRESERVE OR RESET

```jsx
function Scoreboard() {
  const [isPlayerA, setIsPlayerA] = useState(true);

  return (
    <div>
      {/* Same position + type: state is kept */}
      {isPlayerA
        ? <Counter person="Taylor" />
        : <Counter person="Sarah" />}

      {/* Different key: state resets */}
      {isPlayerA
        ? <Counter key="taylor" person="Taylor" />
        : <Counter key="sarah" person="Sarah" />}

      <button onClick={() => setIsPlayerA(!isPlayerA)}>
        Next player
      </button>
    </div>
  );
}

// Reset a form when the selected contact changes
function Messenger({ contact }) {
  return <ChatForm key={contact.id} contact={contact} />;
}

// ✗ Nested definition: new type each render
function Parent() {
  const Input = () => <input />;  // state lost on every render
  return <Input />;
}
```

WHAT HAPPENS

CHANGE	STATE
Same type, same spot, new props	kept
Different type at that spot	reset
Same type, different key	reset
Parent type changes	whole subtree reset
Removed then re-added	reset

USE KEY TO

GOAL	EXAMPLE
Reset a form	`<Form key={userId} />`
Restart an animation	`<Fade key={slide} />`
Separate instances	key="a" / key="b"
Avoid effect-based resets	key instead of useEffect

⚠ COMMON MISTAKES

- Resetting state with an effect that watches a prop
- Wrapping a component in a conditional `<div>` and losing its state
- Declaring components inside other components

✓ TIPS & BEST PRACTICES

- Reach for `key` whenever *new item = fresh state*
- Keep the tree shape stable to keep state
- Lift state up if it must survive the component being removed

QUICK SUMMARY

- State is tied to tree position
- Same type + spot = preserved
- New type or key = reset
- Never nest component definitions

useReducer

REDUCERS FOR COMPLEX UPDATES

`useReducer` moves update logic out of event handlers into a single **reducer** function: `(state, action) => newState`. Handlers **dispatch** actions that describe *what happened*; the reducer decides *how state changes*. This keeps related transitions in one place, makes them easy to test, and suits state where many fields change together.

NOTES

- Reducers must be pure: no fetches, timers or mutation
- Actions are plain objects, usually `{ type: 'added', ...payload }`
- `dispatch` has a stable identity; pass it down freely
- A third argument `init` lazily builds the initial state

♿ ACCESSIBILITY

- Centralized transitions make it easier to also update focus or announcement state in one place

VERSION NOTES

- Unchanged in React 19; pairs with context for app-wide state (sheet 40)
- Immer's `useImmerReducer` lets reducers write draft-style updates

CODE EXPLAINED

→ Each `case` returns a new array; nothing is mutated

→ The `default` throws so typos in action names fail loudly

→ The ID is generated in the handler, keeping the reducer pure

→ Handlers only describe events; the reducer owns the logic

→ `init(arg)` runs once to build the starting state

TODO REDUCER

```
import { useReducer } from 'react';

function todosReducer(todos, action) {
  switch (action.type) {
    case 'added':
      return [...todos, { id: action.id, text: action.text,
        done: false }];
    case 'toggled':
      return todos.map((t) =>
        t.id === action.id ? { ...t, done: !t.done } : t);
    case 'deleted':
      return todos.filter((t) => t.id !== action.id);
    default:
      throw new Error(`Unknown action: ${action.type}`);
  }
}

function TodoApp() {
  const [todos, dispatch] = useReducer(todosReducer, []);

  function handleAdd(text) {
    dispatch({ type: 'added', id: crypto.randomUUID(), text });
  }

  return (
    <>
      <AddTodo onAdd={handleAdd} />
      <TodoList
        todos={todos}
        onToggle={(id) => dispatch({ type: 'toggled', id })}
        onDelete={(id) => dispatch({ type: 'deleted', id })}
      />
    </>
  );
}

// Lazy init: useReducer(reducer, arg, init)
const [s, d] = useReducer(reducer, userId, loadInitial);
```

USEREDUCER API

PART	MEANING
`useReducer(reducer, initial)`	Declare
`useReducer(reducer, arg, init)`	Lazy init
`state`	Current state
`dispatch(action)`	Queue an action
`reducer(state, action)`	Return next state

USESTATE VS USEREDUCER

USESTATE	USEREDUCER
Independent simple values	Related fields
Logic in handlers	Logic in one function
Less code	Easier to test
Hard to trace many setters	Log every action

⚠ COMMON MISTAKES

- Mutating `state` inside the reducer
- Generating IDs or reading `Date.now()` inside the reducer
- Naming actions as commands (`setX`) instead of events (`added`)

✓ TIPS & BEST PRACTICES

- Unit-test reducers as plain functions
- Use a TypeScript discriminated union for actions
- Log `action` in the reducer while debugging

QUICK SUMMARY

- `(state, action) => next`
- `dispatch({ type, ...payload })`
- Pure reducer, events not commands
- Use for related, complex state

Event Handling

RESPONDING TO THE USER

Attach event handlers as props named `onClick`, `onChange`, `onSubmit` and so on. Pass a **function**, not the result of calling one: `onClick={handleClick}`, or an inline arrow when you need arguments. Handlers receive a **synthetic event**, a cross-browser wrapper with the same interface as the native event (`e.target`, `e.preventDefault()`) plus `e.nativeEvent`.

NOTES

- Define handlers inside the component so they can read props and state
- Convention: `handleX` for the function, `onX` for the prop that receives it
- Inline arrows (`() => remove(id)`) are fine; the React Compiler memoizes them
- Custom components only get events you wire up: `<Button onClick>` must forward it

♿ ACCESSIBILITY

- Put `onClick` on a `<button>` or `<a>`, not a `<div>`; native elements give keyboard and screen-reader support for free
- If you must use a non-native element, add `role`, `tabIndex={0}` and key handling

VERSION NOTES

- React 17+ attaches listeners to the root container instead of `document`
- Event pooling was removed in React 17; events can be read asynchronously

CODE EXPLAINED

→ `Toolbar` receives handlers as props and passes them to `Button`

→ The inline arrow lets you pass `'photo'` as an argument

→ `e.type` and `e.target` work just like native events

→ `handlePlay()` with parentheses runs during render, not on click

→ A `<div>` can't be focused or activated with Enter/Space

HANDLERS AND CALLBACK PROPS

```jsx
function Toolbar({ onPlay, onUpload }) {
  return (
    <div>
      <Button onClick={onPlay}>Play</Button>
      <Button onClick={() => onUpload('photo')}>Upload</Button>
    </div>
  );
}

function Button({ onClick, children }) {
  return <button onClick={onClick}>{children}</button>;
}

function App() {
  const [log, setLog] = useState([]);

  function handlePlay(e) {
    console.log(e.type, e.target.tagName);   // click BUTTON
    setLog((l) => [...l, 'play']);
  }

  return <Toolbar onPlay={handlePlay}
    onUpload={(kind) => setLog((l) => [...l, kind])} />;
}

// ✗ Calls immediately during render
<button onClick={handlePlay()}>Play</button>

// ✗ Clickable div: no keyboard, no role
<div onClick={handlePlay}>Play</div>
```

COMMON EVENTS

PROP	FIRES ON
`onClick`	click, Enter/Space on buttons
`onChange`	every input change
`onSubmit`	form submit
`onInput`	native input event
`onKeyDown`	key press
`onFocus / onBlur`	focus in / out (bubbles in React)

SYNTHETIC EVENT

MEMBER	USE
`e.target`	element that triggered
`e.currentTarget`	element with the handler
`e.preventDefault()`	stop default action
`e.stopPropagation()`	stop bubbling
`e.nativeEvent`	underlying DOM event

⚠ COMMON MISTAKES

- `onClick={doThing()}` instead of `onClick={doThing}`
- Lowercase `onclick` (ignored with a warning)
- Attaching click handlers to non-interactive elements

✓ TIPS & BEST PRACTICES

- Name props by meaning (`onPlayMovie`) rather than DOM event names when they represent app actions
- Keep handler logic short; call named functions for real work
- Use `<button type="button">` inside forms to avoid accidental submits

QUICK SUMMARY

- `onEvent={fn}`, not `fn()`
- `handleX` → `onX`
- Synthetic events mirror native
- Buttons, not divs

Event Propagation

BUBBLING, CAPTURE AND DEFAULTS

Most React events **bubble**: a click on a button also runs `onClick` handlers on its ancestors, innermost first. Call `e.stopPropagation()` to stop it. Add `Capture` to a prop name (`onClickCapture`) to run during the **capture phase**, before children. `e.preventDefault()` stops the browser's default action (submitting a form, following a link) but doesn't stop bubbling.

NOTES

- `onScroll` is the main React event that doesn't bubble
- `e.target` is where the event started; `e.currentTarget` is the element whose handler is running
- React delegates listeners to the root container
- Native listeners added with `addEventListener` run separately from React's

♿ ACCESSIBILITY

- Stopping propagation can break shortcut handlers and focus traps higher up; stop it only where needed
- Don't `preventDefault` on keyboard events that users rely on (Tab, Escape) unless you replace the behavior

VERSION NOTES

- React 17 moved delegation from `document` to the root, which made nested React apps and native listeners interoperate

CODE EXPLAINED

→ Without `stopPropagation`, clicking Delete would also call `onOpen`

→ `preventDefault` stops the full-page form submission

→ `e.currentTarget` is the `<form>`, so `FormData` reads all its fields

→ `onClickCapture` sees every click inside before children handle it

→ Delegating on `<ul>` and reading `e.target` handles many items with one listener

PROPAGATION IN PRACTICE

```
function Card({ onOpen, onDelete }) {
  return (
    <article onClick={onOpen} className="card">
      <h3>Report</h3>
      <button
        onClick={(e) => {
          e.stopPropagation();      // don't also open the card
          onDelete();
        }}
      >
        Delete
      </button>
    </article>
  );
}

function SignupForm({ onSignup }) {
  return (
    <form
      onSubmit={(e) => {
        e.preventDefault();         // no page reload
        onSignup(new FormData(e.currentTarget));
      }}
    >
      <input name="email" type="email" required />
      <button>Sign up</button>
    </form>
  );
}

// Capture phase runs first, top-down
<div onClickCapture={() => track('click')}>
  <button onClick={save}>Save</button>
</div>

// target vs currentTarget
<ul onClick={(e) => {
  console.log(e.target);          // the <li> clicked
  console.log(e.currentTarget);   // the <ul>
}}>
```

PHASES

PROP	ORDER
`onClickCapture on parent`	1st
`onClickCapture on child`	2nd
`onClick on child`	3rd
`onClick on parent`	4th

STOP WHAT?

CALL	EFFECT
`e.stopPropagation()`	no more handlers up the tree
`e.preventDefault()`	no browser default action
`both`	neither
`return false`	nothing (not supported)

⚠ COMMON MISTAKES

- Returning `false` from a handler to prevent defaults
- Using `e.target` when you mean the element with the handler
- Stopping propagation everywhere and breaking analytics or menus

✓ TIPS & BEST PRACTICES

- Prefer passing a callback and letting the child decide over relying on bubbling
- Use capture handlers for logging, not for app logic
- Add native listeners in effects and remove them in cleanup

QUICK SUMMARY

- Events bubble child → parent
- `stopPropagation` stops bubbling
- `preventDefault` stops browser action
- `*Capture` runs first

Keyboard, Pointer & Focus Events

onKeyDown · e.key · onPointerDown · onFocus / onBlur · onChange per keystroke · onScroll · passive listeners

KEYS, POINTERS AND FOCUS

Keyboard events (`onKeyDown`, `onKeyUp`) report `e.key` (`'Enter'`, `'Escape'`, `'ArrowDown'`) plus modifier flags. **Pointer events** (`onPointerDown`, `onPointerMove`) unify mouse, pen and touch. React's `onFocus` and `onBlur` **bubble** like the DOM's `focusin`/`focusout`, so a container can track focus inside it. `onChange` fires on **every keystroke**, unlike the DOM's `change` event.

NOTES

- Use `e.key`, not the deprecated `e.keyCode`
- `e.relatedTarget` on blur tells you where focus went
- Wheel and touch listeners attached by React are **passive**; call `preventDefault` from a native non-passive listener
- `onScroll` doesn't bubble in React

♿ ACCESSIBILITY

- Every pointer interaction needs a keyboard equivalent
- Close popups on Escape and return focus to the trigger
- Don't trap focus unintentionally with `onBlur` logic that refocuses

VERSION NOTES

- React 17 stopped `onScroll` from bubbling and made `onFocus` / `onBlur` use `focusin` / `focusout`
- Pointer events have been supported since React 16.4

CODE EXPLAINED

→ `e.key` compares readable names; check `metaKey` or `ctrlKey` for shortcuts

→ `setPointerCapture` keeps move events flowing even if the pointer leaves the element

→ `e.buttons === 1` means the primary button is held

→ `onBlur` bubbles, so the container hears when any child loses focus

→ `relatedTarget` outside the menu means focus left; dismiss it

KEYBOARD, POINTER, FOCUS

```
function SearchBox({ onSearch, onClose }) {
  return (
    <input
      type="search"
      onKeyDown={(e) => {
        if (e.key === 'Enter') onSearch(e.currentTarget.value);
        if (e.key === 'Escape') onClose();
        if (e.key === 'k' && (e.metaKey || e.ctrlKey)) {
          e.preventDefault();
        }
      }}
    />
  );
}

function Draggable() {
  const [pos, setPos] = useState({ x: 0, y: 0 });
  return (
    <div
      className="handle"
      onPointerDown={(e) =>
        e.currentTarget.setPointerCapture(e.pointerId)}
      onPointerMove={(e) => {
        if (e.buttons !== 1) return;
        setPos((p) => ({ x: p.x + e.movementX,
          y: p.y + e.movementY }));
      }}
      style={{ translate: `${pos.x}px ${pos.y}px` }}
    />
  );
}

function Menu({ onDismiss, children }) {
  return (
    <div
      onBlur={(e) => {
        if (!e.currentTarget.contains(e.relatedTarget)) onDismiss();
      }}
    >{children}</div>
  );
}
```

KEYBOARD

PROPERTY	VALUE
`e.key`	`'a'`, `'Enter'`, `'ArrowUp'`
`e.code`	physical key: `'KeyA'`
`e.shiftKey / altKey`	modifiers
`e.metaKey / ctrlKey`	⌘ / Ctrl
`e.repeat`	key held down

REACT VS DOM

REACT	DOM EQUIVALENT
`onChange (inputs)`	`input` event
`onFocus / onBlur`	`focusin` / `focusout`
`onDoubleClick`	`dblclick`
`onMouseEnter`	`mouseenter` (no bubbling)
`onScroll`	`scroll` (no bubbling)

⚠ COMMON MISTAKES

- Using `keyCode` or `which`
- Expecting `onChange` to wait until the input loses focus
- Calling `preventDefault` in React's passive `onWheel` / `onTouchMove`

✓ TIPS & BEST PRACTICES

- Use `e.code` for game-style controls that depend on key position
- Prefer `onPointer*` over separate mouse and touch handlers
- Debounce expensive work triggered by `onChange` or `onPointerMove`

QUICK SUMMARY

- `e.key` for keys
- Pointer events = mouse + touch + pen
- Focus events bubble in React
- `onChange` = every keystroke

Controlled Inputs

value + onChange · checked · <select> · multiple select · <textarea> · number inputs as strings

REACT OWNS THE VALUE

A **controlled input** gets its value from state and reports every change through `onChange`. React state becomes the single source of truth, so you can validate, format or disable as the user types. Text inputs, textareas and selects use `value`; checkboxes and radios use `checked`. Passing `value` without `onChange` makes the field read-only.

NOTES

- `e.target.value` is always a string, even for `type="number"`
- `<textarea value>` replaces the HTML text-content form
- `<select value>` replaces `selected` on options; `multiple` takes an array
- Switching between `undefined` and a defined `value` triggers a controlled/uncontrolled warning

♿ ACCESSIBILITY

- Every input needs a `<label>`; `htmlFor` must match the input's `id`
- Don't reformat input while the user types in a way that moves the caret

VERSION NOTES

- Controlled inputs work the same in React 19
- For simple submit-only forms, React 19 Actions (sheet 28) can replace controlled state

CODE EXPLAINED

→ One `update` handler serves many fields by reading `name`

→ `[name]: …` is a computed key that sets the matching field

→ Checkboxes read `checked`; everything else reads `value`

→ `<select value>` marks the matching option as selected

→ `selectedOptions` collects all chosen values for a multi-select

CONTROLLED FIELDS

```
function Signup() {
  const [form, setForm] = useState({
    name: '', bio: '', plan: 'free', agree: false, tags: [],
  });

  function update(e) {
    const { name, value, type, checked } = e.target;
    setForm((f) => ({
      ...f, [name]: type === 'checkbox' ? checked : value,
    }));
  }

  return (
    <form>
      <label htmlFor="name">Name</label>
      <input id="name" name="name"
        value={form.name} onChange={update} />

      <textarea name="bio" value={form.bio} onChange={update} />

      <select name="plan" value={form.plan} onChange={update}>
        <option value="free">Free</option>
        <option value="pro">Pro</option>
      </select>

      <label>
        <input type="checkbox" name="agree"
          checked={form.agree} onChange={update} />
        I agree
      </label>

      <select multiple value={form.tags} onChange={(e) =>
        setForm({ ...form, tags: [...e.target.selectedOptions]
          .map((o) => o.value) })}>
        <option>react</option><option>css</option>
      </select>
    </form>
  );
}
```

CONTROLLED PROPS

ELEMENT	VALUE PROP
`<input type=text>`	`value`
`<input type=checkbox>`	`checked`
`<input type=radio>`	`checked`
`<textarea>`	`value`
`<select>`	`value` (array if `multiple`)
`<input type=file>`	none: always uncontrolled

NUMBER INPUTS

NEED	DO
Read a number	`e.target.valueAsNumber`
Allow empty	keep string state, convert on submit
Store as number	`Number(value)` + handle `NaN`
Limit range	`min`, `max`, `step`

⚠ COMMON MISTAKES

- `value` without `onChange` (field won't change)
- Initializing `value` as `undefined` then setting a string
- Storing numbers and losing the user's partially typed `1.` input

✓ TIPS & BEST PRACTICES

- Use `name` attributes so one handler covers the form
- Use `defaultValue` when you don't need the value until submit
- Use a form library (sheet 30) for large forms

QUICK SUMMARY

- `value` / `checked` + `onChange`
- State is the source of truth
- Inputs give strings
- File inputs are never controlled

Uncontrolled Inputs

defaultValue · defaultChecked · FormData · useRef · file inputs · form reset

THE DOM OWNS THE VALUE

An **uncontrolled input** keeps its own value in the DOM. Set a starting value with `defaultValue` or `defaultChecked`, then read the values when you need them: with **`FormData`** on submit, or with a **ref**. This means less code and fewer re-renders, and it's how React 19's form Actions work. File inputs are always uncontrolled.

NOTES

- `new FormData(form)` reads every named field at once
- `formData.get('x')` returns one value; `getAll('x')` returns all (checkbox groups)
- Unchecked checkboxes are missing from `FormData`
- `form.reset()` restores every field to its default value

♿ ACCESSIBILITY

- Native constraint attributes (`required` , `type="email"`) give built-in validation messages
- Uncontrolled forms still need labels and error text tied with `aria-describedby`

VERSION NOTES

- React 19 resets uncontrolled forms automatically after a successful form Action
- `defaultValue` updates after mount are ignored; change the `key` to reset

CODE EXPLAINED

→ `FormData` collects all named fields when the form submits

→ `getAll('topic')` returns every checked `topic` value

→ `data.has('subscribe')` is `true` only when the box is checked

→ `reset()` puts every field back to its default

→ The file input is read through a ref because its value can't be set

FORMDATA AND REFS

```jsx
function Feedback() {
  function handleSubmit(e) {
    e.preventDefault();
    const data = new FormData(e.currentTarget);
    const payload = {
      email: data.get('email'),
      rating: Number(data.get('rating')),
      topics: data.getAll('topic'),
      subscribe: data.has('subscribe'),
    };
    send(payload);
    e.currentTarget.reset();
  }

  return (
    <form onSubmit={handleSubmit}>
      <input name="email" type="email" required
        defaultValue="me@example.com" />
      <input name="rating" type="range" min="1" max="5"
        defaultValue="3" />
      <label><input type="checkbox" name="topic"
        value="ui" defaultChecked /> UI</label>
      <label><input type="checkbox" name="topic"
        value="api" /> API</label>
      <label><input type="checkbox" name="subscribe" />
        Newsletter</label>
      <button>Send</button>
    </form>
  );
}

function Upload() {
  const fileRef = useRef(null);
  const upload = () => {
    const [file] = fileRef.current.files;
    if (file) api.upload(file);
  };
  return <>
    <input type="file" ref={fileRef} accept="image/*" />
    <button type="button" onClick={upload}>Upload</button>
  </>;
}
```

FORMDATA METHODS

METHOD	RETURNS
`get(name)`	first value or `null`
`getAll(name)`	array of values
`has(name)`	boolean
`Object.fromEntries(fd)`	plain object (last value wins)
`fd.entries()`	iterator of pairs

CONTROLLED VS UNCONTROLLED

NEED	CHOOSE
`Validate while typing`	controlled
`Live preview of value`	controlled
`Read only on submit`	uncontrolled
`Form Actions (19)`	uncontrolled
`File uploads`	uncontrolled

⚠ COMMON MISTAKES

- Changing `defaultValue` and expecting the field to update
- Forgetting `name` attributes, so `FormData` misses fields
- Using `Object.fromEntries` on multi-value fields and losing values

✓ TIPS & BEST PRACTICES

- Start uncontrolled; switch a field to controlled only when you need live logic
- Use `key` on the form to reset it to new defaults
- Pair with Zod to parse `FormData` into typed data (sheet 31)

QUICK SUMMARY

- `defaultValue` / `defaultChecked`
- Read with `FormData` or refs
- Fewer re-renders
- Files are always uncontrolled

Form Actions

FORMS THAT CALL FUNCTIONS

In React 19, a `<form>`'s `action` prop can be a **function**. When the form submits, React calls it with the form's `FormData`, wraps it in a **transition** (so the UI stays responsive), and, for uncontrolled fields, **resets the form** after it succeeds. Actions can be `async`. Individual submit buttons can override the action with `formAction`.

NOTES

- No `preventDefault` needed: React handles the submit
- Async actions run inside a transition; pending state is available via hooks
- Errors thrown in an action go to the nearest error boundary
- `requestFormReset(form)` from `react-dom` resets a form manually

♿ ACCESSIBILITY

- Disable the submit button and show progress while pending (sheet 29)
- After a successful action, move focus or announce the result

VERSION NOTES

- Function `action` / `formAction` props are new in React 19
- With a framework, the function can be a **Server Function** (`'use server'` , sheet 87)

CODE EXPLAINED

→ `action={subscribe}` receives `FormData` ; no submit handler needed

→ React awaits the async function inside a transition

→ Fields reset after success because they are uncontrolled

→ `formAction` on the second button runs `saveDraft` instead of `publish`

→ `requestFormReset` must be called inside a transition or action

FORM ACTIONS

```jsx
function NewsletterForm() {
  async function subscribe(formData) {
    const email = formData.get('email');
    await api.subscribe(email);      // async is fine
  }

  return (
    <form action={subscribe}>
      <input name="email" type="email" required />
      <button>Subscribe</button>
    </form>
  );
}

function PostEditor({ post }) {
  async function publish(formData) {
    await api.save({ ...post, body: formData.get('body'),
      status: 'published' });
  }
  async function saveDraft(formData) {
    await api.save({ ...post, body: formData.get('body') });
  }

  return (
    <form action={publish}>
      <textarea name="body" defaultValue={post.body} />
      <button>Publish</button>
      <button formAction={saveDraft}>Save draft</button>
    </form>
  );
}

// Manual reset from inside a transition
import { requestFormReset } from 'react-dom';
startTransition(() => requestFormReset(formEl));
```

ACTION PROP VALUES

VALUE	BEHAVIOR
`URL string`	normal HTML form post
`Function`	called with `FormData`
`Async function`	runs in a transition
`Server Function`	posts to the server
`formAction on button`	per-button override

ACTION LIFECYCLE

STEP	WHAT HAPPENS
`Submit`	React prevents navigation
`Start`	transition begins, `pending` true
`Await`	your async function runs
`Success`	uncontrolled fields reset
`Error`	nearest error boundary

⚠ COMMON MISTAKES

- Calling `e.preventDefault()` inside an action (there's no event)
- Controlling every field with state and losing the auto-reset
- Expecting `action` to receive the event instead of `FormData`

✓ TIPS & BEST PRACTICES

- Return validation errors with `useActionState` instead of throwing
- Keep actions small: parse, call the API, return a result
- Use `formAction` for secondary buttons like *Save draft*

QUICK SUMMARY

- `<form action={fn}>` gets `FormData`
- Runs as a transition
- Auto-reset on success
- `formAction` per button

useActionState & useFormStatus

[state, formAction, isPending] · previous state · returning errors · useFormStatus · pending · permalink

RESULTS AND PENDING UI

`useActionState` wraps an action and keeps its **result** in state. Your function receives the **previous state** and the `FormData`, and whatever it returns becomes the new state, which is perfect for validation errors or success messages. It also returns `isPending`. `useFormStatus`, from `react-dom`, lets any component rendered **inside** a form read whether that form is submitting.

NOTES

- Signature: `const [state, formAction, isPending] = useActionState(fn, initial)`
- Pass `formAction` to `<form action>` or `<button formAction>`
- `useFormStatus()` returns `{ pending, data, method, action }`
- `useFormStatus` must be called in a child of the `<form>`, not the component that renders it

♿ ACCESSIBILITY

- Render returned errors in an element with `role="alert"` or link them with `aria-describedby`
- Use `aria-disabled` or `disabled` on the submit button while pending, and show visible progress text

VERSION NOTES

- Both hooks are new in React 19; `useActionState` replaced the canary-era `useFormState`
- The optional third argument `permalink` supports progressive enhancement with Server Functions

CODE EXPLAINED

→ The action returns error state instead of throwing

→ `defaultValue={state.email}` restores the input after the auto-reset

→ `aria-invalid` and `aria-describedby` tie the error to the field

→ `SubmitButton` reads the parent form's status with `useFormStatus`

→ `isPending` is also available in `SignUpForm` if needed

ERRORS AND PENDING STATE

```
import { useActionState } from 'react';
import { useFormStatus } from 'react-dom';

async function signUp(prevState, formData) {
  const email = String(formData.get('email'));
  if (!email.includes('@')) {
    return { error: 'Enter a valid email', email };
  }
  const res = await api.signUp(email);
  if (!res.ok) return { error: res.message, email };
  return { success: true };
}

function SignUpForm() {
  const [state, formAction, isPending] =
    useActionState(signUp, { error: null });

  if (state.success) return <p role="status">Check your inbox!</p>;

  return (
    <form action={formAction}>
      <label htmlFor="email">Email</label>
      <input id="email" name="email" defaultValue={state.email}
        aria-invalid={!!state.error}
        aria-describedby="email-error" />
      {state.error &&
        <p id="email-error" role="alert">{state.error}</p>}
      <SubmitButton />
    </form>
  );
}

function SubmitButton() {
  const { pending } = useFormStatus();
  return <button disabled={pending}>
    {pending ? 'Signing up…' : 'Sign up'}
  </button>;
}
```

USEACTIONSTATE

PART	MEANING
`fn(prevState, formData)`	your action
`initialState`	state before first submit
`state`	last returned value
`formAction`	pass to form or button
`isPending`	true while running

USEFORMSTATUS

FIELD	MEANING
`pending`	parent form is submitting
`data`	`FormData` being sent
`method`	`'get'` or `'post'`
`action`	function or URL being called

⚠ COMMON MISTAKES

- Calling `useFormStatus` in the same component that renders `<form>`
- Forgetting the `prevState` parameter and treating it as `FormData`
- Throwing for validation errors and hitting the error boundary

✓ TIPS & BEST PRACTICES

- Return the submitted values so fields can be refilled after reset
- Build one reusable `<SubmitButton>` with `useFormStatus`
- Keep the action function outside the component so it stays pure and testable

QUICK SUMMARY

- `useActionState(fn, init)`
- → `[state, formAction, isPending]`
- `useFormStatus()` in a form child
- Return errors, don't throw

React Hook Form

useForm · register · handleSubmit · formState.errors · Controller · watch · reset

PERFORMANT FORM STATE

React Hook Form keeps field values in uncontrolled inputs and refs, so typing doesn't re-render the whole form. `useForm()` returns `register` to connect native inputs, `handleSubmit` to validate and call your submit function, and `formState` for errors and status. Use `Controller` for third-party components that only work as controlled inputs.

NOTES

- `register('name', rules)` returns `name`, `ref`, `onChange` and `onBlur` props
- Built-in rules: `required`, `min`, `max`, `minLength`, `maxLength`, `pattern`, `validate`
- `mode: 'onBlur'` or `'onChange'` changes when validation runs (default `'onSubmit'`)
- `watch` subscribes to values and re-renders; `getValues` reads without subscribing

♿ ACCESSIBILITY

- Set `aria-invalid` from `errors` and link messages with `aria-describedby`
- `setFocus` or the default `shouldFocusError` moves focus to the first invalid field

VERSION NOTES

- React Hook Form v7 is current and supports React 19
- Also works alongside React 19 Actions by calling `handleSubmit` from `onSubmit`

CODE EXPLAINED

→ `defaultValues` sets initial values and enables `isDirty`

→ `{...register('name', rules)}` wires the input without state

→ `errors.name.message` holds the message from the rule

→ `valueAsNumber` stores `age` as a number

→ `Controller` adapts a controlled-only component; `reset(data)` clears dirty state after save

USEFORM, REGISTER, CONTROLLER

```jsx
import { useForm, Controller } from 'react-hook-form';
import DatePicker from 'some-date-picker';

function ProfileForm({ onSave }) {
  const {
    register, handleSubmit, control, watch, reset,
    formState: { errors, isSubmitting, isDirty },
  } = useForm({
    defaultValues: { name: '', age: 18, birthday: null },
    mode: 'onBlur',
  });

  const name = watch('name');

  return (
    <form onSubmit={handleSubmit(async (data) => {
      await onSave(data);
      reset(data);
    })}>
      <input {...register('name', { required: 'Name is required',
        minLength: { value: 2, message: 'Too short' } })}
        aria-invalid={!!errors.name} />
      {errors.name && <p role="alert">{errors.name.message}</p>}

      <input type="number"
        {...register('age', { valueAsNumber: true, min: 13 })} />

      <Controller name="birthday" control={control}
        render={({ field }) => <DatePicker {...field} />} />

      <p>Hello {name || 'stranger'}</p>
      <button disabled={isSubmitting || !isDirty}>Save</button>
    </form>
  );
}
```

USEFORM RETURNS

MEMBER	USE
`register`	connect native inputs
`handleSubmit(fn)`	validate then call `fn`
`formState.errors`	per-field errors
`watch(name)`	live value (re-renders)
`reset(values)`	reset values and state
`setValue / setError`	set programmatically

FORMSTATE FLAGS

FLAG	TRUE WHEN
`isDirty`	any field changed
`isValid`	no errors (needs mode)
`isSubmitting`	submit handler running
`isSubmitSuccessful`	last submit succeeded
`touchedFields`	fields blurred

⚠ COMMON MISTAKES

- Passing `register` result to a component that doesn't forward `ref`
- Calling `watch()` with no arguments in a large form (re-renders everything)
- Mixing controlled `value` with `register` on the same input

✓ TIPS & BEST PRACTICES

- Use a schema resolver (sheet 31) instead of inline rules for big forms
- Use `useWatch` in a child to isolate re-renders
- Disable submit with `isSubmitting` to prevent double posts

QUICK SUMMARY

- `useForm({ defaultValues })`
- `{...register(name, rules)}`
- `handleSubmit(onValid)`
- `Controller` for custom inputs

Form Validation

constraint validation · Zod schemas · safeParse · zodResolver · field errors · aria-invalid / aria-describedby

VALIDATE ONCE, EVERYWHERE

Start with **native constraint validation** (`required`, `type`, `pattern`, `min`) for free browser checks. For real rules, define a **Zod schema** once and use it everywhere: `safeParse` returns either typed data or a list of field errors. With React Hook Form, `zodResolver` plugs the schema straight into `useForm`. Always validate again on the server.

NOTES

- `schema.safeParse(data)` never throws; check `result.success`
- `z.infer<typeof schema>` gives the TypeScript type for free
- `z.coerce.number()` converts form strings to numbers
- `noValidate` on `<form>` turns off browser bubbles when you render your own messages

♿ ACCESSIBILITY

- Set `aria-invalid="true"` on invalid fields and connect the message with `aria-describedby`
- Show errors as text, not only red borders
- Move focus to the first invalid field on submit

VERSION NOTES

- Zod 4 is current; `z.email()` and other top-level string formats are new in v4 (`z.string().email()` still works)
- `@hookform/resolvers/zod` supports Zod 4

CODE EXPLAINED

→ The schema is the single definition of what valid data looks like

→ `z.coerce.number()` turns the `'17'` string from `FormData` into `17`

→ `z.flattenError(...).fieldErrors` maps each field to its messages

→ `zodResolver` runs the same schema inside React Hook Form

→ `noValidate` hides native bubbles so only your messages show

ONE SCHEMA, TWO USES

```tsx
import { z } from 'zod';
import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';

const Signup = z.object({
  email: z.email('Enter a valid email'),
  age: z.coerce.number().int().min(13, 'Must be 13+'),
  password: z.string().min(8, 'At least 8 characters'),
});
type SignupData = z.infer<typeof Signup>;

// 1) With a React 19 action
async function signUp(prev, formData) {
  const result = Signup.safeParse(Object.fromEntries(formData));
  if (!result.success) {
    return { errors: z.flattenError(result.error).fieldErrors };
  }
  await api.signUp(result.data);
  return { errors: {} };
}

// 2) With React Hook Form
function SignupForm() {
  const { register, handleSubmit, formState: { errors } } =
    useForm<SignupData>({ resolver: zodResolver(Signup) });
  return (
    <form noValidate onSubmit={handleSubmit(api.signUp)}>
      <input {...register('email')}
        aria-invalid={!!errors.email}
        aria-describedby="email-err" />
      <p id="email-err">{errors.email?.message}</p>
    </form>
  );
}
```

NATIVE CONSTRAINTS

ATTRIBUTE	CHECKS
`required`	not empty
`type="email" / "url"`	format
`minLength / maxLength`	text length
`min / max / step`	numbers, dates
`pattern`	regex match

ZOD BASICS

SCHEMA	VALIDATES
`z.string().min(1)`	non-empty string
`z.email()`	email format
`z.coerce.number()`	numeric string → number
`z.enum(['a','b'])`	one of a set
`.optional() / .nullable()`	may be missing / null
`.refine(fn, msg)`	custom rule

⚠ COMMON MISTAKES

- Validating only on the client
- Using `parse` (throws) where `safeParse` fits better
- Showing all errors before the user has touched a field

✓ TIPS & BEST PRACTICES

- Share schemas between client and server code
- Validate on blur, then on change once a field has an error
- Keep messages specific: say what to fix

QUICK SUMMARY

- Native constraints first
- Zod schema = single source
- `safeParse` → data or errors
- `zodResolver` for RHF

useRef

ref.current · no re-render · timer IDs · previous values · refs vs state · no reads during render

VALUES THAT DON'T RE-RENDER

`useRef(initial)` returns a mutable object, `{ current: initial }`, that lives for the whole life of the component. Changing `ref.current` does **not** trigger a re-render, which makes refs right for values the UI doesn't display: timer IDs, the previous value of a prop, a flag that an effect already ran, or a DOM node. If the screen should change when the value changes, use state.

NOTES

- The same ref object is returned on every render
- Read and write `ref.current` in handlers and effects, not during render
- Lazy init: `if (ref.current === null) ref.current = create()` is the one allowed render-time write
- Refs are per component instance, unlike module variables

♿ ACCESSIBILITY

- Refs are how you move focus programmatically (sheet 33)

VERSION NOTES

- React 19 requires an argument: `useRef()` with no initial value is a TypeScript error in `@types/react` 19
- Under the React Compiler, reading `ref.current` during render is flagged

CODE EXPLAINED

→ `intervalRef` keeps the interval ID between renders without displaying it

→ Writing `intervalRef.current` in a handler causes no re-render

→ `now` and `start` are state because the output shows them

→ `clearInterval` before starting avoids two intervals running

→ The `null` check creates the `Player` once instead of every render

REFS AS INSTANCE VARIABLES

```jsx
import { useRef, useState } from 'react';

function Stopwatch() {
  const [start, setStart] = useState(null);
  const [now, setNow] = useState(null);
  const intervalRef = useRef(null);

  function handleStart() {
    setStart(Date.now());
    setNow(Date.now());
    clearInterval(intervalRef.current);
    intervalRef.current = setInterval(() => {
      setNow(Date.now());
    }, 10);
  }

  function handleStop() {
    clearInterval(intervalRef.current);
  }

  const seconds = start && now ? (now - start) / 1000 : 0;
  return (
    <>
      <output>{seconds.toFixed(2)}s</output>
      <button onClick={handleStart}>Start</button>
      <button onClick={handleStop}>Stop</button>
    </>
  );
}

// Lazy, one-time creation
function Video() {
  const player = useRef(null);
  if (player.current === null) player.current = new Player();
}
```

REF VS STATE

REF	STATE
`ref.current = x`	`setX(x)`
`No re-render`	Re-renders
`Mutable`	Immutable snapshot
`Read in handlers/effects`	Read anywhere in render
`Timer IDs, DOM nodes`	Displayed data

GOOD USES

STORE	WHY
`Timeout / interval IDs`	clear later
`DOM elements`	focus, measure
`Previous value`	compare in effects
`Latest callback`	avoid re-subscribing
`Third-party instances`	create once

⚠ COMMON MISTAKES

- Displaying `ref.current` and expecting the UI to update
- Reading or writing refs during render
- Using a module-level variable shared by every instance

✓ TIPS & BEST PRACTICES

- Think of refs as an escape hatch; most data belongs in state
- Name refs with a `Ref` suffix
- Wrap ref patterns in custom hooks (`useInterval` , `usePrevious`)

QUICK SUMMARY

- `useRef(init)` → `{ current }`
- Mutating doesn't re-render
- Use in handlers and effects
- State if it's shown on screen

Refs to DOM Elements

ref attribute · focus() / scrollIntoView() · ref as a prop · forwardRef (legacy) · ref callback cleanup · useImperativeHandle

REACHING THE DOM

Pass a ref object to a JSX element's `ref` attribute and React sets `ref.current` to the DOM node after mounting, and back to `null` on unmount. Use it for focus, scrolling, measuring, and third-party DOM libraries. In React 19, **`ref` is a regular prop** for function components, so a child can forward it to its own element without `forwardRef`. **Ref callbacks** can now return a cleanup function.

NOTES

- DOM refs are available in effects and handlers, not during the first render
- Ref callback: `ref={(node) => { …; return () => cleanup }}`
- `useImperativeHandle(ref, () => ({ focus }))` exposes a limited API instead of the node
- For lists, store nodes in a `Map` via ref callbacks

♿ ACCESSIBILITY

- Move focus to dialogs, error summaries and new content with `ref.current.focus()`
- Return focus to the triggering button when a dialog closes
- Don't steal focus on page load unless the page has a single obvious task

VERSION NOTES

- React 19: `ref` as a prop; `forwardRef` still works but will be deprecated
- React 19: ref callbacks may return a cleanup function
- String refs (`ref="input"`) were removed in React 19

CODE EXPLAINED

→ `TextInput` destructures `ref` like any prop and attaches it to `<input>`

→ The parent's `inputRef.current` becomes the `<input>` element

→ The ref callback's returned function runs when the node unmounts

→ `useImperativeHandle` hands parents an object with only `play` and `pause`

→ `forwardRef` is no longer needed for new components

DOM REFS IN REACT 19

```jsx
import { useRef, useImperativeHandle } from 'react';

// ref is just a prop now
function TextInput({ label, ref, ...props }) {
  return (
    <label>
      {label}
      <input ref={ref} {...props} />
    </label>
  );
}

function SearchPage() {
  const inputRef = useRef(null);
  return (
    <>
      <TextInput label="Search" ref={inputRef} />
      <button onClick={() => inputRef.current.focus()}>
        Focus search
      </button>
    </>
  );
}

// Ref callback with cleanup
function Measured() {
  return (
    <div ref={(node) => {
      const ro = new ResizeObserver(() => console.log(node.offsetWidth));
      ro.observe(node);
      return () => ro.disconnect();
    }} />
  );
}

// Expose only selected methods
function VideoPlayer({ ref, src }) {
  const video = useRef(null);
  useImperativeHandle(ref, () => ({
    play: () => video.current.play(),
    pause: () => video.current.pause(),
  }));
  return <video ref={video} src={src} />;
}

// Legacy: const TextInput = forwardRef((props, ref) => …)
```

REF FORMS

FORM	USE
`ref={refObject}`	Keep one node
`ref={node => …}`	Run code on attach
`Callback returning fn`	Cleanup on detach (19)
`ref prop on component`	Forward to a child (19)
`useImperativeHandle`	Custom handle

DOM TASKS

TASK	CALL
Focus	`el.focus()`
Scroll into view	`el.scrollIntoView({ block: 'nearest' })`
Measure	`el.getBoundingClientRect()`
Select text	`el.select()`
Play media	`el.play()`

⚠ COMMON MISTAKES

- Reading `ref.current` during render (it's `null` on first render)
- Changing DOM content that React manages (React will overwrite it)
- Wrapping new components in `forwardRef` out of habit

✓ TIPS & BEST PRACTICES

- Use refs for reading the DOM, state for changing what's rendered
- Keep a `Map` of item refs for scroll-to-item lists
- Prefer Fragment refs (sheet 09) over wrapper divs

QUICK SUMMARY

- `<el ref={r} />` → `r.current` = node
- `ref` is a prop in React 19
- Ref callbacks can clean up
- `useImperativeHandle` limits access

useEffect

syncing with external systems · dependency array · cleanup · runs after paint · StrictMode double run · mount/update/unmount

SYNCHRONIZING WITH THE OUTSIDE

`useEffect(setup, deps)` runs code **after** React commits a render to the screen. Use it to **synchronize** a component with something outside React: a network connection, a browser API subscription, a third-party widget, a timer. The setup can return a **cleanup** function that undoes it. React runs cleanup before the next setup and when the component unmounts.

NOTES

- No array: runs after every render; `[]` : after mount only; `[a, b]` : when `a` or `b` change
- Effects run after paint, so they don't block the browser
- In development, StrictMode runs setup → cleanup → setup once on mount to test your cleanup
- Effects only run on the client, not during server rendering

♿ ACCESSIBILITY

- Effects that move focus or announce changes must run after the content exists; `useEffect` timing is right for that

VERSION NOTES

- Unchanged API since 16.8; StrictMode double-invoke since React 18
- React 19.2 adds `useEffectEvent` for non-reactive logic inside effects (sheet 35)

CODE EXPLAINED

- → `ChatRoom` connects after rendering and disconnects on cleanup
- → Changing `roomId` runs cleanup for the old room, then setup for the new one
- → `[]` means the resize listener is added once and removed on unmount
- → The `title` effect re-runs only when `count` changes
- → Each effect handles one concern; split unrelated logic into separate effects

SETUP AND CLEANUP

```jsx
import { useEffect, useState } from 'react';

function ChatRoom({ roomId, serverUrl }) {
  useEffect(() => {
    const conn = createConnection(serverUrl, roomId);
    conn.connect();
    return () => conn.disconnect();   // cleanup
  }, [roomId, serverUrl]);             // re-sync when these change

  return <h1>Welcome to {roomId}</h1>;
}

function WindowWidth() {
  const [width, setWidth] = useState(window.innerWidth);

  useEffect(() => {
    const onResize = () => setWidth(window.innerWidth);
    window.addEventListener('resize', onResize);
    return () => window.removeEventListener('resize', onResize);
  }, []);

  return <p>{width}px</p>;
}

function DocumentTitle({ count }) {
  useEffect(() => {
    document.title = `${count} new messages`;
  }, [count]);
  return null;
}

// Timeline when roomId changes 'a' -> 'b':
//   render(b) -> commit -> cleanup(a) -> setup(b)
```

DEPENDENCY ARRAY

DEPS	RUNS
`omitted`	after every render
`[]`	after mount (cleanup on unmount)
`[a, b]`	mount + when a or b change
`Comparison`	`Object.is` per item

TYPICAL EFFECTS

SYNC WITH	CLEANUP
`WebSocket / chat`	disconnect
`addEventListener`	`removeEventListener`
`setInterval`	`clearInterval`
`Third-party widget`	`widget.destroy()`
`Fetch`	abort or ignore result

⚠ COMMON MISTAKES

- Missing cleanup, which StrictMode reveals as duplicate connections
- Using effects to transform data for rendering (sheet 36)
- Making the setup function `async` (it must return cleanup, not a promise)

✓ TIPS & BEST PRACTICES

- Write the cleanup first: *how do I stop this?*
- One effect per external system
- Move the logic into a custom hook once it works

QUICK SUMMARY

- `useEffect(setup, deps)`
- Runs after paint
- Return cleanup
- Deps decide when it re-syncs

Effect Dependencies

exhaustive-deps · object & function deps · removing deps · updater functions · useEffectEvent · never suppress the linter

DEPS ARE NOT A CHOICE

Every **reactive value** an effect reads (props, state, and anything computed from them in the component body) must be in its dependency array. You don't pick dependencies; the code does, and the `react-hooks/exhaustive-deps` lint rule checks them. To remove a dependency, change the code so the effect doesn't need it. `useEffectEvent` lets an effect read the latest values **without** re-running when they change.

NOTES

- Objects and functions created during render are new each time, so they re-run the effect every render
- Move such objects inside the effect, or outside the component if they don't use props
- Use updater functions (`setX(x => …)`) so the effect doesn't depend on `x`
- Values outside the component, refs and setters are not reactive

♿ ACCESSIBILITY

- Over-firing effects can re-announce live regions or steal focus repeatedly; correct deps prevent it

VERSION NOTES

- `useEffectEvent` is stable in React 19.2
- The React Compiler lint rules in `eslint-plugin-react-hooks` also catch dependency problems

CODE EXPLAINED

→ `options` is a new object each render, so `[options]` always changes

→ Building the object inside the effect leaves only `roomId` as a dependency

→ `setCount(c => c + 1)` doesn't read `count`, so the deps can be `[]`

→ `onConnected` always sees the latest `theme` but isn't a dependency

→ Effect Events are called only from inside effects

FIXING DEPENDENCIES

```jsx
import { useEffect, useEffectEvent, useState } from 'react';

// ✗ options is new every render -> reconnects every render
function Chat({ roomId }) {
  const options = { serverUrl, roomId };
  useEffect(() => {
    const c = createConnection(options);
    c.connect();
    return () => c.disconnect();
  }, [options]);
}

// ✓ Create it inside the effect; depend on primitives
function Chat2({ roomId }) {
  useEffect(() => {
    const c = createConnection({ serverUrl, roomId });
    c.connect();
    return () => c.disconnect();
  }, [roomId]);
}

// ✓ Updater removes the state dependency
useEffect(() => {
  const id = setInterval(() => setCount((c) => c + 1), 1000);
  return () => clearInterval(id);
}, []);

// ✓ Read latest theme without reconnecting on theme change
function Room({ roomId, theme }) {
  const onConnected = useEffectEvent(() => {
    showToast('Connected!', theme);
  });
  useEffect(() => {
    const c = createConnection({ serverUrl, roomId });
    c.on('connected', () => onConnected());
    c.connect();
    return () => c.disconnect();
  }, [roomId]);   // theme is not a dependency
}
```

REMOVE A DEPENDENCY

PROBLEM	FIX
`Object/array created in render`	create inside effect
`Function created in render`	move inside effect
`Reads state only to update it`	updater function
`Reads value but shouldn't re-run`	`useEffectEvent`
`Doesn't use props/state`	move outside component

REACTIVE?

VALUE	IN DEPS?
`Props, state`	yes
`Values computed in render`	yes
`useState setters, dispatch`	no (stable)
`ref objects`	no
`Module constants`	no
`useEffectEvent functions`	no (never list)

⚠ COMMON MISTAKES

- Silencing the linter with `// eslint-disable-next-line`
- Wrapping everything in `useCallback` to stabilize deps instead of restructuring
- Passing Effect Events to other components or hooks

✓ TIPS & BEST PRACTICES

- Treat a lint warning as a bug report
- Prefer primitive deps (`roomId`) over objects (`room`)
- Ask whether the code needs to be in an effect at all (sheet 36)

QUICK SUMMARY

- Every reactive value is a dep
- Change code, not the array
- Updaters remove state deps
- `useEffectEvent` for non-reactive reads

You Might Not Need an Effect

derive during render · logic in handlers · reset with key · chained effects · notifying parents · caching with useMemo

EFFECTS ARE AN ESCAPE HATCH

Effects are for synchronizing with **external systems**. If no external system is involved, you probably don't need one. Compute derived values **during render**, handle user actions in **event handlers**, reset state with a **key**, and cache expensive calculations with `useMemo`. Unnecessary effects cause extra renders, flicker and bugs from chained updates.

NOTES

- Rule of thumb: code that runs *because the component was shown* → effect; *because the user did something* → handler
- Setting state in an effect based on other state causes an extra render pass
- Chains of effects that set state for each other are hard to follow
- Notifying a parent should happen in the same handler that changed the state

♿ ACCESSIBILITY

- Fewer render passes mean less flicker in live regions and focus targets

VERSION NOTES

- From react.dev's *You Might Not Need an Effect*; unchanged in React 19
- The React Compiler makes `useMemo` mostly automatic (sheet 60)

CODE EXPLAINED

→ `fullName` needs no state at all

→ `useMemo` recomputes the filter only when inputs change

→ A new `key` remounts `Profile`, clearing every piece of its state

→ The POST belongs to the click, not to a `submitted` flag

→ Calling `onChange` in the handler avoids a second render cycle

EFFECT → NO EFFECT

```jsx
// ✗ Derived state in an effect
const [fullName, setFullName] = useState('');
useEffect(() => setFullName(`${first} ${last}`), [first, last]);
// ✓ Compute during render
const fullName = `${first} ${last}`;

// ✗ Expensive filter in an effect
useEffect(() => setVisible(filter(todos, tab)), [todos, tab]);
// ✓ Cache it
const visible = useMemo(() => filter(todos, tab), [todos, tab]);

// ✗ Reset on prop change
useEffect(() => setComment(''), [userId]);
// ✓ Reset with a key
<Profile key={userId} userId={userId} />

// ✗ Event logic in an effect
useEffect(() => {
  if (submitted) post('/api/register', data);
}, [submitted]);
// ✓ In the handler
function handleSubmit() { post('/api/register', data); }

// ✗ Notify parent after state changes
useEffect(() => onChange(isOn), [isOn]);
// ✓ Update both in the handler
function toggle() {
  setIsOn(!isOn);
  onChange(!isOn);
}

// ✓ Legit effect: external system
useEffect(() => {
  const unsub = store.subscribe(sync);
  return unsub;
}, []);
```

INSTEAD OF AN EFFECT

GOAL	USE
Value from props/state	compute in render
Expensive calculation	useMemo / Compiler
Reset all state on prop change	key
Adjust some state on prop change	compare during render
Respond to a click	event handler
Subscribe to a store	useSyncExternalStore

EFFECT IS RIGHT FOR

CASE	EXAMPLE
Network connection	chat socket
Browser API	IntersectionObserver
Non-React widget	map library
Analytics on display	page view
Fetch on display	(prefer a data library)

⚠ COMMON MISTAKES

- `useEffect` + `setState` to mirror props
- Effect chains: A sets B, B's effect sets C
- Triggering POST requests from state flags

✓ TIPS & BEST PRACTICES

- Before writing an effect, name the external system it syncs with
- If you can't, look for a render-time or handler solution
- Move data fetching to a library or framework loader

QUICK SUMMARY

- Derive in render
- Act in handlers
- Reset with `key`
- Effects only for external systems

Fetching Data in Effects

fetch in useEffect · AbortController · ignore flag · loading/error state · race conditions · when to use a library

FETCHING WITHOUT A LIBRARY

You can fetch inside `useEffect`, but you must handle **loading**, **errors** and **race conditions**: if the input changes quickly, an older response can arrive after a newer one. Cancel stale requests with an `AbortController` in the cleanup, or set an `ignore` flag. For real apps, prefer **TanStack Query**, **SWR** or a framework loader, which add caching, deduping and retries.

NOTES

- The effect callback can't be `async`; define an async function inside and call it
- `AbortController` cancels the network request; ignore flags just drop the result
- `fetch` only rejects on network errors; check `res.ok` for HTTP errors
- Effects don't run on the server, so this pattern can't prefetch during SSR

♿ ACCESSIBILITY

- Render loading and error text inside an element with `role="status"` or `aria-live="polite"`
- Use `aria-busy="true"` on the region being updated

VERSION NOTES

- React 19's `use()` with Suspense offers a different approach for cached promises (sheet 47)
- React docs recommend frameworks or data libraries over effect-based fetching

CODE EXPLAINED

- → One `status` field avoids contradictory loading/error booleans
- → `signal` ties the request to the controller
- → Cleanup aborts the old request when `userId` changes or the component unmounts
- → `AbortError` is expected and ignored
- → `res.ok` turns 404s and 500s into errors

USEEFFECT + ABORTCONTROLLER

```jsx
function UserProfile({ userId }) {
  const [state, setState] = useState({
    status: 'loading', user: null, error: null,
  });

  useEffect(() => {
    const controller = new AbortController();
    setState({ status: 'loading', user: null, error: null });

    async function load() {
      try {
        const res = await fetch(`/api/users/${userId}`, {
          signal: controller.signal,
        });
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        const user = await res.json();
        setState({ status: 'success', user, error: null });
      } catch (err) {
        if (err.name === 'AbortError') return;
        setState({ status: 'error', user: null, error: err });
      }
    }

    load();
    return () => controller.abort();
  }, [userId]);

  if (state.status === 'loading')
    return <p role="status">Loading…</p>;
  if (state.status === 'error')
    return <p role="alert">{state.error.message}</p>;
  return <h2>{state.user.name}</h2>;
}

// Same idea, ignore flag:
// let ignore = false; … if (!ignore) setX(data);
// return () => { ignore = true; };
```

PROBLEMS TO HANDLE

PROBLEM	FIX
Race conditions	abort or ignore in cleanup
HTTP errors	check `res.ok`
Loading state	status field
StrictMode double fetch	abort makes it harmless
No cache	use a data library

EFFECT VS LIBRARY

USEEFFECT	TANSTACK QUERY / SWR
Manual loading/error	built in
No caching	cache + dedupe
Manual retries	automatic retries
Refetch by hand	refetch on focus/reconnect
No SSR prefetch	SSR/hydration support

⚠ COMMON MISTAKES

- `useEffect(async () => …)`, which returns a promise instead of cleanup
- Ignoring stale responses and showing the wrong user's data
- Treating a 404 as success because `fetch` didn't throw

✓ TIPS & BEST PRACTICES

- Extract the pattern into a `useFetch` hook while learning
- Switch to TanStack Query (sheet 77) before adding caching by hand
- Show stale data with a spinner rather than blanking the UI on refetch

QUICK SUMMARY

- Async function inside the effect
- Abort in cleanup
- Check `res.ok`
- Prefer a data library

useLayoutEffect & useInsertionEffect

before paint · measuring the DOM · tooltip positioning · flicker · useInsertionEffect for CSS-in-JS · SSR caveats

EFFECTS BEFORE PAINT

`useLayoutEffect` has the same signature as `useEffect` but runs **synchronously after DOM changes and before the browser paints**. Use it when you must measure layout and update state before the user sees anything, such as positioning a tooltip, to avoid a visible jump. It blocks painting, so keep it rare and fast. `useInsertionEffect` runs even earlier and exists for CSS-in-JS libraries to inject styles.

NOTES

- Order: render → DOM mutation → insertion effects → layout effects → paint → effects
- State updates inside `useLayoutEffect` re-render synchronously before paint
- `useLayoutEffect` does nothing on the server and warns in older React versions
- `useInsertionEffect` can't update state or read refs

♿ ACCESSIBILITY

- Positioning popovers before paint prevents them from appearing over the trigger and confusing magnifier users

VERSION NOTES

- React 19 no longer warns about `useLayoutEffect` during server rendering
- CSS anchor positioning can replace many measure-and-position effects

CODE EXPLAINED

→ The first render places the tooltip at height `0`, invisibly

→ The layout effect measures and sets `height` before the browser paints

→ The second render positions it correctly; users never see the first

→ If there's no room above, the tooltip flips below the target

→ `useInsertionEffect` adds styles before any layout effect reads them

MEASURE BEFORE PAINT

```
import { useLayoutEffect, useRef, useState } from 'react';

function Tooltip({ targetRect, children }) {
  const ref = useRef(null);
  const [height, setHeight] = useState(0);

  useLayoutEffect(() => {
    const { height } = ref.current.getBoundingClientRect();
    setHeight(height);              // re-renders before paint
  }, []);

  let top = targetRect.top - height;
  if (top < 0) top = targetRect.bottom;   // flip below

  return (
    <div ref={ref} role="tooltip"
      style={{ position: 'fixed', top, left: targetRect.left }}>
      {children}
    </div>
  );
}

// For CSS-in-JS authors only
import { useInsertionEffect } from 'react';
function useCSS(rule) {
  useInsertionEffect(() => {
    const style = document.createElement('style');
    style.textContent = rule;
    document.head.append(style);
    return () => style.remove();
  }, [rule]);
}

// Timeline:
// render -> DOM -> insertion -> layout -> PAINT -> useEffect
```

EFFECT TIMING

HOOK	RUNS
`useInsertionEffect`	before layout effects; no state
`useLayoutEffect`	after DOM update, before paint
`useEffect`	after paint
`Ref callbacks`	during commit, before layout effects

WHICH EFFECT?

TASK	HOOK
`Measure then position`	`useLayoutEffect`
`Scroll restore without flash`	`useLayoutEffect`
`Subscribe / fetch / log`	`useEffect`
`Inject <style> (library)`	`useInsertionEffect`
`Position popover`	CSS anchor positioning

⚠ COMMON MISTAKES

- Using `useLayoutEffect` by default and slowing every render
- Heavy work in layout effects that blocks paint
- Using `useInsertionEffect` in app code

✓ TIPS & BEST PRACTICES

- Start with `useEffect`; switch only if you see a flicker
- Check whether CSS (anchor positioning, `position: sticky`) solves it first
- Use a positioning library such as Floating UI for complex popovers

QUICK SUMMARY

- `useLayoutEffect` = before paint
- For measure-then-render
- Blocks paint: use sparingly
- `useInsertionEffect` = CSS-in-JS only

Context Basics

PASSING DATA DEEPLY

Context lets a parent make a value available to every component below it without passing props through each level. Create a context with `createContext(default)`, provide a value by rendering `<MyContext value={…}>` around a subtree, and read it with `useContext(MyContext)` (or `use(MyContext)`). A component reads the value from the **nearest** provider above it; with none, it gets the default.

NOTES

- Good fits: theme, current user, locale, router, a store or client instance
- When the provided value changes, every component that reads it re-renders
- Providers can be nested; inner ones override outer ones for their subtree
- Export the context object or, better, a custom hook that reads it

♿ ACCESSIBILITY

- A locale or reduced-motion preference in context lets every component adapt text and animation consistently

VERSION NOTES

- React 19: render `<ThemeContext value>` directly; `<ThemeContext.Provider>` still works but will be deprecated
- React 19: `use(Context)` can read context conditionally
- React 19.3: `<Context>` can be rendered directly in Server Components

CODE EXPLAINED

- → `createContext('light')` sets the default used with no provider
- → `<ThemeContext value={theme}>` provides the value to everything inside
- → `Page` doesn't pass `theme`; `Button` reads it directly
- → Toggling `theme` re-renders every component that reads the context
- → An inner provider overrides the value for its subtree

CREATECONTEXT, PROVIDER, USECONTEXT

```jsx
import { createContext, useContext, useState } from 'react';

export const ThemeContext = createContext('light');

export default function App() {
  const [theme, setTheme] = useState('dark');
  return (
    <ThemeContext value={theme}>
      <Page />
      <button onClick={() =>
        setTheme(theme === 'dark' ? 'light' : 'dark')}>
        Toggle theme
      </button>
    </ThemeContext>
  );
}

function Page() {
  return <Panel title="Welcome"><Button>Sign up</Button></Panel>;
}

function Button({ children }) {
  const theme = useContext(ThemeContext);
  return <button className={`btn-${theme}`}>{children}</button>;
}

// Nested override
<ThemeContext value="light">
  <Sidebar />   {/* always light */}
</ThemeContext>

// Legacy provider syntax
<ThemeContext.Provider value={theme}>…</ThemeContext.Provider>
```

CONTEXT API

API	USE
`createContext(default)`	create
`<Ctx value={v}>`	provide (19)
`<Ctx.Provider value>`	provide (legacy)
`useContext(Ctx)`	read
`use(Ctx)`	read, allowed in conditions
`<Ctx.Consumer>`	legacy render-prop reader

PROPS VS CONTEXT

PROPS	CONTEXT
Explicit data flow	Implicit, any depth
Easy to trace	Harder to trace
Re-render only receivers	Re-render all readers
Use by default	Use for widely shared values

⚠ COMMON MISTAKES

- Reading context in the same component that renders its provider (it gets the outer value)
- Using context for values only one or two levels down
- Forgetting the provider and silently using the default

✓ TIPS & BEST PRACTICES

- Wrap `useContext` in a custom hook with a helpful error (sheet 40)
- Default to `null` and throw when no provider is found
- Try passing `children` before reaching for context

QUICK SUMMARY

- `createContext` → `<Ctx value>` → `useContext`
- Nearest provider wins
- All readers re-render on change
- `.Provider` is legacy in 19

Context Patterns

reducer + context · provider component · useX() hook with guard · splitting state & dispatch · use(Context)

PATTERNS THAT SCALE

In real apps, context usually comes wrapped in a **provider component** and a **custom hook**. The provider owns the state (often with `useReducer`), and the hook reads the context and throws a clear error when used outside the provider. Splitting **state** and **dispatch** into two contexts lets components that only send actions avoid re-rendering when state changes.

NOTES

- Keep the context object private to the module; export the provider and hook
- `dispatch` is stable, so a dispatch-only context never changes
- Initialize context with `null` to detect a missing provider
- `use(Ctx)` can be called inside `if` blocks and loops, unlike `useContext`

♿ ACCESSIBILITY

- A notifications provider is a good place to render one shared `aria-live` region

VERSION NOTES

- `use(Context)` is new in React 19
- The reducer + context pattern is documented on react.dev as *Scaling Up with Reducer and Context*

CODE EXPLAINED

→ `TasksProvider` hides the reducer and both contexts

→ Two contexts let readers and writers subscribe separately

→ The hooks throw a clear error if a component is outside the provider

→ `AddTask` only reads `dispatch`, which never changes

→ Consumers import hooks, not context objects

REDUCER + CONTEXT PROVIDER

```
import { createContext, use, useReducer } from 'react';

const TasksContext = createContext(null);
const DispatchContext = createContext(null);

function tasksReducer(tasks, action) {
  switch (action.type) {
    case 'added': return [...tasks, action.task];
    case 'deleted': return tasks.filter((t) => t.id !== action.id);
    default: throw new Error(action.type);
  }
}

export function TasksProvider({ children }) {
  const [tasks, dispatch] = useReducer(tasksReducer, []);
  return (
    <TasksContext value={tasks}>
      <DispatchContext value={dispatch}>
        {children}
      </DispatchContext>
    </TasksContext>
  );
}

export function useTasks() {
  const ctx = use(TasksContext);
  if (ctx === null) throw new Error('useTasks needs <TasksProvider>');
  return ctx;
}

export function useTasksDispatch() {
  const ctx = use(DispatchContext);
  if (ctx === null) throw new Error('needs <TasksProvider>');
  return ctx;
}

// Anywhere below the provider
function AddTask() {
  const dispatch = useTasksDispatch();   // never re-renders on task changes
  return <button onClick={() => dispatch({ type: 'added',
    task: { id: crypto.randomUUID(), text: 'New' } })}>Add</button>;
}
```

PATTERN PIECES

PIECE	ROLE
`Context (private)`	carries the value
`Provider component`	owns state, renders contexts
`useX() hook`	reads + validates
`Reducer`	update logic
`Split contexts`	fewer re-renders

USECONTEXT VS USE

USECONTEXT	USE
`Top level only`	Can be conditional
`React 16.8+`	React 19+
`Context only`	Context or promise
`Same value semantics`	Same value semantics

⚠ COMMON MISTAKES

- Exporting the raw context so callers skip validation
- Putting state and dispatch in one object and re-rendering every writer
- Creating one giant app-wide context for unrelated data

✓ TIPS & BEST PRACTICES

- Compose providers in a single `Providers` component
- Keep each context focused on one concern
- Move to an external store (sheet 84) when updates are frequent

QUICK SUMMARY

- Provider component + custom hook
- Throw outside the provider
- Split state and dispatch
- `use(Ctx)` allowed in conditions

Context Performance & Alternatives

consumers re-render · memoizing value · splitting contexts · composition before context · when to use a store

WHEN CONTEXT GETS SLOW

Every component that reads a context re-renders when its value changes, even if it only uses one field. Providing a **new object every render** (`value={{ user, setUser }}`) makes all readers re-render whenever the provider re-renders. Memoize the value, **split** contexts by how often they change, and remember that **composition** often removes the need for context. For high-frequency global state, use a store with selectors.

NOTES

- React compares the provided value with `Object.is`
- `memo` on a child does not block context updates it reads
- The React Compiler memoizes `{ user, setUser }` objects for you
- Context is a transport, not a state manager: it has no selectors

♿ ACCESSIBILITY

- Frequent context churn can re-render focus-sensitive widgets and reset their transient state

VERSION NOTES

- Context selectors are not part of React 19; libraries like `use-context-selector` or stores fill the gap
- React Compiler 1.0 (October 2025) auto-memoizes provider values

CODE EXPLAINED

→ The object literal is recreated each render, so the value always differs

→ `useMemo` keeps the same object until `user` or `login` change

→ Cursor updates re-render only `CursorContext` readers

→ Passing `<Avatar user>` as JSX skips the middle layers entirely

→ A selector re-renders only when `cart.count` changes

STABLE VALUES AND SPLIT CONTEXTS

```
// ✗ New object every render -> every reader re-renders
function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  return (
    <AuthContext value={{ user, setUser }}>{children}</AuthContext>
  );
}

// ✓ Memoize (or let the React Compiler do it)
function AuthProvider2({ children }) {
  const [user, setUser] = useState(null);
  const login = useCallback(async (c) => setUser(await api.login(c)), []);
  const value = useMemo(() => ({ user, login }), [user, login]);
  return <AuthContext value={value}>{children}</AuthContext>;
}

// ✓ Split fast-changing from slow-changing data
<SettingsContext value={settings}>       {/* rarely changes */}
  <CursorContext value={cursor}>         {/* changes on mousemove */}
    <Canvas />
  </CursorContext>
</SettingsContext>

// ✓ Composition instead of context
function Layout({ user }) {
  return <Page header={<Avatar user={user} />} />;
}

// ✓ Store with selectors for high-frequency state
const count = useStore((s) => s.cart.count);  // Zustand
```

RE-RENDER FIXES

PROBLEM	FIX
`Inline object value`	`useMemo` / Compiler
`Readers need one field`	split contexts
`Fast + slow data together`	separate providers
`Deep prop drilling only`	composition
`Frequent global updates`	store with selectors

CONTEXT IS GOOD FOR

VALUE	CHANGES
`Theme / locale`	rarely
`Current user`	on login/logout
`Feature flags`	rarely
`Router / query client`	never
`Mouse position`	✗ too often

⚠ COMMON MISTAKES

- Putting rapidly changing values (scroll, input text) in app-wide context
- Expecting `memo` to stop context-triggered re-renders
- Memoizing the value but recreating the functions inside it each render

✓ TIPS & BEST PRACTICES

- Profile before optimizing; context is often fast enough
- Pass IDs through context and let components fetch details
- Move the provider down to wrap only the part of the tree that needs it

QUICK SUMMARY

- Readers re-render on value change
- Stable value object
- Split by update frequency
- Stores for hot global state

Rules of Hooks

top level only · components & hooks only · call order · use prefix · eslint-plugin-react-hooks · use() exception

TWO RULES

Hooks are functions whose names start with `use`. React tracks them by **call order**, so two rules apply. **Only call hooks at the top level** of a component or custom hook: never inside conditions, loops, nested functions or after an early `return`. **Only call hooks from React functions**: components and custom hooks, not regular functions or class components. The `react-hooks` ESLint plugin enforces both.

NOTES

- React stores each hook's state in a list indexed by call position
- A conditional hook shifts every later hook's position and mixes up state
- Put the condition inside the hook: `useEffect(() => { if (x) … })`
- `use()` is the exception: it may be called in conditions and loops

♿ ACCESSIBILITY

- Hook-order bugs often surface as the wrong `aria-expanded` or focus state on the wrong item

VERSION NOTES

- Rules unchanged since 16.8; `use()` (React 19) is an API, not a hook, and can be conditional
- `eslint-plugin-react-hooks` adds compiler-based rules for purity and ref access

CODE EXPLAINED

→ The early `return` means `useState` runs on some renders but not others

→ Moving hooks above the branch keeps their order fixed

→ Hooks can't run in loops or in callbacks that run later

→ Effects can contain conditions; they just can't be conditional themselves

→ `use(promise)` after a branch is allowed

BREAKING AND FOLLOWING THE RULES

```
// ✗ Conditional hook
function Profile({ user }) {
  if (!user) return <SignIn />;          // early return...
  const [tab, setTab] = useState('info');// ...then a hook
}

// ✓ Hooks first, then branch
function Profile2({ user }) {
  const [tab, setTab] = useState('info');
  if (!user) return <SignIn />;
  return <Tabs value={tab} onChange={setTab} />;
}

// ✗ Hook in a loop or callback
items.forEach(() => useState(0));
const onClick = () => { useEffect(…); };

// ✓ Condition inside the effect
useEffect(() => {
  if (!enabled) return;
  const id = setInterval(tick, 1000);
  return () => clearInterval(id);
}, [enabled]);

// ✓ use() may be conditional
function Message({ show, messagePromise }) {
  if (!show) return null;
  const msg = use(messagePromise);
  return <p>{msg}</p>;
}

// eslint.config.js
import reactHooks from 'eslint-plugin-react-hooks';
export default [reactHooks.configs.recommended];
```

ALLOWED?

CALL SITE	OK?
`Top of component`	✓
`Top of custom hook`	✓
`Inside if / loop`	✗ (except `use`)
`After early return`	✗
`Event handler / callback`	✗
`Class component`	✗
`Plain utility function`	✗

LINT RULES

RULE	CATCHES
`rules-of-hooks`	conditional or misplaced hooks
`exhaustive-deps`	missing effect/memo deps
`Compiler rules`	impure render, ref reads, setState in render
`use prefix`	hooks named like functions

⚠ COMMON MISTAKES

- Early returns before hooks
- Naming a regular function `useSomething` without calling hooks
- Disabling the lint rule to make a conditional hook compile

✓ TIPS & BEST PRACTICES

- Order component bodies: hooks → derived values → handlers → early returns → JSX
- Split a component when hooks differ between branches
- Run ESLint in CI so violations never merge

QUICK SUMMARY

- Top level only
- Only in components and hooks
- Order must never change
- `use()` is the exception

Writing Custom Hooks

extracting logic · naming · return values · composing hooks · state isn't shared · useDebugValue

EXTRACTING LOGIC

A **custom hook** is a function whose name starts with `use` and that calls other hooks. It packages stateful logic (subscriptions, timers, form state, data fetching) so several components can share it. Each component that calls the hook gets its **own independent state**; hooks share logic, not data. Return whatever is most convenient: a value, a tuple, or an object.

NOTES

- Name it after what it does: `useOnlineStatus`, `useChatRoom`
- Pass reactive values in as arguments; the hook re-syncs when they change
- Return tuples `[value, setValue]` for two items, objects for more
- `useDebugValue(label)` shows a label next to the hook in React DevTools

♿ ACCESSIBILITY

- Encapsulate accessibility behavior (focus traps, roving tabindex, live announcements) in hooks so every component gets it right

VERSION NOTES

- Custom hooks are plain functions; nothing new in React 19
- The React Compiler optimizes custom hooks like components

CODE EXPLAINED

- → `useOnlineStatus` hides two listeners and their cleanup
- → `useDebugValue` labels the hook in DevTools
- → `useToggle` returns a tuple, like `useState`
- → `useAutosave` composes `useOnlineStatus` and re-runs when `value` changes
- → Every component calling `useOnlineStatus` gets its own state and listeners

RETURN SHAPES

SHAPE	WHEN
`Single value`	`const online = useOnline()`
`Tuple`	value + setter, easy renaming
`Object`	many values, named access
`Nothing`	side effect only (`useTitle`)

WRITING AND COMPOSING HOOKS

```jsx
import { useDebugValue, useEffect, useState } from 'react';

export function useOnlineStatus() {
  const [online, setOnline] = useState(navigator.onLine);
  useEffect(() => {
    const on = () => setOnline(true);
    const off = () => setOnline(false);
    window.addEventListener('online', on);
    window.addEventListener('offline', off);
    return () => {
      window.removeEventListener('online', on);
      window.removeEventListener('offline', off);
    };
  }, []);
  useDebugValue(online ? 'Online' : 'Offline');
  return online;
}

export function useToggle(initial = false) {
  const [on, setOn] = useState(initial);
  const toggle = () => setOn((v) => !v);
  return [on, toggle];
}

// Hooks can call hooks
export function useAutosave(value, save) {
  const online = useOnlineStatus();
  useEffect(() => {
    if (!online) return;
    const id = setTimeout(() => save(value), 1000);
    return () => clearTimeout(id);
  }, [value, online, save]);
  return online;
}

function StatusBar() {
  const online = useOnlineStatus();   // own state
  const [open, toggle] = useToggle();
  return <p>{online ? '✅ Online' : '❌ Offline'}</p>;
}
```

GOOD CANDIDATES

LOGIC	HOOK
`Browser API subscription`	`useMediaQuery`
`Timer`	`useInterval`
`Form field state`	`useField`
`Data loading`	`useUser(id)`
`Keyboard shortcut`	`useHotkey`

⚠ COMMON MISTAKES

- Expecting two components using the same hook to share state
- Hooks named `useX` that don't call hooks (make them plain functions)
- Lifecycle-style hooks (`useMount`) that hide missing dependencies

✓ TIPS & BEST PRACTICES

- Extract a hook when two components copy the same effect
- Keep hooks focused on one concern
- Test hooks with `renderHook` (sheet 94)

QUICK SUMMARY

- `use*` + calls hooks
- Shares logic, not state
- Compose hooks freely
- `useDebugValue` for DevTools

Common Custom Hooks

useToggle · useLocalStorage · useDebounce · useMediaQuery · useOnClickOutside · usePrevious · useOnlineStatus

A STARTER KIT

A handful of small hooks show up in almost every codebase. They wrap browser APIs (`matchMedia`, `localStorage`, `online` events), timers (debounce) and DOM events (click outside) behind a simple interface. Write your own for learning; for production, libraries like **usehooks-ts** or **react-use** offer tested versions with SSR handling.

NOTES

- Guard browser APIs for server rendering (`typeof window !== 'undefined'`)
- Subscriptions to external sources are best built on `useSyncExternalStore` (sheet 45)
- Debounce values for search inputs; throttle for scroll and resize
- `usePrevious` reads a value from the previous render

♿ ACCESSIBILITY

- `useMediaQuery('(prefers-reduced-motion: reduce)')` lets components disable animation
- Click-outside handlers should also close on Escape

VERSION NOTES

- Plain hook code; works on React 18 and 19
- React 19.2's `useEffectEvent` simplifies hooks that call user callbacks

CODE EXPLAINED

- → `useLocalStorage` reads once lazily and writes on every change
- → `useDebounce` resets its timer whenever `value` changes
- → `useMediaQuery` uses `useSyncExternalStore`, with `false` on the server
- → `useEffectEvent` lets the latest `handler` run without re-subscribing
- → `ref.current?.` guards against the element not being mounted yet

COMMON HOOKS

```
export function useLocalStorage(key, initial) {
  const [value, setValue] = useState(() => {
    const saved = localStorage.getItem(key);
    return saved !== null ? JSON.parse(saved) : initial;
  });
  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);
  return [value, setValue];
}

export function useDebounce(value, delay = 300) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id);
  }, [value, delay]);
  return debounced;
}

export function useMediaQuery(query) {
  return useSyncExternalStore(
    (cb) => {
      const mql = matchMedia(query);
      mql.addEventListener('change', cb);
      return () => mql.removeEventListener('change', cb);
    },
    () => matchMedia(query).matches,
    () => false,
  );
}

export function useOnClickOutside(ref, handler) {
  const onOutside = useEffectEvent(handler);
  useEffect(() => {
    const listener = (e) => {
      if (!ref.current?.contains(e.target)) onOutside(e);
    };
    const doc = document;
    doc.addEventListener('pointerdown', listener);
    return () => doc.removeEventListener('pointerdown', listener);
  }, [ref]);
}
```

HOOK CATALOG

HOOK	RETURNS
`useToggle(init)`	`[on, toggle]`
`useLocalStorage(k, init)`	`[value, setValue]`
`useDebounce(v, ms)`	debounced value
`useMediaQuery(q)`	boolean
`useOnClickOutside(ref, fn)`	nothing
`usePrevious(v)`	previous value
`useOnlineStatus()`	boolean

LIBRARIES

LIBRARY	NOTES
`usehooks-ts`	TypeScript, SSR-safe
`react-use`	large collection
`@uidotdev/usehooks`	small, modern
`ahooks`	data and UI hooks

⚠ COMMON MISTAKES

- Reading `localStorage` during render on the server
- Debouncing the handler instead of the value and losing the latest input
- Forgetting to remove document listeners

✓ TIPS & BEST PRACTICES

- Debounce search: `const q = useDebounce(input, 300)`
- Wrap `JSON.parse` in `try` for corrupted storage
- Prefer CSS media queries when only styles change

QUICK SUMMARY

- Wrap browser APIs in hooks
- Guard for SSR
- Debounce values, not handlers
- Use a library in production

useSyncExternalStore & useId

subscribe · getSnapshot · getServerSnapshot · tearing · useId · label & aria linking

EXTERNAL STORES AND STABLE IDS

`useSyncExternalStore(subscribe, getSnapshot, getServerSnapshot?)` reads a value from a source React doesn't own (a browser API, a third-party store) and re-renders when it changes, without **tearing** during concurrent rendering. `useId()` generates a unique, **stable ID** that matches between server and client, ideal for linking labels and ARIA attributes.

NOTES

- `subscribe(callback)` must return an unsubscribe function
- `getSnapshot` must return the same value (by `Object.is`) if nothing changed
- Define `subscribe` outside the component, or React re-subscribes every render
- Never use `useId` for list keys; keys should come from your data

♿ ACCESSIBILITY

- `useId` links `<label htmlFor>`, `aria-describedby` and `aria-labelledby` without hard-coded, duplicated IDs
- Generate one ID and suffix it for related elements: `${id}-hint`

VERSION NOTES

- Both hooks arrived in React 18
- React 19.1 changed `useId` format to `«r1»`; 19.2 changed it to `_r_1_` so it's valid in CSS selectors and `view-transition-name`

CODE EXPLAINED

→ `subscribe` is defined at module level so its identity never changes

→ The snapshot returns a boolean, so equal values skip re-renders

→ The server snapshot avoids touching `navigator` during SSR

→ `useId` produces the same ID on server and client

→ Suffixes create related IDs from one generated base

SUBSCRIBE AND LINK

```
import { useId, useSyncExternalStore } from 'react';

// Browser API store
function subscribe(callback) {
  window.addEventListener('online', callback);
  window.addEventListener('offline', callback);
  return () => {
    window.removeEventListener('online', callback);
    window.removeEventListener('offline', callback);
  };
}

function useOnline() {
  return useSyncExternalStore(
    subscribe,
    () => navigator.onLine,   // client snapshot
    () => true,               // server snapshot
  );
}

// Third-party store
const count = useSyncExternalStore(
  store.subscribe, () => store.getState().count,
);

// Accessible IDs
function PasswordField() {
  const id = useId();
  return (
    <>
      <label htmlFor={id}>Password</label>
      <input id={id} type="password"
        aria-describedby={`${id}-hint`} />
      <p id={`${id}-hint`}>At least 8 characters</p>
    </>
  );
}

// Several fields from one prefix
const base = useId();
<input id={`${base}-first`} />;
<input id={`${base}-last`} />;
```

USESYNCEXTERNALSTORE

ARGUMENT	MUST
`subscribe(cb)`	call `cb` on change, return unsubscribe
`getSnapshot()`	return cached/immutable value
`getServerSnapshot()`	return SSR value
`Result`	current snapshot

USEID DO / DON'T

USE	AVOID
`htmlFor / id pairs`	list keys
`aria-describedby`	data IDs from your API
`aria-labelledby`	calling inside loops
`Prefix for groups`	one `useId` per item in `map`

⚠ COMMON MISTAKES

- Returning a new object from `getSnapshot` every call (infinite loop)
- Using `useEffect` + `setState` to mirror an external store
- Using `Math.random()` IDs that differ between server and client

✓ TIPS & BEST PRACTICES

- Prefer a store's own hook (Zustand, Redux) if it exists; they use this API internally
- Call `useId` once per component and derive suffixes
- Set `identifierPrefix` on roots when several React apps share a page

QUICK SUMMARY

- `useSyncExternalStore` for outside data
- Stable `subscribe`, cached snapshot
- `useId` for accessible IDs
- Not for list keys

Suspense

DECLARATIVE LOADING STATES

`<Suspense fallback={…}>` shows a fallback while anything inside it is **suspended**: waiting for lazy-loaded code, a promise read with `use()`, or Suspense-enabled data from a framework or library. When everything is ready, React swaps in the content. Nest boundaries to reveal a page in stages, and place them where a loading state makes sense to the user.

NOTES

- Only Suspense-enabled sources trigger it: `lazy`, `use(promise)`, framework loaders, `useSuspenseQuery`
- Fetching in `useEffect` does **not** suspend
- The nearest boundary above the suspending component catches it
- Updates inside a transition keep showing old content instead of the fallback

♿ ACCESSIBILITY

- Give fallbacks accessible text (`role="status"`, *Loading results…*), not only a spinner
- Avoid many tiny boundaries that pop in one by one and shift focus targets

VERSION NOTES

- React 19 shows the fallback immediately and pre-warms siblings in the background
- React 19.2 batches Suspense reveals during SSR streaming for smoother reveals
- Suspense for data needs a framework or library; React has no built-in fetch cache for client components

CODE EXPLAINED

→ The outer boundary covers the article; the inner one only covers comments

→ The article can appear while comments are still loading

→ `lazy` suspends the first time `Comments` renders

→ `startTransition` keeps the current tab visible while the next one loads

→ `isPending` dims the old content as a hint

BOUNDARIES AND NESTING

```jsx
import { Suspense, lazy } from 'react';

const Comments = lazy(() => import('./Comments'));

export default function ArticlePage({ id }) {
  return (
    <Suspense fallback={<PageSkeleton />}>
      <Article id={id} />

      {/* Reveal comments separately */}
      <Suspense fallback={<p role="status">Loading comments…</p>}>
        <Comments articleId={id} />
      </Suspense>
    </Suspense>
  );
}

// Suspending sources:
//   lazy(() => import(...))
//   use(promise)
//   useSuspenseQuery(...)          (TanStack Query)
//   framework loaders / RSC data

// Keep old UI during navigation instead of fallback
function Tabs() {
  const [tab, setTab] = useState('home');
  const [isPending, startTransition] = useTransition();
  return (
    <>
      <button onClick={() => startTransition(() => setTab('posts'))}>
        Posts
      </button>
      <div style={{ opacity: isPending ? 0.6 : 1 }}>
        <Suspense fallback={<Spinner />}>
          <TabPanel tab={tab} />
        </Suspense>
      </div>
    </>
  );
}
```

BOUNDARY PLACEMENT

WHERE	EFFECT
`Around the whole route`	one big loading state
`Around slow sections`	staged reveal
`Around each list item`	popcorn effect (avoid)
`Inside a transition`	old UI stays

SUSPENDS?

SOURCE	SUSPENDS
`lazy() component`	✓
`use(promise)`	✓
`useSuspenseQuery`	✓
`useEffect fetch`	✗
`useQuery (non-suspense)`	✗

⚠ COMMON MISTAKES

- Expecting effect-based fetching to show the Suspense fallback
- Creating the promise inside the suspended component (refetches forever)
- One boundary per item, causing layout jumps

✓ TIPS & BEST PRACTICES

- Design fallbacks the same size as the content to avoid shifts
- Pair Suspense with an error boundary at the same level
- Use transitions for navigation so users keep context

QUICK SUMMARY

- `<Suspense fallback>` catches waits
- Needs Suspense-enabled sources
- Nest for staged reveal
- Transitions keep old UI

The use() API

use(promise) · use(context) · conditional calls · caching promises · no try/catch · error boundaries

READING RESOURCES IN RENDER

`use(resource)` reads the value of a **promise** or a **context**. With a promise, the component suspends until it resolves, then receives the value; if it rejects, the nearest error boundary renders. Unlike hooks, `use` can be called inside conditions and loops. The promise must be **cached or created outside** the component, typically passed down from a Server Component, a loader or a cache.

NOTES

- Promises created during render are new each time, which suspends forever
- Pass promises from Server Components to Client Components and `use` them there
- `use` can't be called inside `try` / `catch`; use an error boundary or `.catch()`
- Resolved values must be serializable when crossing the server/client boundary

♿ ACCESSIBILITY

- Pair `use(promise)` with a Suspense fallback that announces loading

VERSION NOTES

- `use` is new in React 19
- React 19.2 adds `cacheSignal` for Server Components to abort work when a cached call is no longer needed

CODE EXPLAINED

- → The Server Component starts the fetch but passes the unresolved promise
- → `Message` suspends until the promise resolves, showing the fallback
- → A rejection is caught by the `ErrorBoundary`
- → `use(ThemeContext)` runs only on the non-plain branch
- → The `Map` cache returns the same promise for the same `id`

USE(PROMISE) AND USE(CONTEXT)

```
import { Suspense, use } from 'react';
import { ErrorBoundary } from 'react-error-boundary';

// Server Component (or loader) creates the promise once
export default async function Page({ id }) {
  const messagePromise = fetchMessage(id);   // not awaited
  return (
    <ErrorBoundary fallback={<p>Couldn't load.</p>}>
      <Suspense fallback={<p>Loading…</p>}>
        <Message messagePromise={messagePromise} />
      </Suspense>
    </ErrorBoundary>
  );
}

// Client Component reads it
'use client';
function Message({ messagePromise }) {
  const message = use(messagePromise);
  return <p>{message.text}</p>;
}

// Conditional context read
function Heading({ children, plain }) {
  if (plain) return <h1>{children}</h1>;
  const theme = use(ThemeContext);
  return <h1 className={theme}>{children}</h1>;
}

// Client-only: cache the promise outside render
const cache = new Map();
function getUser(id) {
  if (!cache.has(id)) cache.set(id, fetchUser(id));
  return cache.get(id);
}
function User({ id }) {
  const user = use(getUser(id));
  return <span>{user.name}</span>;
}
```

USE VS ALTERNATIVES

NEED	USE
Read context conditionally	use(Ctx)
Read a passed-in promise	use(promise)
Fetch with caching on client	TanStack Query
Await on the server	await in Server Component
Fetch in effect	useEffect (manual states)

RULES

RULE	WHY
Stable promise	new promise = suspend again
No try/catch	use error boundaries
Inside components/hooks	it's a React API
Conditionals allowed	unlike hooks

⚠ COMMON MISTAKES

- `use(fetch(url))` directly in a client component
- Wrapping `use` in `try/catch`
- Awaiting the promise on the server and losing streaming

✓ TIPS & BEST PRACTICES

- Start fetches early (in the parent) and `use` them late (in the child)
- Handle rejections with `promise.catch(() => fallback)` when you want a value instead of an error
- Prefer framework or library caches over hand-rolled maps

QUICK SUMMARY

- `use(promise)` suspends until resolved
- `use(context)` can be conditional
- Promise must be stable
- Errors → error boundary

useTransition & startTransition

isPending · non-blocking updates · async actions · transitions + Suspense · not for text inputs

NON-BLOCKING UPDATES

A **transition** marks a state update as non-urgent. React renders it in the background, keeps the current UI interactive, and can interrupt it if something more urgent (like typing) happens. `useTransition()` returns `[isPending, startTransition]`. In React 19, `startTransition` accepts **async functions**, called *Actions*, and `isPending` stays true until they finish.

NOTES

- Use transitions for tab switches, navigation, filtering large lists
- Updates inside a transition don't show Suspense fallbacks for already-visible content
- Text input updates must be urgent; never put the controlled `value` update in a transition
- Standalone `startTransition` (imported from `react`) works without `isPending`

♿ ACCESSIBILITY

- Show `isPending` as visible text or `aria-busy` so users know something is happening
- Don't disable the controls that started the transition; users may change their mind

VERSION NOTES

- React 18 introduced transitions
- React 19: async functions in `startTransition` (Actions); state set after `await` must be wrapped in another `startTransition`

CODE EXPLAINED

→ `setTab` inside `startTransition` lets a click on *About* interrupt rendering *Posts*

→ `aria-busy` and `isPending` signal the background work

→ The async function makes `isPending` true for the whole request

→ State updates after `await` need their own `startTransition` (a current limitation)

→ `name` updates stay urgent so typing never lags

USETRANSITION

```jsx
import { useState, useTransition } from 'react';

function TabContainer() {
  const [tab, setTab] = useState('about');
  const [isPending, startTransition] = useTransition();

  function selectTab(next) {
    startTransition(() => setTab(next));   // interruptible
  }

  return (
    <>
      <TabButton onClick={() => selectTab('about')}>About</TabButton>
      <TabButton onClick={() => selectTab('posts')}>Posts (slow)</TabButton>
      <section aria-busy={isPending}>
        {tab === 'about' ? <About /> : <Posts />}
      </section>
    </>
  );
}

// Async Action (React 19)
function UpdateName() {
  const [name, setName] = useState('');
  const [error, setError] = useState(null);
  const [isPending, startTransition] = useTransition();

  const submit = () => startTransition(async () => {
    const err = await updateName(name);
    startTransition(() => {
      if (err) setError(err);
      else redirect('/profile');
    });
  });

  return (
    <>
      <input value={name} onChange={(e) => setName(e.target.value)} />
      <button onClick={submit} disabled={isPending}>Update</button>
      {error && <p role="alert">{error}</p>}
    </>
  );
}
```

URGENT VS TRANSITION

UPDATE	KIND
Typing in an input	urgent
Clicking a toggle	urgent
Switching tabs	transition
Filtering a big list	transition
Router navigation	transition
Form submit (Actions)	transition

APIS

API	GIVES
`useTransition()`	`[isPending, start]`
`startTransition(fn)`	no pending flag
`useDeferredValue(v)`	lagging copy of a value
`useOptimistic`	temporary state during an action

⚠ COMMON MISTAKES

- Wrapping a controlled input's `setValue` in a transition
- Expecting `setTimeout` callbacks inside `startTransition` to be transitions
- Forgetting the inner `startTransition` for updates after `await`

✓ TIPS & BEST PRACTICES

- Use transitions before reaching for debouncing
- Show pending state near the thing that is changing
- Routers (React Router, Next.js) already wrap navigation in transitions

QUICK SUMMARY

- `[isPending, startTransition]`
- Non-urgent, interruptible updates
- Async functions = Actions (19)
- Never for input values

useDeferredValue

deferred value · initialValue · stale indicator · vs debounce / throttle · pairing with memo

A VALUE THAT LAGS BEHIND

`useDeferredValue(value)` returns a version of `value` that may **lag behind** during heavy rendering. On an update, React first re-renders with the old deferred value (fast), then renders again in the background with the new one, and discards that work if `value` changes again. It's ideal for keeping an input responsive while a slow list or chart updates.

NOTES

- Works only if the slow child is wrapped in `memo` (or compiled by the React Compiler)
- No fixed delay: it adapts to the device's speed
- `value !== deferred` tells you the shown content is stale
- `initialValue` (React 19) lets the first render use a cheap placeholder

♿ ACCESSIBILITY

- Mark stale results visually (dimmed) and with `aria-busy` so users don't act on outdated data

VERSION NOTES

- React 18 introduced `useDeferredValue`; React 19 added the `initialValue` argument
- Deferred renders that suspend show the old content, not the fallback

CODE EXPLAINED

→ Typing updates `query` immediately, so the input never lags

→ `SlowList` receives `deferredQuery`, which updates in the background

→ `memo` lets React skip `SlowList` while the deferred value is unchanged

→ `isStale` dims the list until it catches up

→ `initialValue []` renders an empty chart first, then the real data

RESPONSIVE SEARCH

```
import { memo, useDeferredValue, useState } from 'react';

function Search() {
  const [query, setQuery] = useState('');
  const deferredQuery = useDeferredValue(query);
  const isStale = query !== deferredQuery;

  return (
    <>
      <input
        value={query}
        onChange={(e) => setQuery(e.target.value)}
        placeholder="Search 10,000 items"
      />
      <div aria-busy={isStale}
        style={{ opacity: isStale ? 0.5 : 1 }}>
        <SlowList query={deferredQuery} />
      </div>
    </>
  );
}

const SlowList = memo(function SlowList({ query }) {
  const items = filterItems(allItems, query);   // expensive
  return (
    <ul>{items.map((i) => <li key={i.id}>{i.name}</li>)}</ul>
  );
});

// Cheap first render, full one after
function Dashboard({ data }) {
  const deferredData = useDeferredValue(data, []);
  return <Chart data={deferredData} />;
}
```

DEFERRED VS DEBOUNCE

USEDEFERREDVALUE	DEBOUNCE
No fixed delay	Fixed delay (ms)
Interruptible rendering	Delays the update itself
Adapts to device speed	Same delay everywhere
For slow rendering	For slow network calls

DEFERRED VS TRANSITION

USEDEFERREDVALUE	USETRANSITION
Wraps a value	Wraps a state update
Use when you don't own the setter	Use when you call the setter
Pending: `v !== deferred`	Pending: `isPending`

⚠ COMMON MISTAKES

- Forgetting `memo` on the slow child, so nothing is skipped
- Using it to throttle network requests (use debounce)
- Passing a new object each render as the value

✓ TIPS & BEST PRACTICES

- Combine with a debounced fetch for search-as-you-type
- Dim stale content instead of showing a spinner
- Profile first: if the list is fast, you don't need it

QUICK SUMMARY

- Lagging copy of a value
- Keeps inputs responsive
- Needs `memo` on the slow child
- `initialValue` in React 19

useOptimistic

SHOW THE RESULT EARLY

`useOptimistic(state, updateFn)` shows a **temporary** value while an async Action is running. You call `addOptimistic(input)` at the start of the action; React renders the optimistic value immediately, and when the action finishes it **switches back** to the real `state` (which by then includes the server result). If the action fails, the optimistic value simply disappears.

NOTES

- `updateFn(currentState, optimisticValue)` returns the optimistic state
- Call `addOptimistic` inside an Action or `startTransition`
- The real state must be updated by the action for the item to stay
- Mark optimistic items (`sending: true`) so users can see they're pending

♿ ACCESSIBILITY

- Mark pending items with text such as *Sending…*, not only reduced opacity
- If the action fails, announce the failure; silently removing the item is confusing

VERSION NOTES

- `useOptimistic` is new in React 19
- TanStack Query offers its own optimistic update pattern (sheet 78)

CODE EXPLAINED

→ `optimistic` equals `messages` except while an action is running

→ `addOptimistic(text)` runs the update function and renders right away

→ When `sendAction` finishes, React drops optimistic state and shows `messages`

→ On failure, the real state was never updated, so the temp message disappears

→ `sending: true` lets the UI label the pending item

OPTIMISTIC MESSAGES

```jsx
import { useOptimistic, useState } from 'react';

function Thread() {
  const [messages, setMessages] = useState([
    { id: 1, text: 'Hi!', sending: false },
  ]);

  const [optimistic, addOptimistic] = useOptimistic(
    messages,
    (state, text) => [
      ...state,
      { id: `temp-${state.length}`, text, sending: true },
    ],
  );

  async function sendAction(formData) {
    const text = formData.get('message');
    addOptimistic(text);                  // shows instantly
    try {
      const saved = await api.send(text);
      setMessages((m) => [...m, saved]); // real state
    } catch {
      toast('Message failed to send');   // optimistic item vanishes
    }
  }

  return (
    <>
      <ul>
        {optimistic.map((m) => (
          <li key={m.id}>
            {m.text} {m.sending && <small>(Sending…)</small>}
          </li>
        ))}
      </ul>
      <form action={sendAction}>
        <input name="message" required />
        <button>Send</button>
      </form>
    </>
  );
}
```

USEOPTIMISTIC

PART	MEANING
`state`	source of truth
`updateFn(cur, value)`	build optimistic state
`optimistic`	value to render
`addOptimistic(value)`	apply during an action

GOOD FITS

UI	OPTIMISTIC VALUE
`Like button`	count + 1, liked
`Chat message`	appended with *sending*
`Todo toggle`	flipped checkbox
`Rename`	new title
`Delete`	item hidden

⚠ COMMON MISTAKES

- Calling `addOptimistic` outside an action or transition
- Forgetting to update the real state, so the item vanishes on success
- Hiding failures instead of telling the user

✓ TIPS & BEST PRACTICES

- Generate stable temp IDs and replace them with server IDs
- Keep optimistic logic pure and quick
- Use for low-risk actions; confirm destructive ones first

QUICK SUMMARY

- Temporary state during an action
- `addOptimistic(v)` at the start
- Reverts to real state when done
- Label pending items

Activity

HIDE WITHOUT LOSING STATE

`<Activity mode="visible" | "hidden">` hides part of the UI while **keeping its state and DOM**. Hidden content gets `display: none`, its effects are cleaned up, and its updates run at low priority. Showing it again restores everything instantly. Use it for tabs, sidebars and back-navigation where users expect to return to where they were, and to **pre-render** content they're likely to open next.

NOTES

- Hidden trees unmount effects (cleanup runs) but keep state
- Effects re-run when the tree becomes visible again
- Hidden trees still re-render on prop changes, at low priority
- DOM-only state (video playback, input text) is preserved because the DOM stays

♿ ACCESSIBILITY

- `display: none` removes hidden content from the accessibility tree
- Move focus into newly shown panels the same way you would after mounting

VERSION NOTES

- `<Activity>` is stable in React 19.2 (October 2025); earlier canaries called it `<Offscreen>`
- Supported modes: `visible` and `hidden`

CODE EXPLAINED

- → Each tab stays mounted; only its visibility changes
- → Switching away runs effect cleanups (stopping subscriptions, timers)
- → Typed text in `ContactForm` survives switching tabs
- → A hidden `Checkout` can load code and data before the user clicks
- → Conditional rendering would destroy the form's state

TABS THAT REMEMBER

```jsx
import { Activity, useState } from 'react';

function App() {
  const [tab, setTab] = useState('home');

  return (
    <>
      <nav>
        <button onClick={() => setTab('home')}>Home</button>
        <button onClick={() => setTab('posts')}>Posts</button>
        <button onClick={() => setTab('contact')}>Contact</button>
      </nav>

      <Activity mode={tab === 'home' ? 'visible' : 'hidden'}>
        <Home />
      </Activity>
      <Activity mode={tab === 'posts' ? 'visible' : 'hidden'}>
        <Posts />          {/* keeps scroll + filters */}
      </Activity>
      <Activity mode={tab === 'contact' ? 'visible' : 'hidden'}>
        <ContactForm />    {/* keeps typed text */}
      </Activity>
    </>
  );
}

// Pre-render the likely next screen
<Activity mode="hidden">
  <Checkout />             {/* loads data + code early */}
</Activity>

// Compare: conditional rendering loses state
{tab === 'contact' && <ContactForm />}
```

HIDDEN VS UNMOUNTED

ACTIVITY HIDDEN	CONDITIONAL UNMOUNT
State kept	State lost
DOM kept (`display: none`)	DOM removed
Effects cleaned up	Effects cleaned up
Low-priority re-renders	No renders
Instant re-show	Full remount

USE CASES

CASE	BENEFIT
Tabs	keep scroll, inputs
Sidebar / drawer	instant toggle
Back navigation	restore previous page
Pre-rendering	faster next screen
Modals with forms	keep drafts

⚠ COMMON MISTAKES

- Hiding heavy trees forever and paying their memory cost
- Effects that don't clean up, so hidden trees keep side effects alive
- Expecting hidden content to stay in the accessibility tree

✓ TIPS & BEST PRACTICES

- Use for a few high-value screens, not every conditional
- Make sure every effect has a correct cleanup
- Pair with Suspense to pre-load data for hidden screens

QUICK SUMMARY

- `<Activity mode>` hides, keeps state
- Effects clean up while hidden
- Great for tabs and back nav
- Stable since 19.2

View Transitions

ANIMATED UI CHANGES

`<ViewTransition>` animates elements when they change inside a **transition**, using the browser's View Transitions API. Wrap the part of the UI you want to animate; when an update happens in `startTransition`, `useDeferredValue` or Suspense reveals, React captures the before and after and cross-fades by default. Give elements on two screens the same `name` for **shared-element** animations, and style everything with CSS.

NOTES

- Only updates inside transitions (and Suspense reveals) animate; urgent updates don't
- Props `enter`, `exit`, `update`, `share` pick CSS classes for each kind of change
- `name` pairs an element that disappears with one that appears: a shared transition
- `addTransitionType('forward')` lets CSS choose animations by navigation direction

♿ ACCESSIBILITY

- Respect `prefers-reduced-motion`: disable or shorten view-transition animations in CSS
- Animations must not move focus or hide content longer than necessary

VERSION NOTES

- `<ViewTransition>` and `addTransitionType` are stable in React 19.3 (September 2026)
- Requires browser support for the View Transitions API; unsupported browsers skip the animation

CODE EXPLAINED

→ `startTransition` makes the change eligible for animation

→ Matching `name` s morph the thumbnail into the hero image

→ `addTransitionType('open')` tags the transition for direction-specific CSS

→ `enter` / `exit` classes target the browser's view-transition pseudo-elements

→ The reduced-motion query turns animations off

VIEWTRANSITION

```jsx
import {
  ViewTransition, startTransition, addTransitionType,
} from 'react';

function Gallery({ photos }) {
  const [selected, setSelected] = useState(null);

  function open(photo) {
    startTransition(() => {
      addTransitionType('open');
      setSelected(photo);
    });
  }

  return selected ? (
    <ViewTransition name={`photo-${selected.id}`}>
      <img className="hero" src={selected.src} alt={selected.alt} />
    </ViewTransition>
  ) : (
    <ul>
      {photos.map((p) => (
        <li key={p.id}>
          <ViewTransition name={`photo-${p.id}`}>
            <img src={p.thumb} alt={p.alt} onClick={() => open(p)} />
          </ViewTransition>
        </li>
      ))}
    </ul>
  );
}

<ViewTransition enter="slide-in" exit="slide-out">
  {show && <Panel />}
</ViewTransition>

/* CSS */
::view-transition-old(.slide-out) { animation: out 200ms; }
::view-transition-new(.slide-in) { animation: in 200ms; }
@media (prefers-reduced-motion: reduce) {
  ::view-transition-group(*) { animation: none; }
}
```

ACTIVATION

TRIGGER	ANIMATION
`enter`	element mounts in a transition
`exit`	element unmounts in a transition
`update`	element's DOM changes
`share`	same `name` leaves + appears
`default`	cross-fade

ANIMATES?

UPDATE VIA	ANIMATES
`startTransition`	✓
`useDeferredValue`	✓
`Suspense reveal`	✓
`Plain setState`	✗
`Router navigation (transition)`	✓

⚠ COMMON MISTAKES

- Calling `setState` outside a transition and expecting animation
- Reusing the same `name` on two visible elements at once
- Long animations that delay interaction

✓ TIPS & BEST PRACTICES

- Start with the default cross-fade, then add named shared elements
- Keep animations under ~300ms
- Use transition types for forward/back navigation styles

QUICK SUMMARY

- `<ViewTransition>` animates transitions
- `name` = shared element
- Style with `::view-transition-*`
- Stable in 19.3

Document Metadata & Resources

HEAD TAGS FROM COMPONENTS

In React 19, components can render `<title>`, `<meta>` and `<link>` anywhere in the tree, and React **hoists** them into `<head>`. Stylesheets rendered with a `precedence` prop are deduplicated, inserted in order, and Suspense waits for them before revealing content. `react-dom` also exports **resource hints** such as `preload`, `preinit` and `prefetchDNS` to start loading fonts, scripts and styles early.

NOTES

- Works in client rendering, SSR and Server Components
- `<link rel="stylesheet" precedence="default">` is deduplicated by `href`
- `<script async src>` anywhere in the tree is deduplicated and loaded once
- Libraries like `react-helmet` are mostly unnecessary for basic metadata

♿ ACCESSIBILITY

- Set a unique, descriptive `<title>` per page; it's the first thing screen readers announce after navigation

VERSION NOTES

- Metadata hoisting, stylesheet precedence and resource APIs are new in React 19
- Frameworks (Next.js `metadata` export) may provide their own higher-level APIs

CODE EXPLAINED

→ `<title>` and `<meta>` inside `BlogPost` end up in `<head>`

→ Template literal makes `<title>` receive a single string child

→ `precedence` lets React order and dedupe the stylesheet

→ Rendering `<CodeBlock>` many times still loads `prism.js` once

→ Resource APIs start downloads before components that need them render

METADATA, STYLES AND PRELOADING

```jsx
import { preload, preinit, prefetchDNS, preconnect } from 'react-dom';

function BlogPost({ post }) {
  return (
    <article>
      <title>{`${post.title} | My Blog`}</title>
      <meta name="description" content={post.summary} />
      <meta name="author" content={post.author} />
      <link rel="canonical" href={post.url} />

      <h1>{post.title}</h1>
      <p>{post.body}</p>
    </article>
  );
}

function CodeBlock({ code }) {
  return (
    <>
      <link rel="stylesheet" href="/prism.css" precedence="default" />
      <script async src="/prism.js" />
      <pre>{code}</pre>
    </>
  );
}

function App() {
  preinit('/analytics.js', { as: 'script' }); // load + run
  preload('/fonts/inter.woff2', { as: 'font', type: 'font/woff2',
    crossOrigin: 'anonymous' });
  prefetchDNS('https://api.example.com');
  preconnect('https://cdn.example.com');
  return <Routes />;
}
```

HOISTED ELEMENTS

ELEMENT	NOTES
`<title>`	one string child
`<meta>`	name/property/charset
`<link rel="canonical">`	any `rel` except stylesheet without precedence
`<link rel="stylesheet" precedence>`	ordered, deduped
`<script async>`	deduped

RESOURCE APIS

FUNCTION	DOES
`prefetchDNS(url)`	DNS lookup early
`preconnect(url)`	open connection
`preload(url, {as})`	download early
`preloadModule(url)`	download ES module
`preinit(url, {as})`	download + execute/apply
`preinitModule(url)`	download + run module

⚠ COMMON MISTAKES

- `<title>Hi {name}</title>` producing multiple children (use a template literal)
- Two components rendering different `<title>` s at once
- Stylesheets without `precedence` (not hoisted or deduplicated)

✓ TIPS & BEST PRACTICES

- Render the title in each route's top component
- Preload fonts used above the fold
- Let your framework's metadata API handle Open Graph images and sitemaps

QUICK SUMMARY

- `<title>` / `<meta>` / `<link>` hoist to head
- `precedence` for stylesheets
- `<script async>` dedupes
- `preload` / `preinit` for resources

Error Boundaries

CATCHING RENDER ERRORS

An **error boundary** catches errors thrown while rendering the components below it, shows a fallback, and keeps the rest of the app running. Without one, a render error unmounts the whole root. Boundaries are **class components** that implement `static getDerivedStateFromError` (to show the fallback) and usually `componentDidCatch` (to log). Most apps use the `react-error-boundary` package instead of writing the class.

NOTES

- Boundaries catch errors in render, lifecycle methods, effects-during-commit and `use()` rejections
- They don't catch errors in event handlers, async code outside React, or the boundary itself
- Rejected form Actions and transitions are also caught
- Place boundaries around routes and independent widgets

♿ ACCESSIBILITY

- Make the fallback a clear message with `role="alert"` and a button to retry or go back
- Move focus to the fallback heading so keyboard users know the content changed

VERSION NOTES

- There is still no hook equivalent in React 19; boundaries remain class-based
- React 19 logs caught errors once (with component stack) instead of re-throwing them

CODE EXPLAINED

→ `getDerivedStateFromError` stores the error so `render` shows the fallback

→ `componentDidCatch` reports to your logging service

→ `resetErrorBoundary` clears the error and re-renders the children

→ `resetKeys` resets automatically when `userId` changes

→ Handler errors don't reach boundaries, so catch them and set state

CLASS BOUNDARY AND LIBRARY

```jsx
import { Component } from 'react';

class ErrorBoundary extends Component {
  state = { error: null };

  static getDerivedStateFromError(error) {
    return { error };                 // render fallback next
  }

  componentDidCatch(error, info) {
    logError(error, info.componentStack);
  }

  render() {
    if (this.state.error) return this.props.fallback;
    return this.props.children;
  }
}

// react-error-boundary
import { ErrorBoundary as EB } from 'react-error-boundary';

function Fallback({ error, resetErrorBoundary }) {
  return (
    <div role="alert">
      <h2>Something went wrong</h2>
      <pre>{error.message}</pre>
      <button onClick={resetErrorBoundary}>Try again</button>
    </div>
  );
}

<EB FallbackComponent={Fallback}
  onReset={() => queryClient.resetQueries()}
  resetKeys={[userId]}>
  <UserPanel userId={userId} />
</EB>

// Errors in handlers: catch them yourself
async function handleSave() {
  try { await save(); } catch (e) { setError(e); }
}
```

CAUGHT?

ERROR IN	CAUGHT BY BOUNDARY
`Render`	✓
`useEffect / lifecycle`	✓
`use(promise) rejection`	✓
`Form Action / transition`	✓
`Event handler`	✗ (try/catch)
`setTimeout callback`	✗
`The boundary itself`	✗ (parent boundary)

REACT-ERROR-BOUNDARY

PROP / API	USE
`fallback`	static element
`FallbackComponent`	gets `error`, `resetErrorBoundary`
`onError(error, info)`	log
`onReset`	clean up before retry
`resetKeys`	auto-reset on change
`useErrorBoundary()`	throw handler errors into it

⚠ COMMON MISTAKES

- One boundary at the root so any error blanks the whole app
- Expecting boundaries to catch errors in click handlers
- Showing raw stack traces to users in production

✓ TIPS & BEST PRACTICES

- Wrap each route and each independent widget
- Pair every Suspense boundary with an error boundary
- Use `showBoundary` from `useErrorBoundary` to surface async errors

QUICK SUMMARY

- Boundaries catch render-phase errors
- Class-only or `react-error-boundary`
- Not event handlers or async code
- Offer a retry

Sheet 55

Root Error Handling & StrictMode

onCaughtError · onUncaughtError · onRecoverableError · StrictMode checks · console warnings · owner stacks

ERRORS AT THE ROOT AND DEV CHECKS

React 19 lets you handle errors where the root is created. `createRoot` and `hydrateRoot` accept **`onCaughtError`** (an error boundary caught it), **`onUncaughtError`** (nothing caught it, so the tree unmounted) and **`onRecoverableError`** (React recovered automatically, e.g. after a hydration mismatch). In development, **StrictMode** double-runs renders, effects and ref callbacks to expose bugs, and **owner stacks** show which component rendered the one that failed.

NOTES

- Root callbacks receive `(error, errorInfo)` with `componentStack`
- Uncaught errors unmount the entire root; boundaries prevent that
- Recoverable errors include hydration mismatches that React fixed by client rendering
- `captureOwnerStack()` returns the owner stack in development for custom logging

♿ ACCESSIBILITY

- When an uncaught error unmounts the app, render a minimal static message outside React so the page isn't blank

VERSION NOTES

- Root error callbacks are new in React 19
- Owner stacks and `captureOwnerStack` shipped in React 19.1
- React 19 reports errors once instead of logging and re-throwing

CODE EXPLAINED

- → `onCaughtError` fires after a boundary handled the error
- → `onUncaughtError` is your last chance to report and show a static message
- → `onRecoverableError` often signals hydration problems worth fixing
- → `StrictMode` wraps the app to run development checks
- → `captureOwnerStack` is only available in development builds

ROOT CALLBACKS AND STRICTMODE

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';

const root = createRoot(document.getElementById('root'), {
  onCaughtError(error, info) {
    // a boundary rendered a fallback
    reporter.warn(error, { stack: info.componentStack });
  },
  onUncaughtError(error, info) {
    // no boundary: the tree is gone
    reporter.error(error, { stack: info.componentStack });
    document.getElementById('fatal').hidden = false;
  },
  onRecoverableError(error, info) {
    // e.g. hydration mismatch recovered by client render
    reporter.info(error, { cause: error.cause });
  },
});

root.render(
  <StrictMode>
    <App />
  </StrictMode>,
);

// Dev-only owner stack in custom logging
import { captureOwnerStack } from 'react';
if (process.env.NODE_ENV !== 'production') {
  console.log(captureOwnerStack());
}

// StrictMode in dev runs, on mount:
//   render x2, effect setup -> cleanup -> setup,
//   ref callback attach -> detach -> attach
```

ROOT CALLBACKS

CALLBACK	WHEN
`onCaughtError`	boundary caught it
`onUncaughtError`	no boundary; root unmounted
`onRecoverableError`	React recovered automatically
`identifierPrefix`	(option) prefix for `useId`

STRICTMODE EXPOSES

SYMPTOM IN DEV	REAL BUG
`Logs twice`	impure render
`Two connections open`	missing effect cleanup
`Listener added twice`	missing ref cleanup
`Warning about API`	deprecated usage
`State changes on remount`	not resilient to reuse

⚠ COMMON MISTAKES

- Treating StrictMode double calls as a React bug
- Ignoring recoverable hydration errors
- Not reporting errors to a service in production

✓ TIPS & BEST PRACTICES

- Wire root callbacks to Sentry or a similar service
- Fix the cause of each StrictMode symptom instead of guarding with flags
- Keep a static `#fatal` message in `index.html` for uncaught errors

QUICK SUMMARY

- `onCaughtError` / `onUncaughtError`
- `onRecoverableError` for hydration
- StrictMode double-runs in dev
- Owner stacks show who rendered what

React DevTools

Components tab · editing props & state · Profiler tab · highlight updates · owner tree · Compiler badge

INSPECTING COMPONENTS

React DevTools is a browser extension (Chrome, Firefox, Edge) that adds **Components** and **Profiler** tabs. Components shows the tree as React sees it: select a component to read and edit its props, state, hooks and context, jump to its source, and see its **owner** (the component that rendered it). Profiler records renders so you can see what rendered, why, and how long it took.

NOTES

- `$r` in the console refers to the selected component
- Hook values are listed in call order; `useDebugValue` labels custom hooks
- Enable *Highlight updates when components render* to spot extra renders
- Standalone `react-devtools` (npm) debugs React Native and Safari

♿ ACCESSIBILITY

- Use the browser's Accessibility panel alongside React DevTools to check the rendered roles and names

VERSION NOTES

- DevTools shows a **Memo** ✨ badge on components optimized by the React Compiler
- React 19.2 adds Performance-panel tracks (*Scheduler* and *Components*) in Chrome DevTools

CODE EXPLAINED

- → `$r` lets you poke at the selected component in the console
- → The second argument to `useDebugValue` formats the label lazily
- → Named functions inside `memo` show as `Row` instead of *Anonymous*
- → `<Profiler onRender>` logs timing for one subtree
- → *Record why each component rendered* explains each render in the Profiler

DEBUGGING WORKFLOW

```
// 1. Select a component in the Components tab, then:
$r                    // selected component instance/props
$r.props              // (class components)

// 2. Label custom hooks
function useCart() {
  const cart = useSyncExternalStore(store.subscribe, store.get);
  useDebugValue(cart, (c) => `${c.items.length} items`);
  return cart;
}

// 3. Name components so they show up clearly
const Row = memo(function Row(props) { /* … */ });

// 4. Measure in code
import { Profiler } from 'react';
<Profiler id="Feed" onRender={(id, phase, ms) =>
  console.log(id, phase, ms.toFixed(1))}>
  <Feed />
</Profiler>

// 5. Log why a render happened (dev)
useEffect(() => {
  console.log('render', { props });
});

// Profiler tab settings:
//   ✓ Record why each component rendered
//   ✓ Highlight updates when components render
```

COMPONENTS TAB

FEATURE	USE
`Props / state / hooks`	inspect and edit live
`Rendered by (owners)`	trace who rendered it
`Source link`	jump to code
`Suspend toggle`	preview Suspense fallback
`Error toggle`	preview error boundary

PROFILER TAB

VIEW	SHOWS
`Flame graph`	each commit's component tree
`Ranked`	slowest components
`Why did this render?`	props/state/hooks/parent
`Commit list`	each commit's duration
`Timeline`	transitions, Suspense, scheduling

⚠ COMMON MISTAKES

- Profiling the development build and trusting absolute timings
- Anonymous arrow components that show as *Anonymous* in the tree
- Leaving `console.log` render tracing in committed code

✓ TIPS & BEST PRACTICES

- Use the profiling build of `react-dom` for realistic numbers
- Toggle Suspense and error states from DevTools to test fallbacks
- Check the Compiler badge to confirm components are optimized

QUICK SUMMARY

- Components tab: props, state, owners
- Profiler tab: what rendered and why
- `useDebugValue` labels hooks
- Compiler badge shows auto-memo

How Rendering Works

TRIGGER, RENDER, COMMIT

Every screen update goes through three steps. A **trigger** (initial mount or a state update) asks React to render. In the **render** phase, React calls your components to compute new JSX and **reconciles** it with the previous output. In the **commit** phase, it applies the minimal DOM changes and runs effects. Rendering a component doesn't mean the DOM changed; React only touches what differs.

NOTES

- A state change re-renders that component and, by default, all its children
- Parents re-rendering is the most common cause of child re-renders
- Context changes re-render every reader
- React bails out when a setter receives a value equal (`Object.is`) to the current one

♿ ACCESSIBILITY

- Committing only changed nodes keeps focus and selection stable during updates

VERSION NOTES

- Concurrent rendering (React 18+) can pause and resume the render phase; commit is always synchronous
- The React Compiler skips re-rendering children whose props didn't change

CODE EXPLAINED

- → Clicking re-renders `App` and every child, even ones with unchanged props
- → React patches only the `className` attribute on the `<input>`
- → Switching element types replaces the node and loses its state
- → Moving state into `Counter` keeps siblings from re-rendering
- → `children` elements come from the parent, so `ScrollTracker` 's state changes skip them

WHAT RE-RENDERS

```
function App() {
  const [count, setCount] = useState(0);
  console.log('App renders');
  return (
    <>
      <button onClick={() => setCount(count + 1)}>{count}</button>
      <Header />            {/* re-renders: parent rendered */}
      <List items={items} />{/* re-renders too */}
    </>
  );
}

// Same type + position -> React updates in place
<input className={dark ? 'dark' : 'light'} />
// Only the className attribute is changed in the DOM

// Different type -> unmount old, mount new
{isEditing ? <input /> : <span />}

// Moving state down limits re-renders
function App2() {
  return (
    <>
      <Counter />           {/* only this re-renders */}
      <Header />
      <List items={items} />
    </>
  );
}

// Passing children keeps them from re-rendering
function ScrollTracker({ children }) {
  const [y, setY] = useState(0);
  // children were created by the parent; not re-rendered
  return <div onScroll={(e) => setY(e.currentTarget.scrollTop)}>
    {children}
  </div>;
}
```

RE-RENDER CAUSES

CAUSE	SCOPE
`Own state change`	component + children
`Parent re-render`	component + children
`Context value change`	every reader
`Store subscription change`	subscribed components
`key change`	full remount

PHASES

PHASE	DOES
`Trigger`	`setState`, `root.render`
`Render`	call components, diff output
`Commit`	apply DOM changes
`Layout effects`	run before paint
`Paint + effects`	browser paints, `useEffect` runs

⚠ COMMON MISTAKES

- Assuming a re-render means slow DOM updates
- Optimizing before measuring
- Lifting state higher than needed and re-rendering large trees

✓ TIPS & BEST PRACTICES

- Keep state as close as possible to where it's used
- Pass expensive subtrees as `children`
- Measure with the Profiler before adding `memo`

QUICK SUMMARY

- Trigger → render → commit
- Children re-render with parents
- DOM changes are minimal
- Move state down, lift content up

memo

React.memo · shallow comparison · custom arePropsEqual · props that break memo · children props

SKIPPING CHILD RENDERS

`memo(Component)` returns a version of a component that **skips re-rendering** when its props are unchanged, compared with `Object.is` per prop (a shallow comparison). It helps when a component renders often with the same props and its render is expensive. It does nothing if a prop is a new object, array or function each render, which is why it's often paired with `useMemo` and `useCallback`.

NOTES

- State changes and context changes inside the memoized component still re-render it
- `children` JSX is a new object every render, which defeats `memo`
- A second argument `arePropsEqual(prev, next)` customizes the comparison
- With the React Compiler, most manual `memo` calls are unnecessary

♿ ACCESSIBILITY

- Memoization doesn't affect output, but stale custom comparisons can leave ARIA attributes out of date

VERSION NOTES

- `memo` is unchanged in React 19
- React Compiler 1.0 memoizes components and JSX automatically

CODE EXPLAINED

- → `Chart` re-renders only when `data`, `options` or `onSelect` change
- → Typing in the filter re-renders `Dashboard` but not `Chart`
- → An inline `{ grid: true }` would be a new object every time
- → `useCallback` keeps `onSelect` the same function
- → A custom comparator must check every prop that affects output

MEMO AND ITS PITFALLS

```jsx
import { memo, useCallback, useMemo, useState } from 'react';

const Chart = memo(function Chart({ data, options, onSelect }) {
  return <svg>{/* expensive drawing */}</svg>;
});

function Dashboard({ rows }) {
  const [filter, setFilter] = useState('');

  // ✗ New object/function each render: memo never skips
  // <Chart data={rows} options={{ grid: true }}
  //   onSelect={(p) => select(p)} />

  // ✓ Stable props
  const options = useMemo(() => ({ grid: true }), []);
  const onSelect = useCallback((p) => select(p), []);

  return (
    <>
      <input value={filter}
        onChange={(e) => setFilter(e.target.value)} />
      <Chart data={rows} options={options} onSelect={onSelect} />
    </>
  );
}

// Custom comparison: only re-render when id changes
const Row = memo(
  function Row({ item }) { return <li>{item.name}</li>; },
  (prev, next) => prev.item.id === next.item.id
              && prev.item.name === next.item.name,
);
```

MEMO WORKS WHEN

CONDITION	NEEDED
Props primitive or stable	✓
Render is expensive	✓
Re-renders often with same props	✓
No children JSX passed	✓
Compiler not enabled	(else automatic)

BREAKS MEMO

PROP	FIX
`{...}` literal	useMemo / hoist constant
`[...]` literal	useMemo
`() => …` inline	useCallback
`<Child />` as children	memoize the element
New context value	memoize provider value

⚠ COMMON MISTAKES

- Wrapping every component in `memo` by default
- Custom comparators that forget a prop and show stale UI
- Expecting `memo` to stop context-driven re-renders

✓ TIPS & BEST PRACTICES

- Enable the React Compiler instead of hand-tuning
- Measure with the Profiler before and after
- Fix structure first: move state down, pass children

QUICK SUMMARY

- `memo` skips equal-prop renders
- Shallow `Object.is` per prop
- Needs stable object/function props
- Compiler makes it automatic

useMemo & useCallback

caching calculations · stable callbacks · dependency arrays · when it's worth it · measuring first

CACHING VALUES AND FUNCTIONS

`useMemo(fn, deps)` caches the **result** of a calculation and recomputes only when a dependency changes. `useCallback(fn, deps)` caches the **function itself**. Use them for expensive calculations, for props passed to `memo` components, and for values used as effect dependencies. They're performance hints, not guarantees: React may discard caches, so code must still be correct without them.

NOTES

- `useCallback(fn, deps)` equals `useMemo(() => fn, deps)`
- Dependencies follow the same rules as effects; the linter checks them
- *Expensive* means roughly 1ms or more; measure with `console.time`
- The React Compiler inserts equivalent memoization automatically

♿ ACCESSIBILITY

- Not an accessibility concern directly; faster renders keep input and focus responsive

VERSION NOTES

- Unchanged in React 19; with React Compiler 1.0, most new code doesn't need them
- The compiler's lint rules warn when manual memoization conflicts with its own

CODE EXPLAINED

→ `visible` keeps its value while unrelated state changes

→ `console.time` shows whether the work is worth caching

→ `handleAdd` keeps the same identity so a memoized `Rows` can skip renders

→ `options` stays stable, so the effect runs only when `query` changes

→ Simple arithmetic costs less than the hook itself

USEMEMO AND USECALLBACK

```jsx
import { useCallback, useMemo, useState } from 'react';

function ProductTable({ products, query, sortBy }) {
  // Expensive: only recompute when inputs change
  const visible = useMemo(() => {
    console.time('filter');
    const result = products
      .filter((p) => p.name.toLowerCase().includes(query))
      .toSorted((a, b) => a[sortBy] - b[sortBy]);
    console.timeEnd('filter');
    return result;
  }, [products, query, sortBy]);

  // Stable function for a memoized child
  const handleAdd = useCallback((id) => {
    cart.add(id);
  }, []);

  // Stable object for an effect dependency
  const options = useMemo(() => ({ query, limit: 50 }), [query]);
  useEffect(() => { track('search', options); }, [options]);

  return <Rows items={visible} onAdd={handleAdd} />;
}

// ✗ Not worth it: cheap calculation
const total = useMemo(() => a + b, [a, b]);

// ✓ Just compute it
const total2 = a + b;
```

WHEN TO USE

SITUATION	USE
Slow calculation (≥1ms)	useMemo
Object prop to memo child	useMemo
Function prop to memo child	useCallback
Object/function in effect deps	memoize or move inside
Cheap expression	plain variable

COMPARISON

USEMEMO	USECALLBACK
Caches a value	Caches a function
Calls fn	Returns fn
useMemo(() => x, d)	useCallback(x, d)
Skip expensive work	Keep identity stable

⚠ COMMON MISTAKES

- Missing dependencies, which cause stale values
- Memoizing everything and making code harder to read
- Relying on memoization for correctness (e.g. to avoid infinite effects)

✓ TIPS & BEST PRACTICES

- Measure with production builds; dev mode is slower
- Restructure (move state down) before memoizing
- Turn on the React Compiler and delete most manual memos

QUICK SUMMARY

- `useMemo` caches values
- `useCallback` caches functions
- Only for expensive or identity-sensitive cases
- The Compiler automates both

React Compiler

babel-plugin-react-compiler · automatic memoization · Vite setup · 'use no memo' · compiler lint rules · incremental adoption

AUTOMATIC MEMOIZATION

The **React Compiler** is a build-time tool that analyzes components and hooks and inserts memoization automatically, at a finer level than hand-written `useMemo`, `useCallback` and `memo`. It assumes your code follows the **Rules of React** (pure render, no mutation of props or state, Rules of Hooks). Code it can't safely optimize is skipped, not broken. Add it to Vite via the Babel plugin.

NOTES

- Works with React 17, 18 and 19 (17/18 need `react-compiler-runtime`)
- Opt a component out with the `'use no memo'` directive at the top of its body
- Compilation mode `annotation` compiles only functions marked `'use memo'`
- Existing `useMemo` / `useCallback` keep working; the compiler respects them

♿ ACCESSIBILITY

- No direct effect; fewer re-renders keep focus and screen-reader output steadier

VERSION NOTES

- React Compiler 1.0 was released in October 2025
- Next.js, Vite and Expo templates offer it as an option
- Lint rules ship in `eslint-plugin-react-hooks` (formerly `eslint-plugin-react-compiler`)

CODE EXPLAINED

→ The Babel plugin runs inside Vite's React plugin

→ `compilationMode: 'annotation'` lets you adopt it one component at a time

→ Plain code is memoized automatically after compiling

→ `'use no memo'` skips a component that breaks the rules

→ DevTools shows a *Memo* ✨ badge on compiled components

SETUP AND DIRECTIVES

```
# install
npm install -D babel-plugin-react-compiler@latest

// vite.config.ts
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [
    react({
      babel: {
        plugins: [['babel-plugin-react-compiler', {
          // compilationMode: 'annotation',
        }]],
      },
    }),
  ],
});

// Before: manual memoization
const visible = useMemo(() => filter(todos, tab), [todos, tab]);
const onAdd = useCallback((t) => add(t), [add]);

// After: write plain code; the compiler caches it
const visible2 = filter(todos, tab);
const onAdd2 = (t) => add(t);

// Opt out one component
function LegacyWidget() {
  'use no memo';
  // mutates during render; fix later
}

// Opt in (annotation mode)
function Expensive() {
  'use memo';
}
```

REQUIREMENTS

RULE	WHY
`Pure render`	caching assumes same input → same output
`No prop/state mutation`	change detection relies on references
`Rules of Hooks`	static analysis of hook order
`No ref reads in render`	refs aren't tracked
`Lint clean`	lint flags code it will skip

ADOPTION

STEP	DO
1	Enable compiler lint rules
2	Fix reported violations
3	Enable on a directory or annotation mode
4	Check DevTools badges + tests
5	Expand; remove manual memos gradually

⚠ COMMON MISTAKES

- Enabling it on code that mutates during render
- Deleting all manual memoization at once without tests
- Assuming it fixes slow algorithms (it only skips re-work)

✓ TIPS & BEST PRACTICES

- Turn on the lint rules first; they're useful even without the compiler
- Keep `useMemo` where a value must be stable for an external library
- Use `'use no memo'` as a temporary escape hatch with a TODO

QUICK SUMMARY

- Build-time automatic memoization
- Needs Rules of React
- `'use no memo'` to opt out
- 1.0 since Oct 2025

Code Splitting & Lazy Loading

lazy() · dynamic import() · <Suspense> fallback · route-level splitting · preloading · default export requirement

LOADING CODE ON DEMAND

Code splitting breaks the bundle into chunks that load only when needed. `lazy(() => import('./X'))` creates a component whose code downloads the first time it renders; wrap it in <Suspense> to show a fallback while it loads. Split at **routes** first, then at heavy, rarely used widgets such as editors, charts and modals.

NOTES

- `lazy` expects a module with a **default export**
- Declare `lazy` components at module top level, never inside a component
- Start the import early (on hover or route preload) to hide the delay
- Bundlers create a separate chunk for every dynamic `import()`

♿ ACCESSIBILITY

- Give the Suspense fallback accessible loading text
- After a lazy route loads, move focus to its heading

VERSION NOTES

- `lazy` and Suspense for code have been stable since React 16.6
- React Router and Next.js split routes automatically

CODE EXPLAINED

→ Each `lazy` route becomes its own chunk

→ `.then(m => ({ default: m.Editor }))` adapts a named export

→ One Suspense boundary covers both routes

→ Hovering starts the download before the click

→ Calling `lazy` inside a component resets state on every render

LAZY + SUSPENSE

```jsx
import { lazy, Suspense, useState } from 'react';

// Route-level split
const Settings = lazy(() => import('./pages/Settings'));
const Reports = lazy(() => import('./pages/Reports'));

// Named export -> map it to default
const Editor = lazy(() =>
  import('./Editor').then((m) => ({ default: m.Editor })));

function App({ page }) {
  return (
    <Suspense fallback={<p role="status">Loading page…</p>}>
      {page === 'settings' ? <Settings /> : <Reports />}
    </Suspense>
  );
}

// Heavy widget: load only when opened
function Post() {
  const [editing, setEditing] = useState(false);
  return (
    <>
      <button
        onMouseEnter={() => import('./Editor')}   // preload
        onClick={() => setEditing(true)}>
        Edit
      </button>
      {editing && (
        <Suspense fallback={<p>Loading editor…</p>}>
          <Editor />
        </Suspense>
      )}
    </>
  );
}

// ✗ lazy inside a component: new component each render
function Bad() {
  const Chart = lazy(() => import('./Chart'));
  return <Chart />;
}
```

WHERE TO SPLIT

CANDIDATE	WHY
Routes / pages	biggest win
Rich text editor	large dependency
Charts / maps	heavy libraries
Modals and drawers	not shown at first
Admin-only areas	most users never load them

LAZY RULES

RULE	DETAIL
Default export	or map a named one
Top-level declaration	stable identity
Inside <Suspense>	needed for fallback
Errors	caught by error boundary
Preload	call `import()` early

⚠ COMMON MISTAKES

- Calling `lazy` inside a component body
- No error boundary for failed chunk loads (e.g. after a deploy)
- Splitting tiny components and adding network round trips

✓ TIPS & BEST PRACTICES

- Check bundle size with `vite build` output or `rollup-plugin-visualizer`
- Preload the next route on link hover
- Retry failed chunk imports once, then show a reload prompt

QUICK SUMMARY

- `lazy(() => import())`
- Wrap in <Suspense>
- Split routes first
- Preload on hover

Profiling & Large Lists

<Profiler onRender> · flame graph · virtualization · TanStack Virtual · react-window · content-visibility

FINDING AND FIXING SLOW RENDERS

Measure before optimizing. The DevTools **Profiler** records commits and shows which components rendered, why, and how long each took. The `<Profiler>` component logs timing from code. For long lists, the usual fix is **virtualization**: render only the rows in view with **TanStack Virtual** or **react-window**, or let the browser skip off-screen work with CSS `content-visibility: auto`.

NOTES

- Profile production builds; development React is much slower
- Look for components that render often *and* take long
- Virtualization keeps DOM size constant regardless of list length
- `content-visibility: auto` with `contain-intrinsic-size` skips rendering off-screen sections

♿ ACCESSIBILITY

- Virtualized lists remove off-screen rows from the DOM; set `aria-rowcount` / `aria-setsize` and `aria-posinset` so screen readers know the full size
- Browser find-in-page can't see virtualized rows; offer search

VERSION NOTES

- `<Profiler>` API is stable and unchanged in React 19
- React 19.2 adds React tracks to the Chrome Performance panel

CODE EXPLAINED

- → `onRender` warns when a commit takes longer than a frame
- → `useVirtualizer` calculates which rows are visible
- → Only visible rows plus `overscan` are in the DOM
- → `aria-setsize` / `aria-posinset` report the full list size
- → `content-visibility` lets the browser skip off-screen layout and paint

PROFILER AND VIRTUALIZATION

```jsx
import { Profiler, useRef } from 'react';
import { useVirtualizer } from '@tanstack/react-virtual';

function onRender(id, phase, actualDuration, baseDuration) {
  if (actualDuration > 16) {
    console.warn(`${id} ${phase}: ${actualDuration.toFixed(1)}ms`);
  }
}

<Profiler id="Inbox" onRender={onRender}>
  <Inbox />
</Profiler>

function BigList({ rows }) {
  const parentRef = useRef(null);
  const v = useVirtualizer({
    count: rows.length,
    getScrollElement: () => parentRef.current,
    estimateSize: () => 36,
    overscan: 5,
  });

  return (
    <div ref={parentRef} style={{ height: 400, overflow: 'auto' }}>
      <ul role="list"
        style={{ height: v.getTotalSize(), position: 'relative' }}>
        {v.getVirtualItems().map((item) => (
          <li key={item.key}
            aria-setsize={rows.length}
            aria-posinset={item.index + 1}
            style={{ position: 'absolute', top: 0, width: '100%',
              transform: `translateY(${item.start}px)` }}>
            {rows[item.index].name}
          </li>
        ))}
      </ul>
    </div>
  );
}

/* CSS alternative for long pages */
.feed-item { content-visibility: auto;
  contain-intrinsic-size: auto 120px; }
```

ONRENDER ARGS

ARG	MEANING
`id`	Profiler id
`phase`	`mount`, `update`, `nested-update`
`actualDuration`	time this commit
`baseDuration`	time without memoization
`startTime / commitTime`	timestamps

LARGE LIST OPTIONS

OPTION	USE
`TanStack Virtual`	headless, flexible
`react-window`	simple fixed/variable lists
`content-visibility`	long static content
`Pagination / infinite load`	fewer items at once
`useDeferredValue`	keep input responsive

⚠ COMMON MISTAKES

- Optimizing based on development-mode timings
- Virtualizing short lists and adding complexity
- Rendering 10,000 rows and blaming React

✓ TIPS & BEST PRACTICES

- Start with the Profiler's *Ranked* view
- Virtualize lists over a few hundred rows
- Combine virtualization with `memo` on row components

QUICK SUMMARY

- Measure with Profiler first
- `<Profiler onRender>` in code
- Virtualize long lists
- `content-visibility` for long pages

TypeScript: Props & Components

Props type vs interface · ReactNode · optional props & defaults · React.FC · ComponentProps<'button'> · PropsWithChildren

TYPING COMPONENTS

In TypeScript, a component is a function whose **props** parameter has a type. Declare props with a `type` alias or `interface`, mark optional ones with `?`, and give defaults in the destructuring pattern. Use `React.ReactNode` for anything renderable, such as `children`. To accept every attribute of a native element, extend `ComponentProps<'button'>` and spread the rest.

NOTES

- Files containing JSX use the `.tsx` extension
- `React.FC` is optional and no longer adds implicit `children`; many teams skip it
- `PropsWithChildren<P>` adds `children?: ReactNode` to `P`
- Return type is inferred; annotate as `React.JSX.Element` only if you need to

♿ ACCESSIBILITY

- Make accessibility props required in the type (e.g. `alt: string`, `label: string`) so callers can't omit them
- Including `ComponentProps<'button'>` lets callers pass `aria-*` attributes with full type checking

VERSION NOTES

- React 19 types (`@types/react` 19) removed the global `JSX` namespace; use `React.JSX`
- `propTypes` and `defaultProps` for functions are ignored in React 19; TypeScript replaces them

CODE EXPLAINED

- → `CardProps` documents every prop; `?` marks optional ones
- → String literal unions restrict `variant` to known values
- → `ReactNode` accepts strings, elements, arrays, `null` and more
- → `ComponentProps<'button'>` includes `onClick`, `type`, `disabled` and every `aria-*` prop
- → `...rest` forwards all native props, including `ref` in React 19

PROPS TYPES

```tsx
import type { ComponentProps, ReactNode } from 'react';

type CardProps = {
  title: string;
  subtitle?: string;            // optional
  variant?: 'plain' | 'outlined';
  children: ReactNode;
  footer?: ReactNode;
};

export function Card({
  title, subtitle, variant = 'plain', children, footer,
}: CardProps) {
  return (
    <section className={`card ${variant}`}>
      <h2>{title}</h2>
      {subtitle && <p>{subtitle}</p>}
      {children}
      {footer}
    </section>
  );
}

// Extend native element props
type ButtonProps = ComponentProps<'button'> & {
  tone?: 'primary' | 'danger';
};

export function Button({ tone = 'primary', className, ...rest }:
  ButtonProps) {
  return <button className={`btn ${tone} ${className ?? ''}`}
    {...rest} />;
}

<Button tone="danger" type="submit" aria-describedby="hint">
  Delete
</Button>

// Optional FC style
const Badge: React.FC<{ count: number }> = ({ count }) =>
  <span>{count}</span>;
```

COMMON TYPES

TYPE	FOR
`ReactNode`	anything renderable (children)
`React.JSX.Element`	a JSX element
`ReactElement<P>`	element with known props
`ComponentProps<'input'>`	all input props
`ComponentProps<typeof X>`	props of component X
`CSSProperties`	`style` objects

TYPE VS INTERFACE

TYPE	INTERFACE
Unions and intersections	`extends` and merging
`A & B`	`interface A extends B`
Can't be reopened	Declaration merging
Either works for props	Pick one per codebase

⚠ COMMON MISTAKES

- Typing `children` as `JSX.Element` and rejecting strings
- Using the global `JSX` namespace removed in React 19 types
- Forgetting to spread `...rest` after extending native props

✓ TIPS & BEST PRACTICES

- Export props types (`export type CardProps`) for reuse
- Prefer unions of literals over loose `string`
- Use `ComponentProps<typeof X>` to wrap third-party components

QUICK SUMMARY

- `({ a, b }: Props) => …`
- `children: ReactNode`
- `ComponentProps<'tag'>` for natives
- `React.JSX` in 19 types

TypeScript: Hooks

useState<T> · null initial state · useRef<HTMLInputElement>(null) · reducer action unions · typed context guard · custom hook returns

TYPING STATE, REFS AND CONTEXT

Most hooks infer their types: `useState(0)` is `number`. Provide a type argument when the initial value doesn't describe all possible values, such as `null` before data loads or an empty array. DOM refs start as `null` and take the element type. Reducers use a **discriminated union** for actions so each `case` knows its payload. Context typed as `T | null` plus a guard hook avoids non-null assertions.

NOTES

- `useState<User | null>(null)` allows `null` until data arrives
- `useState<string[]>([])` avoids an inferred `never[]`
- `useRef<HTMLInputElement>(null)` gives `RefObject<HTMLInputElement | null>`
- `useRef<number | null>(null)` for mutable values like timer IDs

♿ ACCESSIBILITY

- Type focus-target refs precisely (`HTMLButtonElement`) so `.focus()` calls are checked

VERSION NOTES

- `@types/react` 19: `useRef` requires an argument; all refs are mutable (`RefObject.current` is writable)
- `useReducer` types infer from the reducer; explicit generics are rarely needed

CODE EXPLAINED

→ `User | null` lets you render a loading state before `setUser`

→ `inputRef.current?.focus()` handles the unmounted case

→ Each `Action` member has its own payload; `case 'added'` knows `text` exists

→ Exhaustive `switch` with a return type catches missing cases

→ `as const` makes `useToggle` return a tuple, not `(boolean | fn)[]`

HOOKS WITH TYPES

```tsx
import { createContext, use, useReducer, useRef, useState } from 'react';

type User = { id: string; name: string };

const [user, setUser] = useState<User | null>(null);
const [tags, setTags] = useState<string[]>([]);
const [count, setCount] = useState(0);          // number

const inputRef = useRef<HTMLInputElement>(null);
inputRef.current?.focus();
const timer = useRef<number | null>(null);

type Action =
  | { type: 'added'; text: string }
  | { type: 'toggled'; id: string }
  | { type: 'cleared' };
type Todo = { id: string; text: string; done: boolean };

function reducer(state: Todo[], action: Action): Todo[] {
  switch (action.type) {
    case 'added':
      return [...state, { id: crypto.randomUUID(),
        text: action.text, done: false }];
    case 'toggled':
      return state.map((t) => t.id === action.id
        ? { ...t, done: !t.done } : t);
    case 'cleared':
      return [];
  }
}
const [todos, dispatch] = useReducer(reducer, []);

const AuthContext = createContext<User | null>(null);
export function useAuth(): User {
  const user = use(AuthContext);
  if (!user) throw new Error('useAuth outside <AuthProvider>');
  return user;
}

export function useToggle(init = false) {
  const [on, setOn] = useState(init);
  return [on, () => setOn((v) => !v)] as const;
}
```

WHEN TO ANNOTATE

HOOK	ANNOTATE WHEN
`useState`	initial is `null`, `[]`, `{}`
`useRef (DOM)`	always: `<HTMLDivElement>(null)`
`useRef (value)`	value type differs from initial
`useReducer`	type the reducer, not the hook
`createContext`	always: `<T | null>(null)`

ELEMENT TYPES

ELEMENT	TYPE
`<input>`	`HTMLInputElement`
`<button>`	`HTMLButtonElement`
`<div>`	`HTMLDivElement`
`<form>`	`HTMLFormElement`
`<dialog>`	`HTMLDialogElement`

⚠ COMMON MISTAKES

- `useState([])` inferring `never[]`
- Non-null assertions (`ctx!`) on context instead of a guard hook
- Returning `[a, b]` from a custom hook without `as const`

✓ TIPS & BEST PRACTICES

- Let inference work; annotate only where it can't
- Name action types in past tense (`added`) like events
- Use `satisfies` to check objects without widening

QUICK SUMMARY

- `useState<T | null>(null)`
- `useRef<HTMLElement>(null)`
- Discriminated union actions
- Context `T | null` + guard hook

TypeScript: Events & Forms

ChangeEvent<HTMLInputElement> · MouseEvent · FormEvent · KeyboardEvent · handler prop types · FormData typing

TYPING EVENT HANDLERS

React passes typed synthetic events. Inline handlers get their types inferred from the JSX attribute. When you write a separate handler, annotate the event: `React.ChangeEvent<HTMLInputElement>`, `React.MouseEvent<HTMLButtonElement>`, `React.FormEvent<HTMLFormElement>`, `React.KeyboardEvent`. For callback props, use the matching `…EventHandler` types or write the function signature you actually need.

NOTES

- Hover an inline handler's parameter in your editor to see its type
- `e.currentTarget` is typed as the generic element; `e.target` is a plain `EventTarget`
- `FormData.get()` returns `FormDataEntryValue | null` (string or `File`)
- Prefer callback props with domain types (`onSave(user: User)`) over raw events

♿ ACCESSIBILITY

- Type keyboard handlers with `KeyboardEvent` and check `e.key` so keyboard support compiles alongside mouse support

VERSION NOTES

- React 19 types: `React.FormEvent` remains; form Actions receive `FormData` instead of an event
- `React.SyntheticEvent` is the base of all event types

CODE EXPLAINED

→ `ChangeEvent<HTMLInputElement>` makes `currentTarget.value` a `string`

→ `FormEvent<HTMLFormElement>` types the submit event

→ `KeyboardEvent` exposes `key`, modifiers and `repeat`

→ `ChangeEventHandler<T>` is shorthand for `(e: ChangeEvent<T>) => void`

→ `typeof name === 'string'` narrows `FormDataEntryValue | null`

EVENT AND HANDLER TYPES

```tsx
import type {
  ChangeEvent, FormEvent, KeyboardEvent, MouseEvent,
  ChangeEventHandler,
} from 'react';

function Signup({ onSubmit }: { onSubmit: (email: string) => void }) {
  const [email, setEmail] = useState('');

  const handleChange = (e: ChangeEvent<HTMLInputElement>) => {
    setEmail(e.currentTarget.value);
  };

  const handleSubmit = (e: FormEvent<HTMLFormElement>) => {
    e.preventDefault();
    onSubmit(email);
  };

  const handleKey = (e: KeyboardEvent<HTMLInputElement>) => {
    if (e.key === 'Escape') setEmail('');
  };

  const handleClick = (e: MouseEvent<HTMLButtonElement>) => {
    if (e.shiftKey) console.log('shift-click');
  };

  return (
    <form onSubmit={handleSubmit}>
      <input value={email} onChange={handleChange}
        onKeyDown={handleKey} />
      <button onClick={handleClick}>Join</button>
    </form>
  );
}

// Handler type for props
type FieldProps = { onChange: ChangeEventHandler<HTMLSelectElement> };

// Form Action: FormData, not an event
async function save(formData: FormData) {
  const name = formData.get('name');
  if (typeof name !== 'string') throw new Error('name required');
}
```

EVENT TYPES

JSX PROP	EVENT TYPE
`onChange (input)`	`ChangeEvent<HTMLInputElement>`
`onSubmit`	`FormEvent<HTMLFormElement>`
`onClick`	`MouseEvent<HTMLButtonElement>`
`onKeyDown`	`KeyboardEvent<HTMLElement>`
`onFocus / onBlur`	`FocusEvent<HTMLElement>`
`onPointerDown`	`PointerEvent<HTMLElement>`

HANDLER TYPES

TYPE	EQUALS
`MouseEventHandler<T>`	`(e: MouseEvent<T>) => void`
`ChangeEventHandler<T>`	`(e: ChangeEvent<T>) => void`
`FormEventHandler<T>`	`(e: FormEvent<T>) => void`
`ComponentProps<'button'>['onClick']`	the button's handler type

⚠ COMMON MISTAKES

- Using `e.target.value` on a non-input target and fighting `EventTarget` types
- Importing the DOM's global `MouseEvent` instead of React's
- Typing everything as `any` to silence errors

✓ TIPS & BEST PRACTICES

- Write handlers inline first, then extract with the inferred type
- Use `currentTarget` for the element the handler is attached to
- Expose domain callbacks (`onSelect(id)`) from components, not events

QUICK SUMMARY

- `ChangeEvent<HTMLInputElement>`
- `FormEvent<HTMLFormElement>`
- `…EventHandler<T>` for props
- Actions get `FormData`

TypeScript: Generic & Polymorphic Components

generic props <T,> · as prop · ComponentPropsWithoutRef · discriminated union props · ref prop types · satisfies

FLEXIBLE COMPONENT TYPES

Generic components take a type parameter so their props stay linked: a `List<T>` whose `items` are `T[]` passes T to `renderItem`. In `.tsx` files write `<T,>` so the compiler doesn't read it as JSX. **Polymorphic components** take an `as` prop to choose the rendered element and accept that element's props. **Discriminated union props** make prop combinations explicit and reject invalid ones.

NOTES

- `ComponentPropsWithoutRef<E>` excludes `ref` when you handle it separately
- In React 19, `ref` is a normal prop and included in `ComponentProps<'button'>`
- `Omit<Props, 'x'>` avoids conflicts when merging prop types
- `satisfies` checks an object against a type without widening it

♿ ACCESSIBILITY

- When a polymorphic component renders a non-button element with `onClick`, require `role` and keyboard handling, or restrict `as` to interactive elements

VERSION NOTES

- Generic components are standard TypeScript; no React-specific API
- React 19 removes the need for `forwardRef` generics workarounds

CODE EXPLAINED

- → `List<T,>` infers `T` as `User` from `items`, so `renderItem` gets `User`
- → `E extends ElementType` lets `as` be a tag name or component
- → The props of `E` are merged in, so `href` is valid only with `as="a"`
- → `Omit` removes keys that would conflict with `as` and `children`
- → `kind: 'action'` requires `onAction` and `actionLabel`; `info` forbids them

GENERIC, POLYMORPHIC, UNION PROPS

```tsx
import type { ComponentPropsWithoutRef, ElementType, ReactNode } from 'react';

// Generic list
type ListProps<T> = {
  items: T[];
  getKey: (item: T) => string;
  renderItem: (item: T) => ReactNode;
};
export function List<T,>({ items, getKey, renderItem }: ListProps<T>) {
  return <ul>{items.map((i) => <li key={getKey(i)}>{renderItem(i)}</li>)}</ul>;
}
<List items={users} getKey={(u) => u.id}
  renderItem={(u) => u.name} />;    // u: User

// Polymorphic "as" prop
type TextProps<E extends ElementType> = {
  as?: E;
  children: ReactNode;
} & Omit<ComponentPropsWithoutRef<E>, 'as' | 'children'>;

export function Text<E extends ElementType = 'span'>(
  { as, children, ...rest }: TextProps<E>) {
  const Tag = as ?? 'span';
  return <Tag {...rest}>{children}</Tag>;
}
<Text as="a" href="/docs">Docs</Text>;      // href allowed
<Text as="label" htmlFor="x">Name</Text>;

// Discriminated union props
type AlertProps =
  | { kind: 'info'; message: string }
  | { kind: 'action'; message: string; onAction: () => void;
      actionLabel: string };
function Alert(props: AlertProps) {
  if (props.kind === 'action')
    return <button onClick={props.onAction}>{props.actionLabel}</button>;
  return <p>{props.message}</p>;
}
```

UTILITY TYPES

TYPE	GIVES
`ElementType`	tag name or component
`ComponentPropsWithoutRef<E>`	props of E minus `ref`
`ComponentProps<typeof C>`	props of component C
`Omit<T, K>` / `Pick<T, K>`	remove / keep keys
`Partial<T>` / `Required<T>`	all optional / required

PATTERNS

PATTERN	USE
`Generic <T,>`	lists, tables, selects
`as prop`	typography, layout primitives
`Discriminated union`	mutually exclusive props
`satisfies`	config objects, route maps

⚠ COMMON MISTAKES

- Writing `<T>` in `.tsx` and getting a JSX parse error
- Polymorphic types so complex they slow the editor
- Optional props that only make sense together (use a union)

✓ TIPS & BEST PRACTICES

- Keep polymorphic components to a few low-level primitives
- Let inference pick `T` instead of passing it explicitly
- Use libraries' `asChild` / `Slot` patterns as an alternative to `as`

QUICK SUMMARY

- `function List<T,>(…)`
- `as` + `ElementType` for polymorphism
- Unions forbid invalid combos
- `Omit` to avoid clashes

Styling Basics

CLASSES AND INLINE STYLES

Style React components with plain CSS: import a stylesheet (Vite bundles it) and apply classes with `className`. Build conditional class strings with a template literal or the tiny **`clsx`** helper. The `style` prop takes an object of camelCase properties and is best for truly dynamic values; pass dynamic values into CSS through **custom properties** (`style={{ '--progress': '40%' }}`).

NOTES

- `import './Button.css'` adds the CSS globally; class names are not scoped
- Numbers in `style` get `px` automatically for length properties (`width: 10`)
- Unitless properties (`opacity` , `zIndex` , `lineHeight` , `flex`) stay as numbers
- Vendor prefixes in `style` use PascalCase: `WebkitLineClamp`

♿ ACCESSIBILITY

- Don't convey state with color alone; pair classes like `.error` with text or icons
- Respect `prefers-reduced-motion` in CSS for transitions triggered by class changes

VERSION NOTES

- React 19 hoists `<link rel="stylesheet" precedence>` rendered by components (sheet 53)
- CSS custom properties in `style` are typed as strings in `@types/react`

CODE EXPLAINED

→ `clsx` joins strings and includes object keys whose values are truthy

→ Template literals build variant classes like `btn-primary`

→ `--progress` passes the number to CSS, which handles the drawing

→ `width: size` becomes `size` px automatically

→ The `role="progressbar"` attributes describe the bar to screen readers

CLASSNAME, CLSX, STYLE

```jsx
import clsx from 'clsx';
import './Button.css';

function Button({ variant = 'primary', size, disabled, children }) {
  return (
    <button
      className={clsx('btn', `btn-${variant}`, {
        'btn-sm': size === 'small',
        'is-disabled': disabled,
      })}
      disabled={disabled}
    >
      {children}
    </button>
  );
}

function Progress({ value }) {
  return (
    <div
      className="progress"
      role="progressbar"
      aria-valuenow={value} aria-valuemin={0} aria-valuemax={100}
      style={{ '--progress': `${value}%` }}
    />
  );
}

function Avatar({ size }) {
  return <img style={{ width: size, height: size,
    borderRadius: '50%', objectFit: 'cover' }} alt="" />;
}

/* Button.css */
.btn { padding: 0.5em 1em; border-radius: 6px; }
.btn-primary { background: var(--brand); color: white; }
.progress::after { width: var(--progress); }
```

APPROACHES

APPROACH	SCOPE
`Global CSS import`	global
`CSS Modules`	per file (sheet 68)
`Tailwind utilities`	per element (sheet 69)
`CSS-in-JS`	per component (sheet 70)
`Inline style`	per element, dynamic

STYLE PROP RULES

WRITE	RESULT
`{ marginTop: 8 }`	`margin-top: 8px`
`{ opacity: 0.5 }`	unitless
`{ '--x': '4px' }`	custom property
`{ WebkitLineClamp: 2 }`	vendor prefix
`style="…" string`	✗ error

⚠ COMMON MISTAKES

- Passing a string to `style`
- Using inline styles for hover/focus (they can't express pseudo-classes)
- Global class names that collide across components

✓ TIPS & BEST PRACTICES

- Use custom properties for dynamic values and keep rules in CSS
- Keep a naming convention (BEM) if you stay with global CSS
- `clsx` and `classnames` are interchangeable; pick one

QUICK SUMMARY

- `className` + imported CSS
- `clsx` for conditional classes
- `style` = camelCase object
- Custom props bridge JS → CSS

CSS Modules

.module.css · styles.x · composes · :global · dynamic class names · TypeScript declarations

SCOPED CLASS NAMES

A **CSS Module** is a CSS file named `*.module.css`. Importing it returns an object that maps each class name to a **unique, generated** name, so `.title` in one module never clashes with `.title` in another. Vite supports CSS Modules with no configuration. Use `composes` to reuse classes, `:global` for intentional global selectors, and Sass by naming files `*.module.scss`.

NOTES

- Class names with dashes need bracket access: `styles['card-title']`
- Vite can convert dashes to camelCase with `css.modules.localsConvention`
- Global styles (resets, fonts, variables) stay in a normal `.css` file
- Missing classes are `undefined` at runtime; TypeScript can catch them with typed declarations

♿ ACCESSIBILITY

- Keep visible focus styles in modules: `.button:focus-visible { outline: … }`

VERSION NOTES

- Vite and Next.js support CSS Modules out of the box
- Works with Server Components because the CSS is static

CODE EXPLAINED

- → `styles.primary` resolves to a hashed class name unique to this file
- → `composes: base` adds the `base` class wherever `primary` is used
- → `:global(.dark)` targets the unscoped `dark` class on an ancestor
- → `styles[tone]` selects the class by a prop value
- → `clsx` skips `undefined` when `active` is false

BUTTON.MODULE.CSS + BUTTON.TSX

```
/* Button.module.css */
.base {
  padding: 0.5rem 1rem;
  border-radius: 6px;
}
.primary {
  composes: base;
  background: var(--brand);
  color: white;
}
.danger {
  composes: base;
  background: crimson;
}
.button:focus-visible { outline: 2px solid var(--focus); }
:global(.dark) .primary { background: var(--brand-dark); }

// Button.tsx
import styles from './Button.module.css';
import clsx from 'clsx';

export function Button({ tone = 'primary', active, ...rest }) {
  return (
    <button
      className={clsx(styles[tone], active && styles.active)}
      {...rest}
    />
  );
}

// Rendered HTML (generated names)
// <button class="_primary_x8a1q _base_x8a1q">

// vite.config.ts
// css: { modules: { localsConvention: 'camelCaseOnly' } }
```

MODULE FEATURES

SYNTAX	DOES
`.name`	scoped class
`composes: a`	include class a
`composes: a from './x.module.css'`	include from another file
`:global(.x)`	unscoped selector
`:local(.x)`	explicit scope (default)

CSS MODULES VS OTHERS

CSS MODULES	ALTERNATIVE
Real CSS files	Tailwind: utilities in markup
Zero runtime	CSS-in-JS: often runtime
Works in RSC	Runtime CSS-in-JS doesn't
Scoped by file	Global CSS: manual naming

⚠ COMMON MISTAKES

- Forgetting `.module` in the file name, so classes aren't scoped
- Writing `className="title"` instead of `className={styles.title}`
- Putting resets inside modules and expecting them to be global

✓ TIPS & BEST PRACTICES

- One module per component, next to the component file
- Use `typescript-plugin-css-modules` for autocomplete
- Keep design tokens as global custom properties and use them inside modules

QUICK SUMMARY

- `*.module.css` → scoped classes
- `import styles` → `styles.x`
- `composes` to reuse
- `:global` to escape

Tailwind CSS in React

@tailwindcss/vite · @import "tailwindcss" · cn() + tailwind-merge · cva variants · dark: variant · conditional classes

UTILITY CLASSES IN JSX

Tailwind CSS styles elements with small utility classes (`flex`, `p-4`, `text-sm`) written directly in `className`. Tailwind v4 is configured in CSS: install the Vite plugin, add `@import "tailwindcss";`, and define design tokens with `@theme`. For variants and conditional classes, combine **`clsx`** with **`tailwind-merge`** in a `cn()` helper, and use **`cva`** to define component variants.

NOTES

- Tailwind scans your source files for class names; build them as complete strings
- `tailwind-merge` resolves conflicts: `cn('p-2', 'p-4')` → `'p-4'`
- `dark:`, `hover:`, `focus-visible:`, `md:` are variants
- See the CSS Systems set for Tailwind's full utility reference

♿ ACCESSIBILITY

- Use `focus-visible:` utilities so every interactive element has a visible focus style
- `sr-only` hides text visually but keeps it for screen readers
- `motion-reduce:` variants disable animation for users who ask

VERSION NOTES

- Tailwind v4 (2025) uses CSS-first config (`@theme`) and the `@tailwindcss/vite` plugin; `tailwind.config.js` is optional
- shadcn/ui and many component kits use the `cn()` + cva pattern

CODE EXPLAINED

→ The Vite plugin replaces PostCSS setup in Tailwind v4

→ `@theme` creates `bg-brand`, `text-brand` and similar utilities from the token

→ `cn()` merges classes and lets callers override with `className`

→ `cva` maps `intent` and `size` props to class strings

→ `sr-only` adds context for screen readers without changing the visible label

TAILWIND V4 + CN() + CVA

```
// vite.config.ts
import tailwindcss from '@tailwindcss/vite';
export default defineConfig({ plugins: [react(), tailwindcss()] });

/* src/index.css */
@import "tailwindcss";
@theme {
  --color-brand: oklch(0.62 0.19 255);
}

// lib/cn.ts
import { clsx } from 'clsx';
import { twMerge } from 'tailwind-merge';
export const cn = (...inputs) => twMerge(clsx(inputs));

// Button.tsx
import { cva } from 'class-variance-authority';

const button = cva(
  'inline-flex items-center rounded-md font-medium ' +
  'focus-visible:outline-2 focus-visible:outline-brand',
  {
    variants: {
      intent: {
        primary: 'bg-brand text-white hover:bg-brand/90',
        ghost: 'bg-transparent hover:bg-gray-100',
      },
      size: { sm: 'h-8 px-3 text-sm', md: 'h-10 px-4' },
    },
    defaultVariants: { intent: 'primary', size: 'md' },
  },
);

export function Button({ intent, size, className, ...props }) {
  return <button className={cn(button({ intent, size }),
    className)} {...props} />;
}

<Button intent="ghost" className="w-full dark:text-white">
  Cancel <span className="sr-only">editing profile</span>
</Button>
```

COMMON UTILITIES

CLASS	CSS
`flex gap-2`	`display:flex; gap:.5rem`
`p-4 / px-2`	padding
`text-sm font-medium`	size / weight
`rounded-lg`	border-radius
`grid grid-cols-3`	3-column grid
`hidden md:block`	responsive display

DYNAMIC CLASSES

DO	DON'T
`isOn ? 'bg-green-500' : 'bg-gray-300'`	`bg-${color}-500`
`cn('p-2', className)`	string concat with conflicts
`cva variants`	nested ternaries
`CSS variable for arbitrary values`	generating class names at runtime

⚠ COMMON MISTAKES

- Building class names dynamically so Tailwind can't detect them
- Concatenating conflicting classes without `tailwind-merge`
- Removing focus outlines (`outline-none`) without a replacement

✓ TIPS & BEST PRACTICES

- Extract components, not `@apply` rules, when class lists repeat
- Use the Prettier Tailwind plugin to sort classes
- Use arbitrary values sparingly: `w-[37px]`

QUICK SUMMARY

- Utilities in `className`
- v4: `@import` + `@theme`
- `cn()` = clsx + tailwind-merge
- cva for variants

CSS-in-JS

styled-components · props in styles · ThemeProvider · runtime cost · maintenance mode · zero-runtime options · Server Component limits

STYLES WRITTEN IN JAVASCRIPT

CSS-in-JS libraries let you write CSS inside component files. **styled-components** and **Emotion** create components with attached styles (`styled.button`), interpolate props, and theme through a provider. They generate styles at **runtime**, which costs performance and doesn't work in **Server Components**. Newer **zero-runtime** libraries (vanilla-extract, Panda CSS, StyleX, Linaria) extract static CSS at build time.

NOTES

- Transient props (`$active`) in styled-components aren't passed to the DOM
- Runtime libraries inject `<style>` tags with `useInsertionEffect`
- Server rendering needs a style registry to collect CSS
- See the CSS Systems set for styled-components in depth

♿ ACCESSIBILITY

- Style `:focus-visible` inside styled components like any CSS
- Pass `aria-*` attributes through; styled components forward valid HTML attributes

VERSION NOTES

- styled-components entered maintenance mode in 2025; prefer other options for new projects
- Runtime CSS-in-JS works only in Client Components (`'use client'`) under React Server Components

CODE EXPLAINED

→ `styled.button` creates a component with generated class names

→ Interpolated functions read props and the theme at render time

→ `$primary` is a transient prop, so it never reaches the DOM

→ `styled(Button)` extends existing styles

→ vanilla-extract writes TypeScript that becomes a static `.css` file

STYLED-COMPONENTS AND A ZERO-RUNTIME OPTION

```
import styled, { ThemeProvider } from 'styled-components';

const theme = { brand: '#0b6bcb', radius: '6px' };

const Button = styled.button`
  padding: 0.5rem 1rem;
  border-radius: ${(p) => p.theme.radius};
  background: ${(p) => (p.$primary ? p.theme.brand : 'white')};
  color: ${(p) => (p.$primary ? 'white' : p.theme.brand)};
  &:focus-visible { outline: 2px solid ${(p) => p.theme.brand}; }
`;

const DangerButton = styled(Button)`
  background: crimson;
`;

export function App() {
  return (
    <ThemeProvider theme={theme}>
      <Button $primary>Save</Button>
      <DangerButton>Delete</DangerButton>
    </ThemeProvider>
  );
}

// vanilla-extract: button.css.ts (compiled to static CSS)
import { style } from '@vanilla-extract/css';
export const button = style({
  padding: '0.5rem 1rem',
  selectors: { '&:focus-visible': { outline: '2px solid' } },
});

// Button.tsx
import { button } from './button.css';
<button className={button}>Save</button>;
```

LIBRARY COMPARISON

LIBRARY	RUNTIME
styled-components	runtime (maintenance mode)
Emotion	runtime
vanilla-extract	zero (build time)
Panda CSS	zero (build time)
StyleX	zero (build time)
Linaria	zero (build time)

TRADE-OFFS

RUNTIME CSS-IN-JS	STATIC CSS
Dynamic styles from props	Variants + CSS variables
Style recalculation cost	No runtime cost
Client Components only	Works in RSC
SSR registry needed	Plain <link> tags

⚠ COMMON MISTAKES

- Starting a new RSC app with runtime CSS-in-JS
- Interpolating props for every style and regenerating classes constantly
- Passing non-HTML props to the DOM (use `$` transient props)

✓ TIPS & BEST PRACTICES

- Prefer CSS variables for dynamic values even in CSS-in-JS
- For new projects, choose CSS Modules, Tailwind or a zero-runtime library
- Migrate gradually: the two approaches can coexist

QUICK SUMMARY

- `styled.tag` + template CSS
- Runtime cost, no RSC support
- Zero-runtime options extract CSS
- styled-components in maintenance

React Router: Setup & Modes

declarative / data / framework modes · createBrowserRouter · RouterProvider · BrowserRouter · <Routes> / <Route> · v7 → v8

THREE WAYS TO USE IT

React Router maps URLs to components. It can be used in three **modes**. *Declarative* mode (`<BrowserRouter>` + `<Routes>`) only matches URLs to components. *Data* mode (`createBrowserRouter` + `<RouterProvider>`) adds loaders, actions and pending states. *Framework* mode adds a Vite plugin, file-based route config, type generation and server rendering. Everything imports from the single `react-router` package.

NOTES

- `react-router-dom` was merged into `react-router` in v7; old imports still re-export
- Route objects use `Component` (or `element`) and nest with `children`
- An `index: true` route renders at the parent's exact URL
- `path: '*'` catches unmatched URLs for a 404 page

♿ ACCESSIBILITY

- After navigation, move focus to the new page's `<h1>` or main region so screen-reader users hear the change
- Update `<title>` on every route

VERSION NOTES

- React Router v8 (June 2026) is a non-breaking upgrade from v7: ESM-only, React 19+, Node 22+
- v7 merged Remix into React Router as *framework mode*
- v6 reached end of life

CODE EXPLAINED

→ `<BrowserRouter>` uses the History API for clean URLs

→ `<Route path element>` pairs a URL pattern with UI

→ In data mode, the route config is a plain array of objects

→ `children` routes render inside the parent's `<Outlet>` (sheet 74)

→ `loader` fetches data before the route renders (sheet 75)

DECLARATIVE AND DATA MODE

```jsx
// Declarative mode
import { BrowserRouter, Routes, Route } from 'react-router';

createRoot(el).render(
  <BrowserRouter>
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="about" element={<About />} />
      <Route path="users/:id" element={<User />} />
      <Route path="*" element={<NotFound />} />
    </Routes>
  </BrowserRouter>,
);

// Data mode
import { createBrowserRouter } from 'react-router';
import { RouterProvider } from 'react-router/dom';

const router = createBrowserRouter([
  {
    path: '/',
    Component: Root,
    ErrorBoundary: RootError,
    children: [
      { index: true, Component: Home },
      { path: 'about', Component: About },
      {
        path: 'users/:id',
        Component: User,
        loader: ({ params }) => fetchUser(params.id),
      },
      { path: '*', Component: NotFound },
    ],
  },
]);

createRoot(el).render(<RouterProvider router={router} />);
```

MODES

MODE	ADDS
`Declarative`	URL matching, `Link`, hooks
`Data`	loaders, actions, `useFetcher`, pending UI
`Framework`	Vite plugin, `routes.ts`, typegen, SSR
`Upgrade path`	declarative → data → framework

ROUTER TYPES

ROUTER	USE
`createBrowserRouter`	normal web apps
`createHashRouter`	static hosts without rewrites
`createMemoryRouter`	tests, embedded widgets
`<StaticRouter>`	server rendering (declarative)

⚠ COMMON MISTAKES

- Importing from `react-router-dom` in new v8 code
- Defining the router inside a component (recreated each render)
- Forgetting a catch-all `*` route

✓ TIPS & BEST PRACTICES

- Start new SPAs in data mode; it's easy to grow from
- Keep route config in one file for an overview
- Use framework mode when you want SSR without Next.js

QUICK SUMMARY

- One package: `react-router`
- Declarative, data, framework modes
- Routes nest with `children`
- v8 = ESM, React 19+, Node 22+

React Router: Navigation

MOVING BETWEEN PAGES

Render `<Link to>` for navigation the user triggers; it renders an `<a>` and updates the URL without a page reload. `<NavLink>` adds an active state for menus. For navigation in code (after a form submit, on logout) call the function returned by `useNavigate()`. Render `<Navigate to>` to redirect during render. Paths without a leading slash are **relative** to the current route.

NOTES

- `navigate(-1)` goes back; `navigate('/x', { replace: true })` replaces the history entry
- `state` passes data that isn't in the URL: `navigate('/done', { state: { id } })`
- `useLocation().state` reads it on the next page
- Prefer `<Link>` over `navigate` for anything clickable: it supports middle-click and copy link

♿ ACCESSIBILITY

- `<NavLink>` sets `aria-current="page"` on the active link automatically
- Use `<Link>` for navigation and `<button>` for actions, even if both look like links

VERSION NOTES

- Navigation is wrapped in `startTransition`, so pending UI and Suspense work smoothly
- `viewTransition` prop/option enables View Transitions on navigation

CODE EXPLAINED

- → `end` makes the Home link active only at exactly `/`
- → `className` can be a function of `isActive` and `isPending`
- → `viewTransition` animates the navigation with View Transitions
- → `replace: true` stops the back button returning to a logged-out page
- → `<Navigate>` redirects while rendering

LINK, NAVLINK, USENAVIGATE

```jsx
import { Link, NavLink, Navigate, useNavigate } from 'react-router';

function Nav() {
  return (
    <nav>
      <NavLink to="/" end>Home</NavLink>
      <NavLink
        to="/inbox"
        className={({ isActive, isPending }) =>
          isActive ? 'active' : isPending ? 'pending' : ''}
      >
        Inbox
      </NavLink>
      <Link to="/settings" viewTransition>Settings</Link>
      <Link to="../" relative="path">Up one level</Link>
    </nav>
  );
}

function LogoutButton() {
  const navigate = useNavigate();
  async function logout() {
    await api.logout();
    navigate('/login', { replace: true });
  }
  return <button onClick={logout}>Log out</button>;
}

function Protected({ user, children }) {
  if (!user) return <Navigate to="/login" replace />;
  return children;
}

function Checkout() {
  const navigate = useNavigate();
  const done = (id) => navigate('/thanks', { state: { orderId: id } });
}
```

NAVIGATION APIS

API	USE
`<Link to>`	user-clicked navigation
`<NavLink to>`	menus with active state
`useNavigate()`	navigate in handlers
`<Navigate to>`	redirect in render
`redirect()`	redirect from loaders/actions

NAVIGATE OPTIONS

OPTION	EFFECT
`replace: true`	replace history entry
`state: {...}`	pass hidden data
`relative: 'path'`	resolve `..` by URL segments
`preventScrollReset`	keep scroll position
`viewTransition`	animate navigation

⚠ COMMON MISTAKES

- Using `<a href>` for internal links and reloading the page
- `navigate()` during render instead of `<Navigate>`
- Putting required data only in `state` (lost on refresh)

✓ TIPS & BEST PRACTICES

- Keep important state in the URL so it survives reloads and sharing
- Use `prefetch="intent"` on links in framework mode
- Use `replace` after login and form submissions

QUICK SUMMARY

- `<Link>` for clicks
- `<NavLink>` adds active + `aria-current`
- `useNavigate()` in handlers
- Relative paths by default

React Router: Params & Search Params

:param · optional segments · splat * · useParams · useSearchParams · useLocation

READING THE URL

Dynamic segments (`:id`) capture parts of the path; read them with `useParams()`. Add `?` to make a segment optional (`:lang?`), and use `*` as a **splat** to capture the rest of the URL. The query string is read and written with `useSearchParams()`, which works like `useState` but stores values in the URL. `useLocation()` returns the full location: `pathname`, `search`, `hash` and `state`.

NOTES

- All params are strings (or `undefined`); convert numbers yourself
- `params['*']` holds the splat value
- `setSearchParams` replaces the whole query; use the function form to update one key
- Search params are great for filters, sorting and pagination

♿ ACCESSIBILITY

- Reflect filter state in the URL so users can bookmark and share accessible views
- Announce result counts after search params change

VERSION NOTES

- Unchanged API in v7 and v8
- Framework mode provides typed `params` via generated route types (sheet 76)

CODE EXPLAINED

- → `:userId` and `:postId` become keys on the params object
- → `:lang?` matches `/docs` and `/en/docs`
- → The splat captures everything after `files/`
- → `searchParams.get` returns `null` when missing, so a default follows `??`
- → The updater form keeps other query keys while changing `sort`

PARAMS AND SEARCH PARAMS

```
import { useLocation, useParams, useSearchParams } from 'react-router';

// Route config
// { path: 'users/:userId/posts/:postId', Component: Post }
// { path: ':lang?/docs', Component: Docs }
// { path: 'files/*', Component: FileBrowser }

function Post() {
  const { userId, postId } = useParams();
  return <h1>Post {postId} by user {userId}</h1>;
}

function FileBrowser() {
  const params = useParams();
  const path = params['*'];       // "a/b/readme.md"
  return <code>{path}</code>;
}

function ProductList() {
  const [searchParams, setSearchParams] = useSearchParams();
  const sort = searchParams.get('sort') ?? 'name';
  const page = Number(searchParams.get('page') ?? 1);

  function setSort(next) {
    setSearchParams((prev) => {
      prev.set('sort', next);
      prev.set('page', '1');
      return prev;
    });
  }

  return (
    <>
      <select value={sort} onChange={(e) => setSort(e.target.value)}>
        <option value="name">Name</option>
        <option value="price">Price</option>
      </select>
      <Products sort={sort} page={page} />
    </>
  );
}

function Debug() {
  const { pathname, search, hash, state } = useLocation();
}
```

PATH PATTERNS

PATTERN	MATCHES
`users/:id`	`/users/42`
`:lang?/docs`	`/docs`, `/fr/docs`
`files/*`	`/files/a/b.txt`
`*`	anything (404)
`docs/:slug.html`	✗ (segments only)

URLSEARCHPARAMS

METHOD	DOES
`get(k)`	first value or `null`
`getAll(k)`	all values
`set(k, v)`	replace
`append(k, v)`	add another
`delete(k)`	remove
`toString()`	a=1&b=2

⚠ COMMON MISTAKES

- Treating params as numbers without converting
- Calling `setSearchParams({ sort })` and wiping other keys
- Storing filter state in React state so it's lost on refresh

✓ TIPS & BEST PRACTICES

- Validate params with Zod in loaders
- Use `replace: true` for search updates that shouldn't add history entries
- Keep URLs human-readable: `?sort=price&page=2`

QUICK SUMMARY

- `:param` → `useParams()`
- `:opt?` optional, `*` splat
- `useSearchParams()` for `?query`
- Everything is a string

React Router: Nested Routes & Layouts

LAYOUTS WITH OUTLET

Nested routes share UI. A parent route renders the common layout (header, sidebar) and an **`<Outlet />`** where the matching child route appears. **Index routes** render at the parent's own URL. **Layout routes** have no `path` and only add UI; **pathless** groupings help share loaders or error boundaries. Each route can define an `ErrorBoundary` that catches errors from its loader, action or component.

NOTES

- The URL `/dashboard/settings` renders `Dashboard` with `Settings` in its outlet
- `useOutletContext()` reads a value passed with `<Outlet context={…} />`
- A child's error boundary replaces only that child, keeping the parent layout
- `useRouteError()` reads the error inside an error boundary

♿ ACCESSIBILITY

- Keep landmark structure in the layout: one `<nav>`, one `<main>` that wraps the `<Outlet>`
- Error boundary pages should keep the layout's navigation so users can recover

VERSION NOTES

- Route-level `ErrorBoundary` and `HydrateFallback` exist in data and framework modes
- `isRouteErrorResponse(error)` detects thrown `Response`s and `data()` errors

CODE EXPLAINED

→ `AppLayout` wraps every page; `DashboardLayout` wraps dashboard pages

→ `index: true` renders `Overview` at `/dashboard`

→ `AuthLayout` adds UI for `/login` without adding a URL segment

→ `<Outlet context>` passes `user` to whichever child renders

→ `AppError` handles 404s and thrown errors for the whole app

NESTED ROUTES

```
import { Outlet, useOutletContext, useRouteError,
  isRouteErrorResponse } from 'react-router';

const router = createBrowserRouter([
  {
    path: '/',
    Component: AppLayout,
    ErrorBoundary: AppError,
    children: [
      { index: true, Component: Home },
      {
        path: 'dashboard',
        Component: DashboardLayout,
        children: [
          { index: true, Component: Overview },
          { path: 'settings', Component: Settings },
        ],
      },
      { Component: AuthLayout,            // layout route: no path
        children: [{ path: 'login', Component: Login }] },
    ],
  },
]);

function DashboardLayout() {
  const user = useUser();
  return (
    <div className="dashboard">
      <aside><DashboardNav /></aside>
      <main><Outlet context={{ user }} /></main>
    </div>
  );
}

function Settings() {
  const { user } = useOutletContext();
  return <h1>Settings for {user.name}</h1>;
}

function AppError() {
  const error = useRouteError();
  if (isRouteErrorResponse(error))
    return <h1>{error.status} {error.statusText}</h1>;
  return <h1>Something went wrong</h1>;
}
```

ROUTE KINDS

KIND	CONFIG
Normal	`{ path, Component }`
Index	`{ index: true, Component }`
Layout (pathless)	`{ Component, children }`
Splat / 404	`{ path: '*' }`
Lazy	`{ path, lazy: () => import(...) }`

OUTLET HELPERS

API	USE
`<Outlet />`	render matched child
`<Outlet context={v}>`	pass data to children
`useOutletContext()`	read it
`useMatches()`	all matched routes (breadcrumbs)
`useRouteError()`	error in boundary

⚠ COMMON MISTAKES

- Forgetting `<Outlet />` so child routes never appear
- Leading slashes in child `path`s (`/settings` makes it absolute)
- One app-level error boundary only, which hides the layout

✓ TIPS & BEST PRACTICES

- Use `handle` on routes plus `useMatches()` to build breadcrumbs
- Use `lazy` on routes to split code per route
- Give each major section its own error boundary

QUICK SUMMARY

- Parent layout + `<Outlet />`
- `index: true` for the default child
- Pathless layout routes
- `ErrorBoundary` per route

React Router: Loaders & Actions

loader · useLoaderData · action · <Form> · useFetcher · useNavigation · redirect()

DATA BEFORE RENDER

In data and framework modes, each route can define a **`loader`** that runs before the route renders and returns its data, read with `useLoaderData()`. An **`action`** handles form submissions: `<Form method="post">` sends `FormData` to the route's action, and after it runs, React Router **revalidates** all loaders so the UI shows fresh data. **`useFetcher`** submits or loads without navigating, and `useNavigation` reports pending state.

NOTES

- Loaders receive `{ params, request }`; read query strings from `new URL(request.url)`
- Return plain objects; throw `data('Not found', { status: 404 })` for HTTP errors
- `redirect('/x')` from a loader or action navigates
- Loaders for parent and child routes run in parallel

♿ ACCESSIBILITY

- Show `navigation.state === 'submitting'` as text on the submit button
- Return validation errors from actions and render them next to fields with `aria-describedby`

VERSION NOTES

- `json()` and `defer()` are deprecated since v7; return objects and promises directly
- `<Form>` works without JavaScript in framework mode with SSR

CODE EXPLAINED

→ The loader throws a 404 that the route's error boundary renders

→ The action validates, saves, then redirects

→ Returned `{ error }` shows up in `useActionData()`

→ `useNavigation()` drives the pending button label

→ `fetcher.formData` gives an optimistic star before the server responds

LOADER, ACTION, FORM, USEFETCHER

```
import { Form, data, redirect, useActionData, useFetcher,
  useLoaderData, useNavigation } from 'react-router';

const route = {
  path: 'projects/:id',
  async loader({ params }) {
    const project = await db.project(params.id);
    if (!project) throw data('Not found', { status: 404 });
    return { project };
  },
  async action({ request, params }) {
    const form = await request.formData();
    const title = String(form.get('title'));
    if (!title) return { error: 'Title is required' };
    await db.update(params.id, { title });
    return redirect(`/projects/${params.id}`);
  },
  Component: Project,
};

function Project() {
  const { project } = useLoaderData();
  const result = useActionData();
  const navigation = useNavigation();
  const saving = navigation.state === 'submitting';
  return (
    <Form method="post">
      <input name="title" defaultValue={project.title} />
      {result?.error && <p role="alert">{result.error}</p>}
      <button disabled={saving}>{saving ? 'Saving…' : 'Save'}</button>
      <StarButton id={project.id} starred={project.starred} />
    </Form>
  );
}

function StarButton({ id, starred }) {
  const fetcher = useFetcher();
  const on = fetcher.formData
    ? fetcher.formData.get('starred') === 'true' : starred;
  return (
    <fetcher.Form method="post" action={`/projects/${id}/star`}>
      <button name="starred" value={String(!on)}>{on ? '★' : '☆'}</button>
    </fetcher.Form>
  );
}
```

DATA APIS

API	USE
`loader`	fetch data for a route
`useLoaderData()`	read it
`action`	handle POST/PUT/DELETE
`useActionData()`	action's return value
`<Form method>`	submit to an action
`redirect() / data()`	navigate / status codes

PENDING STATE

SOURCE	STATES
`useNavigation().state`	`idle` · `loading` · `submitting`
`fetcher.state`	`idle` · `loading` · `submitting`
`fetcher.formData`	data being submitted
`fetcher.data`	fetcher's result

⚠ COMMON MISTAKES

- Fetching in `useEffect` inside routes that could use loaders
- Using `<form>` instead of `<Form>` and doing a full page submit
- Navigating with `useFetcher` when the URL should change

✓ TIPS & BEST PRACTICES

- Use `useFetcher` for inline actions: likes, toggles, quick edits
- Validate in actions with Zod and return field errors
- Let revalidation refresh data instead of updating caches by hand

QUICK SUMMARY

- `loader` → `useLoaderData()`
- `<Form>` → `action` → revalidate
- `useFetcher` for non-navigating calls
- `useNavigation` for pending UI

React Router: Framework Mode

routes.ts · route modules · typegen · SSR & prerender · clientLoader · ErrorBoundary export

FULL-STACK REACT ROUTER

Framework mode turns React Router into a full-stack framework (formerly Remix). The `@react-router/dev` Vite plugin reads a route config from `app/routes.ts`; each entry points to a **route module** that exports a default component plus optional `loader`, `action`, `clientLoader`, `meta` and `ErrorBoundary`. **Type generation** produces `Route` types per module, so loader data and params are typed. Choose SSR, static prerendering or SPA output in `react-router.config.ts`.

NOTES

- `npx create-react-router@latest` scaffolds a framework-mode app
- `loader` runs on the server; `clientLoader` runs in the browser
- `react-router typegen` writes `./+types/<route>` files (also runs during dev)
- `ssr: false` builds a SPA; `prerender` lists URLs to render at build time

ACCESSIBILITY

- Server-rendered HTML is readable before JavaScript loads, which helps slow connections and assistive tech
- Export `meta` per route for unique titles

VERSION NOTES

- Framework mode arrived in v7 (Remix merged into React Router)
- v8 keeps the same file conventions; Remix v2 is end of life

CODE EXPLAINED

→ `routes.ts` lists routes explicitly; helpers build the config

→ `./+types/product` is generated, so `params.id` and `loaderData` are typed

→ The loader runs on the server and can query the database directly

→ `meta` sets the page title from loader data

→ The config enables SSR and prerenders two static pages

ROUTES.TS AND A ROUTE MODULE

```ts
// app/routes.ts
import { type RouteConfig, index, layout, route }
  from '@react-router/dev/routes';

export default [
  index('routes/home.tsx'),
  route('products/:id', 'routes/product.tsx'),
  layout('routes/auth-layout.tsx', [
    route('login', 'routes/login.tsx'),
  ]),
] satisfies RouteConfig;

// app/routes/product.tsx
import type { Route } from './+types/product';

export async function loader({ params }: Route.LoaderArgs) {
  const product = await db.product(params.id);   // server only
  if (!product) throw new Response('Not found', { status: 404 });
  return { product };
}

export function meta({ data }: Route.MetaArgs) {
  return [{ title: `${data?.product.name} | Shop` }];
}

export default function Product({ loaderData }: Route.ComponentProps) {
  return <h1>{loaderData.product.name}</h1>;
}

export function ErrorBoundary({ error }: Route.ErrorBoundaryProps) {
  return <p role="alert">Couldn't load this product.</p>;
}

// react-router.config.ts
import type { Config } from '@react-router/dev/config';
export default { ssr: true, prerender: ['/', '/about'] }
  satisfies Config;
```

ROUTE MODULE EXPORTS

EXPORT	RUNS
`default`	component
`loader`	server, before render
`clientLoader`	browser
`action / clientAction`	form submits
`meta / links`	head tags
`ErrorBoundary / HydrateFallback`	errors / SPA loading

RENDERING OPTIONS

CONFIG	OUTPUT
`ssr: true`	server rendering
`ssr: false`	SPA build
`prerender: [...]`	static HTML at build
`ssr: false + prerender`	static site
`Adapters`	Node, Cloudflare, Vercel, Netlify

⚠ COMMON MISTAKES

- Importing server-only modules (DB clients) into code that runs in the browser
- Editing generated `+types` files
- Mixing data-mode `createBrowserRouter` code into a framework app

✓ TIPS & BEST PRACTICES

- Name server-only files `*.server.ts` so they can't be bundled for the client
- Run `react-router typegen` in CI before `tsc`
- Start with SPA mode and switch on SSR later if you need it

QUICK SUMMARY

- Vite plugin + `app/routes.ts`
- Route modules export loader/action/UI
- Generated `Route` types
- SSR, prerender or SPA

TanStack Query: Setup & Queries

QueryClientProvider · useQuery · queryKey · queryFn · isPending / isError · staleTime · gcTime

SERVER STATE, CACHED

TanStack Query manages *server state*: data that lives on a server and is fetched, cached and kept fresh in the browser. Wrap the app in `QueryClientProvider`, then call `useQuery({ queryKey, queryFn })`. The **query key** identifies the data in the cache; any component using the same key shares one request and one cached result. Data is refetched automatically when it's **stale** and the window regains focus or reconnects.

NOTES

- `staleTime` (default 0): how long data counts as fresh; fresh data isn't refetched
- `gcTime` (default 5 min): how long unused data stays in the cache
- Include every variable the `queryFn` uses in the `queryKey`
- Failed queries retry 3 times with backoff by default

♿ ACCESSIBILITY

- Render `isPending` and `isError` states as text, with `role="status"` and `role="alert"`
- Background refetches (`isFetching`) shouldn't replace content with spinners; show a subtle indicator

VERSION NOTES

- TanStack Query v5 is current: object-only signature, `isPending` (renamed from `isLoading`), `gcTime` (renamed from `cacheTime`)
- Supports React 18 and 19, including Suspense

CODE EXPLAINED

→ One `QueryClient` holds the cache for the whole app

→ `staleTime: 60_000` treats data as fresh for a minute

→ `['todos', { status }]` caches each status separately

→ `queryFn` must throw on HTTP errors so `isError` works

→ `isPending` is the first load; `isFetching` includes background refreshes

SETUP AND USEQUERY

```tsx
import {
  QueryClient, QueryClientProvider, useQuery,
} from '@tanstack/react-query';
import { ReactQueryDevtools } from '@tanstack/react-query-devtools';

const queryClient = new QueryClient({
  defaultOptions: { queries: { staleTime: 60_000 } },
});

export function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <Todos status="open" />
      <ReactQueryDevtools />
    </QueryClientProvider>
  );
}

async function fetchTodos(status) {
  const res = await fetch(`/api/todos?status=${status}`);
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return res.json();
}

function Todos({ status }) {
  const { data, error, isPending, isError, isFetching } = useQuery({
    queryKey: ['todos', { status }],
    queryFn: () => fetchTodos(status),
  });

  if (isPending) return <p role="status">Loading…</p>;
  if (isError) return <p role="alert">{error.message}</p>;

  return (
    <>
      {isFetching && <small>Refreshing…</small>}
      <ul>{data.map((t) => <li key={t.id}>{t.title}</li>)}</ul>
    </>
  );
}
```

USEQUERY RESULT

FIELD	MEANING
`data`	cached result
`error`	last error
`isPending`	no data yet
`isError / isSuccess`	status checks
`isFetching`	any request in flight
`refetch()`	fetch now

KEY OPTIONS

OPTION	DEFAULT
`staleTime`	`0`
`gcTime`	5 minutes
`retry`	3
`refetchOnWindowFocus`	`true`
`enabled`	`true`
`placeholderData`	none

⚠ COMMON MISTAKES

- Leaving variables out of the query key, so different inputs share a cache entry
- Not throwing on `res.ok === false`
- Copying query data into `useState`

✓ TIPS & BEST PRACTICES

- Set a sensible global `staleTime` to cut refetches
- Build keys with a factory: `todoKeys.list(status)`
- Use Devtools to inspect cache entries and states

QUICK SUMMARY

- `QueryClientProvider` + `useQuery`
- Key = cache identity
- `staleTime` / `gcTime`
- `isPending` vs `isFetching`

TanStack Query: Mutations & Cache

CHANGING SERVER DATA

`useMutation({ mutationFn })` runs create, update and delete requests. Call `mutate(variables)` (or `await mutateAsync`) to trigger it, and read `isPending`, `isError` and `data`. After a mutation succeeds, keep the cache correct by **invalidating** affected queries so they refetch, or by writing the new data directly with `setQueryData`. For instant feedback, apply an **optimistic update** and roll back on error.

NOTES

- `invalidateQueries({ queryKey: ['todos'] })` matches all keys starting with `['todos']`
- Returning the invalidation promise from `onSuccess` / `onSettled` keeps the mutation pending until refetch completes
- `onMutate` runs before the request; its return value is the context passed to `onError`
- Simplest optimistic UI: render `variables` while `isPending`

♿ ACCESSIBILITY

- Disable or mark buttons while `isPending`, and announce success or failure

VERSION NOTES

- In v5, the value returned by `onMutate` arrives as the third argument of `onError` and `onSettled`
- `useMutationState` reads pending mutations from anywhere

CODE EXPLAINED

- → `invalidateQueries` marks every todos query stale and refetches active ones
- → `add.variables` shows the pending title as a faded item
- → `cancelQueries` stops in-flight fetches from overwriting the optimistic data
- → `onMutate` snapshots and updates the cache, returning the snapshot
- → `onError` restores it; `onSettled` refetches to sync with the server

MUTATIONS AND CACHE UPDATES

```tsx
import { useMutation, useQueryClient } from '@tanstack/react-query';

function AddTodo() {
  const queryClient = useQueryClient();
  const add = useMutation({
    mutationFn: (title) => api.post('/todos', { title }),
    onSuccess: () =>
      queryClient.invalidateQueries({ queryKey: ['todos'] }),
  });

  return (
    <form onSubmit={(e) => {
      e.preventDefault();
      add.mutate(new FormData(e.currentTarget).get('title'));
    }}>
      <input name="title" />
      <button disabled={add.isPending}>Add</button>
      {add.isError && <p role="alert">{add.error.message}</p>}
      {add.isPending && <li style={{ opacity: 0.5 }}>{add.variables}</li>}
    </form>
  );
}

// Optimistic update with rollback
const toggle = useMutation({
  mutationFn: (todo) => api.patch(`/todos/${todo.id}`,
    { done: !todo.done }),
  onMutate: async (todo) => {
    await queryClient.cancelQueries({ queryKey: ['todos'] });
    const previous = queryClient.getQueryData(['todos']);
    queryClient.setQueryData(['todos'], (old) => old.map((t) =>
      t.id === todo.id ? { ...t, done: !t.done } : t));
    return { previous };
  },
  onError: (err, todo, context) =>
    queryClient.setQueryData(['todos'], context.previous),
  onSettled: () =>
    queryClient.invalidateQueries({ queryKey: ['todos'] }),
});
```

USEMUTATION

MEMBER	USE
`mutate(vars)`	fire and forget
`mutateAsync(vars)`	returns a promise
`isPending / isError / isSuccess`	status
`variables`	what's being sent
`reset()`	clear state

CACHE METHODS

METHOD	USE
`invalidateQueries({ queryKey })`	mark stale + refetch
`setQueryData(key, updater)`	write directly
`getQueryData(key)`	read
`cancelQueries({ queryKey })`	stop fetches
`removeQueries({ queryKey })`	delete from cache

⚠ COMMON MISTAKES

- Mutating cached arrays in place inside `setQueryData`
- Forgetting to invalidate, so lists show stale data
- Optimistic updates without `cancelQueries` (race with refetch)

✓ TIPS & BEST PRACTICES

- Prefer invalidation; use `setQueryData` when the response contains the new entity
- Use `variables` -based optimistic UI for simple cases
- Centralize mutations in custom hooks (`useAddTodo`)

QUICK SUMMARY

- `useMutation({ mutationFn })`
- Invalidate after success
- `setQueryData` for direct writes
- `onMutate` + rollback for optimistic UI

TanStack Query: Advanced

queryOptions · useSuspenseQuery · useInfiniteQuery · dependent queries · prefetchQuery · select · Devtools

BEYOND THE BASICS

`queryOptions()` bundles a key and fetcher into a reusable, type-safe object you can pass to `useQuery`, `useSuspenseQuery`, `prefetchQuery` and `getQueryData`. `useSuspenseQuery` integrates with **Suspense**: `data` is always defined and loading is handled by the nearest boundary. `useInfiniteQuery` handles paginated feeds, **dependent queries** wait on other data via `enabled`, and `select` transforms data per component.

NOTES

- `useSuspenseQuery` throws errors to error boundaries
- `enabled: !!userId` waits until `userId` exists
- `select` re-runs only when data or the selector changes
- Prefetch on hover or in a router loader to make navigation instant

♿ ACCESSIBILITY

- Infinite lists need a real *Load more* button as well as scroll triggers, and should announce new items

VERSION NOTES

- `queryOptions` and `useSuspenseQuery` are v5 features
- `initialPageParam` is required for infinite queries in v5

CODE EXPLAINED

- → `userQuery(id)` is the single definition of how to fetch a user
- → `useSuspenseQuery` suspends until data arrives, so no loading branch
- → `enabled` keeps the projects query idle until `userId` exists
- → `getNextPageParam` returns `undefined` to signal the last page
- → `prefetchQuery` warms the cache before the user clicks

QUERYOPTIONS, SUSPENSE, INFINITE

```tsx
import { queryOptions, useInfiniteQuery, useQuery,
  useSuspenseQuery, useQueryClient } from '@tanstack/react-query';

export const userQuery = (id) => queryOptions({
  queryKey: ['user', id],
  queryFn: () => api.get(`/users/${id}`),
  staleTime: 5 * 60_000,
});

function Profile({ id }) {
  const { data: user } = useSuspenseQuery(userQuery(id));
  return <h1>{user.name}</h1>;            // data is defined
}

// Dependent query + select
function Projects({ userId }) {
  const { data: names } = useQuery({
    queryKey: ['projects', userId],
    queryFn: () => api.get(`/projects?owner=${userId}`),
    enabled: !!userId,
    select: (projects) => projects.map((p) => p.name),
  });
}

// Infinite feed
function Feed() {
  const { data, fetchNextPage, hasNextPage, isFetchingNextPage } =
    useInfiniteQuery({
      queryKey: ['feed'],
      queryFn: ({ pageParam }) => api.get(`/feed?cursor=${pageParam}`),
      initialPageParam: 0,
      getNextPageParam: (last) => last.nextCursor ?? undefined,
    });
  const posts = data?.pages.flatMap((p) => p.items) ?? [];
  return <>
    {posts.map((p) => <Post key={p.id} post={p} />)}
    <button onClick={() => fetchNextPage()}
      disabled={!hasNextPage || isFetchingNextPage}>Load more</button>
  </>;
}

// Prefetch on hover
const qc = useQueryClient();
<Link to={`/users/${id}`}
  onMouseEnter={() => qc.prefetchQuery(userQuery(id))} />;
```

ADVANCED HOOKS

HOOK	USE
`useSuspenseQuery`	Suspense + error boundaries
`useInfiniteQuery`	cursor/page feeds
`useQueries`	dynamic list of queries
`useIsFetching`	global loading indicator
`useMutationState`	read pending mutations

OPTIONS

OPTION	USE
`enabled`	dependent / conditional
`select`	derive or pick fields
`placeholderData: keepPreviousData`	smooth pagination
`initialData`	seed from other cache
`refetchInterval`	polling

⚠ COMMON MISTAKES

- Calling `useSuspenseQuery` without a Suspense boundary above
- Building infinite lists with page-number state and manual merging
- Inline `select` functions that do heavy work on every render

✓ TIPS & BEST PRACTICES

- Export `queryOptions` factories from feature modules
- Prefetch in route loaders: `await qc.ensureQueryData(userQuery(id))`
- Use `keepPreviousData` for paginated tables

QUICK SUMMARY

- `queryOptions` = reusable query
- `useSuspenseQuery` for Suspense
- `useInfiniteQuery` for feeds
- `enabled`, `select`, prefetch

SWR

useSWR · fetcher · revalidateOnFocus · mutate · useSWRMutation · conditional keys · SWRConfig

STALE-WHILE-REVALIDATE

SWR, from Vercel, is a small data-fetching hook named after the HTTP caching strategy: return **cached (stale)** data immediately, then **revalidate** in the background and update. `useSWR(key, fetcher)` returns `data`, `error`, `isLoading` and `mutate`. The key (usually the URL) identifies the cache entry; pass `null` to skip fetching. `useSWRMutation` handles remote mutations.

NOTES

- Requests with the same key are deduplicated
- Revalidates on focus, on reconnect and on mount by default
- `mutate()` (bound) revalidates this key; global `mutate(key, data)` updates any key
- `SWRConfig` sets the fetcher and options for a subtree

♿ ACCESSIBILITY

- Show `isLoading` and `error` states with live-region text
- Avoid jumping content when revalidated data arrives; keep layout stable

VERSION NOTES

- SWR 2.x is current; supports React 18 and 19
- `useSWRMutation` lives in `swr/mutation`; `useSWRInfinite` in `swr/infinite`

CODE EXPLAINED

→ The global fetcher in `SWRConfig` is used when `useSWR` gets only a key

→ The URL key caches each user separately

→ A `null` key keeps the team request idle until the user loads

→ Bound `mutate()` revalidates just this key

→ `trigger(arg)` calls `updateUser` and updates the cache with its result

USESWR AND USESWRMUTATION

```
import useSWR, { SWRConfig, useSWRConfig } from 'swr';
import useSWRMutation from 'swr/mutation';

const fetcher = async (url) => {
  const res = await fetch(url);
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return res.json();
};

export function App() {
  return (
    <SWRConfig value={{ fetcher, revalidateOnFocus: true }}>
      <Profile id="42" />
    </SWRConfig>
  );
}

function Profile({ id }) {
  const { data, error, isLoading, mutate } = useSWR(`/api/users/${id}`);
  // Dependent: skip until data.teamId exists
  const { data: team } = useSWR(data ? `/api/teams/${data.teamId}` : null);

  if (isLoading) return <p role="status">Loading…</p>;
  if (error) return <p role="alert">{error.message}</p>;
  return (
    <>
      <h1>{data.name}</h1>
      <button onClick={() => mutate()}>Refresh</button>
    </>
  );
}

async function updateUser(url, { arg }) {
  return fetch(url, { method: 'PATCH', body: JSON.stringify(arg) })
    .then((r) => r.json());
}

function Rename({ id }) {
  const { trigger, isMutating } =
    useSWRMutation(`/api/users/${id}`, updateUser);
  return <button disabled={isMutating}
    onClick={() => trigger({ name: 'Ada' })}>Rename</button>;
}
```

USESWR RESULT

FIELD	MEANING
`data`	cached data
`error`	fetch error
`isLoading`	first load, no data
`isValidating`	any request in flight
`mutate(data?, opts?)`	update/revalidate

SWR VS TANSTACK QUERY

SWR	TANSTACK QUERY
`Smaller API`	More features
`Key = URL string`	Key = array
`mutate for updates`	`invalidateQueries`
`Basic devtools via extension`	Official devtools
`Good for simple reads`	Good for complex apps

⚠ COMMON MISTAKES

- Fetcher that doesn't throw on HTTP errors
- Building keys from objects that change identity every render
- Using global `mutate` with a key that doesn't match exactly

✓ TIPS & BEST PRACTICES

- Use URLs as keys; they're unique and readable
- Set `dedupingInterval` and `refreshInterval` per use case
- Pick one data library per app; don't mix SWR and TanStack Query

QUICK SUMMARY

- `useSWR(key, fetcher)`
- Stale first, then revalidate
- `null` key = don't fetch
- `useSWRMutation` for writes

Choosing State Management

local vs global · server vs client state · URL state · context vs store · decision table

PICKING THE RIGHT HOME

Before adding a library, sort your state by **kind**. *Local UI state* (open/closed, input text) belongs in `useState`. *Server state* (data from an API) belongs in a data library such as TanStack Query. *URL state* (filters, pages, selected tab) belongs in the router. *Form state* belongs in the form or a form library. Only what's left, truly global **client state** shared by distant components, needs context or a store.

NOTES

- Most *global state* problems are really server-state caching problems
- Context is fine for values that change rarely (theme, user, locale)
- Stores (Zustand, Redux, Jotai) add selectors so components re-render only for what they read
- You can combine: TanStack Query for data, Zustand for UI preferences

♿ ACCESSIBILITY

- Keep user preferences such as reduced motion, font size and high contrast in persisted global state so every screen honors them

VERSION NOTES

- React 19 Actions and `useOptimistic` reduce the need for client stores around forms
- Redux Toolkit 2, Zustand 5 and Jotai 2 are the current majors

CODE EXPLAINED

→ `isOpen` matters to one component, so it stays local

→ Orders come from the server, so the data library caches them

→ `page` lives in the URL so it's shareable and survives reloads

→ Theme changes rarely, so context is enough

→ The cart changes often and is read in many places, so a store with selectors fits

WHERE EACH PIECE LIVES

```
// Local UI state
const [isOpen, setIsOpen] = useState(false);

// Server state
const { data: orders } = useQuery({
  queryKey: ['orders', page], queryFn: () => getOrders(page),
});

// URL state
const [params, setParams] = useSearchParams();
const page = Number(params.get('page') ?? 1);

// Form state
<form action={submitOrder}>…</form>

// Rarely-changing global: context
<ThemeContext value={theme}>…</ThemeContext>

// Frequently-changing global client state: store
const useCart = create((set) => ({
  items: [],
  add: (item) => set((s) => ({ items: [...s.items, item] })),
}));
const count = useCart((s) => s.items.length);

// Decision order:
//  1. Can it be derived?            -> compute it
//  2. Used by one component?        -> useState
//  3. From the server?              -> TanStack Query / loader
//  4. Should survive refresh/share? -> URL
//  5. Shared, changes rarely?       -> context
//  6. Shared, changes often?        -> store
```

STATE KINDS

KIND	TOOL
`Local UI`	`useState` / `useReducer`
`Server data`	TanStack Query, SWR, loaders
`URL`	router search params
`Forms`	Actions, React Hook Form
`Global, rare changes`	context
`Global, frequent changes`	Zustand, Redux, Jotai

STORE COMPARISON

LIBRARY	STYLE
`Redux Toolkit`	single store, slices, devtools, middleware
`Zustand`	small hook-based store
`Jotai`	atoms, bottom-up
`Valtio`	proxy-based mutable
`XState`	state machines

⚠ COMMON MISTAKES

- Putting API responses in Redux and hand-writing caching
- Global stores for state only one component uses
- Duplicating URL state in a store

✓ TIPS & BEST PRACTICES

- Start with built-in tools; add a store when you feel the pain
- Keep stores small and split by domain
- Write a short state map for each feature

QUICK SUMMARY

- Classify state first
- Server state → data library
- URL state → router
- Store only for global client state

Redux Toolkit: Store & Slices

MODERN REDUX

Redux Toolkit (RTK) is the official way to write Redux. `configureStore` creates the store with good defaults (DevTools, thunk middleware, mutation checks). `createSlice` defines a slice of state with **reducers that look mutating** but use Immer to produce immutable updates, and generates action creators automatically. React components read state with `useSelector` and send actions with `useDispatch` from **react-redux**.

NOTES

- Each slice owns one domain: `cart`, `auth`, `ui`
- `PayloadAction<T>` types the action payload in TypeScript
- Selectors should return the smallest value needed
- Typed hooks: `useSelector.withTypes<RootState>()`

♿ ACCESSIBILITY

- Redux DevTools time-travel helps reproduce focus and announcement bugs tied to specific state sequences

VERSION NOTES

- Redux Toolkit 2 and react-redux 9 are current; both support React 19
- RTK 2 removed the object syntax for `extraReducers` (use the builder callback)

CODE EXPLAINED

- → `push` looks like mutation, but Immer turns it into a new array
- → `cartSlice.actions` contains `added` and `removed` action creators
- → `selectors` (RTK 2) attaches selectors to the slice
- → `withTypes` creates hooks typed once for the whole app
- → `useAppSelector(selectCount)` re-renders only when the count changes

STORE, SLICE, HOOKS

```ts
// features/cart/cartSlice.ts
import { createSlice, type PayloadAction } from '@reduxjs/toolkit';

type Item = { id: string; name: string; qty: number };

const cartSlice = createSlice({
  name: 'cart',
  initialState: { items: [] as Item[] },
  reducers: {
    added(state, action: PayloadAction<Item>) {
      state.items.push(action.payload);          // Immer
    },
    removed(state, action: PayloadAction<string>) {
      state.items = state.items.filter((i) => i.id !== action.payload);
    },
  },
  selectors: {
    selectCount: (state) => state.items.length,
  },
});
export const { added, removed } = cartSlice.actions;
export const { selectCount } = cartSlice.selectors;
export default cartSlice.reducer;

// app/store.ts
import { configureStore } from '@reduxjs/toolkit';
export const store = configureStore({ reducer: { cart: cartReducer } });
export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;

// app/hooks.ts
export const useAppSelector = useSelector.withTypes<RootState>();
export const useAppDispatch = useDispatch.withTypes<AppDispatch>();

// main.tsx
<Provider store={store}><App /></Provider>

// Component
function CartBadge() {
  const count = useAppSelector(selectCount);
  const dispatch = useAppDispatch();
  return <button onClick={() => dispatch(removed('a1'))}>{count}</button>;
}
```

RTK APIS

API	PURPOSE
`configureStore`	create store
`createSlice`	reducers + actions
`createAsyncThunk`	async logic (sheet 83)
`createSelector`	memoized derived data
`createEntityAdapter`	normalized collections
`createListenerMiddleware`	side effects

REACT-REDUX

API	USE
`<Provider store>`	provide store
`useSelector(fn)`	read state
`useDispatch()`	dispatch actions
`.withTypes<T>()`	typed hooks
`useStore()`	raw store (rare)

⚠ COMMON MISTAKES

- Hand-writing action types and switch reducers (legacy Redux)
- Selecting the whole state object and re-rendering on every change
- Returning *and* mutating the draft in the same reducer

✓ TIPS & BEST PRACTICES

- Organize by feature: `features/cart/cartSlice.ts`
- Use RTK Query (sheet 83) for server data instead of slices
- Install the Redux DevTools browser extension

QUICK SUMMARY

- `configureStore` + `createSlice`
- Immer lets reducers *mutate*
- `useSelector` / `useDispatch`
- Typed hooks via `withTypes`

Redux Toolkit: Async & RTK Query

createAsyncThunk · extraReducers · createApi · fetchBaseQuery · providesTags / invalidatesTags · generated hooks

ASYNC LOGIC AND RTK QUERY

`createAsyncThunk` wraps an async function and dispatches **pending**, **fulfilled** and **rejected** actions that slices handle in `extraReducers`. For API data, **RTK Query** is usually better: `createApi` defines endpoints, generates hooks like `useGetPostsQuery`, and handles caching, deduplication and refetching. **Tags** connect queries and mutations so writes invalidate the right reads.

NOTES

- `thunkAPI.rejectWithValue(x)` sends a custom error payload
- RTK Query adds its reducer and middleware to the store
- `providesTags` labels cached data; `invalidatesTags` refetches it after mutations
- Query hooks return `data`, `isLoading`, `isFetching`, `error` and `refetch`

♿ ACCESSIBILITY

- Use `isLoading` and `isError` to render accessible status messages, as with any data hook

VERSION NOTES

- RTK Query ships inside `@reduxjs/toolkit` (`@reduxjs/toolkit/query/react`)
- RTK 2: `extraReducers` uses only the builder callback

CODE EXPLAINED

- → The thunk dispatches `user/fetch/pending`, then `fulfilled` or `rejected`
- → `rejectWithValue` puts the HTTP status in `action.payload`
- → `extraReducers` handles actions defined outside the slice
- → `createApi` generates `useGetPostsQuery` and `useAddPostMutation`
- → `invalidatesTags: ['Post']` refetches `getPosts` after adding

CREATEASYNCTHUNK AND CREATEAPI

```ts
import { createAsyncThunk, createSlice } from '@reduxjs/toolkit';

export const fetchUser = createAsyncThunk('user/fetch',
  async (id: string, { rejectWithValue }) => {
    const res = await fetch(`/api/users/${id}`);
    if (!res.ok) return rejectWithValue(res.status);
    return (await res.json()) as User;
  });

const userSlice = createSlice({
  name: 'user',
  initialState: { data: null as User | null, status: 'idle' },
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchUser.pending, (s) => { s.status = 'loading'; })
      .addCase(fetchUser.fulfilled, (s, a) => {
        s.status = 'idle'; s.data = a.payload; })
      .addCase(fetchUser.rejected, (s) => { s.status = 'failed'; });
  },
});

// RTK Query
import { createApi, fetchBaseQuery } from '@reduxjs/toolkit/query/react';

export const api = createApi({
  reducerPath: 'api',
  baseQuery: fetchBaseQuery({ baseUrl: '/api' }),
  tagTypes: ['Post'],
  endpoints: (build) => ({
    getPosts: build.query<Post[], void>({
      query: () => 'posts', providesTags: ['Post'] }),
    addPost: build.mutation<Post, Partial<Post>>({
      query: (body) => ({ url: 'posts', method: 'POST', body }),
      invalidatesTags: ['Post'] }),
  }),
});
export const { useGetPostsQuery, useAddPostMutation } = api;

// store: reducer: { [api.reducerPath]: api.reducer },
//   middleware: (gDM) => gDM().concat(api.middleware)

function Posts() {
  const { data = [], isLoading } = useGetPostsQuery();
  const [addPost, { isLoading: saving }] = useAddPostMutation();
}
```

THUNK LIFECYCLE

ACTION	WHEN
`pending`	before the async call
`fulfilled`	resolved; payload = return value
`rejected`	threw or `rejectWithValue`
`.unwrap()`	on dispatch result: throw on reject

RTK QUERY HOOKS

HOOK	GIVES
`useXQuery(arg)`	`data, isLoading, error`
`useLazyXQuery()`	`[trigger, result]`
`useXMutation()`	`[mutate, { isLoading }]`
`skip / skipToken`	conditional fetch
`pollingInterval`	polling

⚠ COMMON MISTAKES

- Storing API data in slices and rebuilding caching by hand
- Forgetting to add `api.middleware` to the store
- Tags that don't match between `providesTags` and `invalidatesTags`

✓ TIPS & BEST PRACTICES

- Use RTK Query for server data, slices for client state
- Use `await dispatch(thunk()).unwrap()` in handlers to catch errors
- Generate endpoints from OpenAPI with `@rtk-query/codegen-openapi`

QUICK SUMMARY

- `createAsyncThunk` → pending/fulfilled/rejected
- Handle in `extraReducers` builder
- `createApi` generates hooks
- Tags invalidate cached queries

Zustand

create · set / get · selectors · useShallow · persist · devtools · slices pattern

A STORE IN ONE HOOK

Zustand creates a global store with a single function: `create((set, get) => ({ …state, …actions }))` returns a **hook**. Components call it with a **selector** and re-render only when the selected value changes. There's no provider, no reducers and no boilerplate. Middleware adds `persist` (localStorage), `devtools` (Redux DevTools) and `immer`.

NOTES

- `set` merges the object you pass into state (one level deep)
- `get()` reads current state inside actions
- Select multiple values with `useShallow` to avoid re-renders from new objects
- Access state outside React with `useStore.getState()` and `useStore.setState()`

♿ ACCESSIBILITY

- Persist accessibility preferences (font size, reduced motion) with the `persist` middleware

VERSION NOTES

- Zustand 5 is current: requires React 18+, uses `useSyncExternalStore`, drops the default export
- TypeScript: use the curried form `create<State>()(…)`

CODE EXPLAINED

→ `create<CartState>()(…)` is the curried form that types middleware correctly

→ `persist` saves state to `localStorage` under `cart`

→ `devtools` connects the store to Redux DevTools

→ Selecting `items.length` means `CartBadge` ignores other changes

→ `useShallow` compares the selected object's fields, not its identity

CREATE, SELECTORS, MIDDLEWARE

```tsx
import { create } from 'zustand';
import { devtools, persist } from 'zustand/middleware';
import { useShallow } from 'zustand/react/shallow';

type CartState = {
  items: { id: string; qty: number }[];
  add: (id: string) => void;
  clear: () => void;
  total: () => number;
};

export const useCart = create<CartState>()(
  devtools(
    persist(
      (set, get) => ({
        items: [],
        add: (id) => set((s) => ({ items: [...s.items, { id, qty: 1 }] })),
        clear: () => set({ items: [] }),
        total: () => get().items.reduce((n, i) => n + i.qty, 0),
      }),
      { name: 'cart' },          // localStorage key
    ),
  ),
);

function CartBadge() {
  const count = useCart((s) => s.items.length);   // re-renders on count
  return <span>{count}</span>;
}

function CartActions() {
  const { add, clear } = useCart(
    useShallow((s) => ({ add: s.add, clear: s.clear })),
  );
  return <button onClick={() => add('a1')}>Add</button>;
}

// Outside React
useCart.getState().clear();
const unsub = useCart.subscribe((s) => console.log(s.items));
```

API

MEMBER	USE
`create(fn)`	make a store hook
`set(partial or fn)`	update
`get()`	read inside actions
`useStore(selector)`	read in components
`useStore.getState()`	read outside React
`useStore.subscribe(fn)`	listen outside React

MIDDLEWARE

MIDDLEWARE	ADDS
`persist`	storage sync
`devtools`	Redux DevTools
`immer`	mutable-style updates
`subscribeWithSelector`	selective subscribe
`combine`	infer types from initial state

⚠ COMMON MISTAKES

- Calling `useStore()` with no selector and re-rendering on every change
- Returning new objects from selectors without `useShallow`
- Storing server data that belongs in TanStack Query

✓ TIPS & BEST PRACTICES

- One store per domain (`useCart`, `useUI`)
- Keep actions in the store so components stay simple
- Reset stores between tests with `setState(initial)`

QUICK SUMMARY

- `create((set, get) => ({…}))`
- Select only what you need
- `useShallow` for multiple fields
- `persist`, `devtools` middleware

Rendering Strategies

WHERE HTML GETS MADE

React can render in the browser, on the server per request, or at build time. **CSR** (client-side rendering) ships an empty HTML shell and builds the UI in JavaScript. **SSR** renders HTML on each request, then **hydrates** it so it becomes interactive. **SSG** renders HTML at build time. **Streaming** sends HTML in chunks as Suspense boundaries resolve. **Server Components** run only on the server and send no JavaScript for themselves.

NOTES

- Hydration attaches event handlers to existing server HTML; output must match
- SSR improves first paint and SEO but needs a server
- SSG suits content that changes rarely (docs, marketing)
- Frameworks (Next.js, React Router framework mode) handle these choices for you

♿ ACCESSIBILITY

- Server-rendered HTML is readable before JavaScript loads and works with slow devices and assistive tech
- Avoid hydration mismatches that cause content to flash and confuse screen readers

VERSION NOTES

- React 18 added streaming SSR with Suspense; React 19 made Server Components and Server Functions stable for frameworks
- React 19.2 added Partial Pre-rendering APIs (`prerender` + `resume`)

CODE EXPLAINED

→ CSR renders everything after JavaScript downloads

→ SSR sends HTML; `hydrateRoot` makes it interactive

→ `prerender` waits for all data and returns static HTML

→ Suspense boundaries let the server stream slow sections later

→ A Server Component queries the database and ships only the result

STRATEGIES AT A GLANCE

```
// CSR: index.html has <div id="root"></div>
createRoot(document.getElementById('root')).render(<App />);

// SSR (per request) + hydration
// server
const html = await renderToReadableStream(<App url={req.url} />);
// client
hydrateRoot(document.getElementById('root'), <App />);

// SSG (build time)
for (const path of ['/', '/about', '/blog/hello']) {
  const { prelude } = await prerender(<App url={path} />);
  await writeFile(`dist${path}/index.html`, prelude);
}

// Streaming: shell first, slow parts later
function Page() {
  return (
    <Layout>
      <Hero />
      <Suspense fallback={<ReviewsSkeleton />}>
        <Reviews />        {/* streams in when ready */}
      </Suspense>
    </Layout>
  );
}

// Server Component (framework): no client JS for this file
export default async function Products() {
  const products = await db.products.findMany();
  return <ProductGrid products={products} />;
}

// Rendering timeline (SSR + streaming):
// request -> shell HTML -> boundary chunks -> JS -> hydrate
```

STRATEGIES

STRATEGY	BEST FOR
`CSR`	dashboards behind login
`SSR`	dynamic, SEO-sensitive pages
`SSG`	docs, blogs, marketing
`Streaming SSR`	pages with slow sections
`Server Components`	data-heavy UI, less client JS
`Partial pre-rendering`	static shell + dynamic holes

TRADE-OFFS

CONCERN	EFFECT
`Time to first content`	SSR/SSG faster than CSR
`Server cost`	SSR highest; SSG/CSR lowest
`Interactivity`	after hydration
`SEO`	SSR/SSG best
`Complexity`	frameworks recommended

⚠ COMMON MISTAKES

- Building SSR by hand when a framework fits
- Rendering browser-only values (dates, `window`) differently on server and client
- One giant Suspense boundary that delays the whole page

✓ TIPS & BEST PRACTICES

- Choose per route: static where possible, dynamic where needed
- Use streaming to show the shell immediately
- Measure Core Web Vitals to pick a strategy

QUICK SUMMARY

- CSR, SSR, SSG, streaming
- Hydration attaches interactivity
- Server Components send no JS
- Frameworks make these choices easy

Server Components

server vs client components · async components · 'use client' · serializable props · passing JSX across the boundary · server-only

COMPONENTS THAT STAY ON THE SERVER

Server Components run only on the server (at request or build time). They can be `async`, read databases and files directly, and use secrets, and their code never ships to the browser. **Client Components**, marked with `'use client'` at the top of a file, run on both server (for HTML) and client, and can use state, effects and event handlers. Server Components can render Client Components and pass them **serializable** props.

NOTES

- In RSC frameworks, components are Server Components by default
- `'use client'` marks a **boundary**: that module and everything it imports become client code
- Server Components can't use `useState`, `useEffect`, context providers or browser APIs
- Pass Server Components *into* Client Components as `children` or other JSX props

♿ ACCESSIBILITY

- Server-rendered markup should be complete and semantic; interactivity added on the client should not change roles or structure

VERSION NOTES

- Server Components are stable in React 19 and require a framework or bundler integration (Next.js App Router, React Router RSC, Parcel, Vite plugin)
- React 19.3: `<Context>` can be rendered in Server Components

CODE EXPLAINED

→ The page is `async` and queries the database during render

→ `import 'server-only'` makes the build fail if this file is imported by client code

→ `AddToCart` is a Client Component because it needs state and a click handler

→ `ProductDetails` stays a Server Component even inside the client `Collapsible`

→ Props crossing to the client must be serializable

SERVER AND CLIENT TOGETHER

```tsx
// app/products/page.tsx  (Server Component)
import { db } from '@/lib/db';
import 'server-only';
import AddToCart from './AddToCart';
import Collapsible from './Collapsible';

export default async function ProductsPage() {
  const products = await db.product.findMany();   // direct DB access
  return (
    <ul>
      {products.map((p) => (
        <li key={p.id}>
          {p.name} ${p.price}
          <AddToCart productId={p.id} />     {/* client island */}
          <Collapsible title="Details">
            <ProductDetails id={p.id} />      {/* server, as children */}
          </Collapsible>
        </li>
      ))}
    </ul>
  );
}

// app/products/AddToCart.tsx  (Client Component)
'use client';
import { useState } from 'react';

export default function AddToCart({ productId }: { productId: string }) {
  const [added, setAdded] = useState(false);
  return (
    <button onClick={() => setAdded(true)} aria-pressed={added}>
      {added ? 'Added' : 'Add to cart'}
    </button>
  );
}

// ✗ Not serializable across the boundary
// <AddToCart onAdd={() => db.add()} />   functions (except Server Functions)
// <AddToCart product={new Product()} />  class instances
```

SERVER VS CLIENT

SERVER COMPONENT	CLIENT COMPONENT
Default in RSC apps	`'use client'` at top
async/await data	hooks, state, effects
DB, files, secrets	browser APIs, events
No JS shipped	JS shipped for hydration
Can render Client Components	Can't import Server Components

SERIALIZABLE PROPS

ALLOWED	NOT ALLOWED
strings, numbers, booleans, null	functions
arrays, plain objects	class instances
Date, Map, Set, BigInt	symbols (unregistered)
JSX / Server Components	DOM nodes
Promises, Server Functions	closures over secrets

⚠ COMMON MISTAKES

- Adding `'use client'` to every file and losing the benefits
- Importing a Server Component into a Client Component (it becomes client code)
- Leaking secrets by passing whole DB rows as props

✓ TIPS & BEST PRACTICES

- Push `'use client'` down to the smallest interactive leaves
- Pass server-rendered content into client wrappers as `children`
- Use `server-only` and `client-only` packages to enforce boundaries

QUICK SUMMARY

- Server by default, `'use client'` opts in
- Server: async, DB, no JS shipped
- Client: state, effects, events
- Serializable props across the boundary

Server Functions

'use server' · calling from the client · forms + actions · validation & auth · framework support · closures

CALLING THE SERVER FROM THE CLIENT

A **Server Function** is an `async` function marked with `'use server'`. The framework turns it into an endpoint, and Client Components can call it like a normal function or pass it to `<form action>`. React serializes the arguments, runs the function on the server, and returns the result. Treat every Server Function as a **public API**: validate inputs and check authorization inside it.

NOTES

- Put `'use server'` at the top of a file to mark every export, or inside a function body
- Server Functions used in `action` props are also called *Server Actions*
- Arguments and return values must be serializable
- Forms using Server Functions work before JavaScript loads (progressive enhancement)

♿ ACCESSIBILITY

- Return field errors and render them next to inputs; with progressive enhancement, the server must re-render the form with errors

VERSION NOTES

- Server Functions are stable in React 19 with a supporting framework
- Frameworks encrypt closed-over values, but you should still never close over secrets

CODE EXPLAINED

- → `'use server'` at the top makes both exports Server Functions
- → Authorization runs inside the function, every time
- → Zod validates `FormData` because anyone can call the endpoint
- → `useActionState` shows the returned error and pending state
- → `likePost` is called directly from a click handler

'USE SERVER' FUNCTIONS

```ts
// app/actions.ts
'use server';
import { z } from 'zod';
import { revalidatePath } from 'next/cache';   // framework API
import { auth } from '@/lib/auth';

const Comment = z.object({
  postId: z.string(), body: z.string().min(1).max(500),
});

export async function addComment(prevState, formData: FormData) {
  const session = await auth();
  if (!session) return { error: 'Please sign in' };

  const parsed = Comment.safeParse(Object.fromEntries(formData));
  if (!parsed.success) return { error: 'Comment is empty or too long' };

  await db.comment.create({ data: { ...parsed.data,
    authorId: session.userId } });
  revalidatePath(`/posts/${parsed.data.postId}`);
  return { error: null };
}

export async function likePost(postId: string) {
  const session = await auth();
  if (!session) throw new Error('Unauthorized');
  return db.like.toggle(postId, session.userId);
}

// CommentForm.tsx
'use client';
import { useActionState } from 'react';
import { addComment, likePost } from './actions';

export function CommentForm({ postId }) {
  const [state, action, pending] = useActionState(addComment,
    { error: null });
  return (
    <form action={action}>
      <input type="hidden" name="postId" value={postId} />
      <textarea name="body" aria-describedby="err" />
      <p id="err" role="alert">{state.error}</p>
      <button disabled={pending}>Post</button>
      <button type="button" onClick={() => likePost(postId)}>♥</button>
    </form>
  );
}
```

CALLING STYLES

STYLE	EXAMPLE
Form action	`<form action={fn}>`
Button override	`<button formAction={fn}>`
With state	`useActionState(fn, init)`
Direct call	`await fn(args)` in a handler
In a transition	`startTransition(() => fn())`

SECURITY CHECKLIST

CHECK	WHY
Authenticate	endpoint is public
Authorize the resource	user owns it?
Validate input	client can send anything
Return minimal data	avoid leaking fields
Rate-limit sensitive actions	abuse

⚠ COMMON MISTAKES

- Assuming a Server Function is private because it isn't linked in the UI
- Trusting hidden form fields without checking ownership
- Returning whole database records to the client

✓ TIPS & BEST PRACTICES

- Keep Server Functions in `actions.ts` files per feature
- Return `{ error }` objects for expected failures; throw for bugs
- Revalidate or update caches after writes

QUICK SUMMARY

- `'use server'` async functions
- Callable from forms and handlers
- Public endpoints: auth + validate
- Serializable args and results

Server Rendering APIs

REACT-DOM/SERVER AND HYDRATION

`react-dom/server` renders a React tree to HTML. `renderToReadableStream` (Web Streams: Deno, Bun, edge, modern Node) and `renderToPipeableStream` (Node streams) **stream** HTML and support Suspense. `prerender` from `react-dom/static` waits for all data, for static generation. On the client, `hydrateRoot` attaches React to the HTML. If server and client output differ, React reports a **hydration mismatch** and re-renders that part on the client.

NOTES

- `bootstrapScripts` / `bootstrapModules` add the client entry script tags
- `onShellReady` / awaiting the stream gives you the shell before Suspense content
- `renderToString` still exists but doesn't support streaming or waiting for Suspense data
- `suppressHydrationWarning` silences one element's text mismatch (e.g. timestamps)

♿ ACCESSIBILITY

- Avoid rendering different text on server and client (dates, random IDs); use `useId` for IDs

VERSION NOTES

- `prerender` / `prerenderToNodeStream` (`react-dom/static`) are new in React 19
- React 19.2 adds `resume` APIs for Partial Pre-rendering
- React 19 shows a single diff-style error for hydration mismatches

CODE EXPLAINED

- → `renderToReadableStream` returns a stream you can send as a `Response`
- → `bootstrapModules` adds the script that will hydrate the page
- → `stream.allReady` waits for every Suspense boundary, useful for crawlers
- → `prerender` resolves all data and returns static HTML
- → `suppressHydrationWarning` allows a known one-level text mismatch

SERVER RENDER AND HYDRATE

```tsx
// server.ts (Web Streams)
import { renderToReadableStream } from 'react-dom/server';

export async function handler(request: Request) {
  let status = 200;
  const stream = await renderToReadableStream(
    <App url={request.url} />,
    {
      bootstrapModules: ['/assets/client.js'],
      onError(error) { status = 500; console.error(error); },
    },
  );
  // await stream.allReady;   // wait for everything (bots)
  return new Response(stream, {
    status, headers: { 'Content-Type': 'text/html' },
  });
}

// Node streams
import { renderToPipeableStream } from 'react-dom/server';
const { pipe } = renderToPipeableStream(<App />, {
  bootstrapScripts: ['/client.js'],
  onShellReady() { res.setHeader('content-type', 'text/html'); pipe(res); },
});

// Static generation
import { prerender } from 'react-dom/static';
const { prelude } = await prerender(<App url="/about" />);

// client.tsx
import { hydrateRoot } from 'react-dom/client';
hydrateRoot(document, <App url={location.href} />, {
  onRecoverableError: (e) => report(e),
});

// Intentional difference
<time suppressHydrationWarning>{new Date().toLocaleTimeString()}</time>
```

SERVER APIS

API	ENVIRONMENT
`renderToReadableStream`	Web Streams (edge, Bun, Deno)
`renderToPipeableStream`	Node streams
`prerender`	static, Web Streams
`prerenderToNodeStream`	static, Node
`renderToString`	legacy, no streaming

MISMATCH CAUSES

CAUSE	FIX
`Date.now()` / `locale formatting`	render on client or pass from server
`Math.random()` `IDs`	`useId`
`typeof window branches`	`useEffect` or `useSyncExternalStore`
`Invalid HTML nesting`	fix markup (in <p>)
`Browser extensions`	usually harmless

⚠ COMMON MISTAKES

- Reading `window` or `localStorage` during render
- Using `suppressHydrationWarning` to hide real bugs
- Choosing `renderToString` for new SSR code

✓ TIPS & BEST PRACTICES

- Use a framework unless you need a custom server
- Send the shell fast; stream the rest
- Wire `onRecoverableError` to logging to catch mismatches in production

QUICK SUMMARY

- Stream with `renderTo*Stream`
- `prerender` for static HTML
- `hydrateRoot` on the client
- Same output on both sides

Accessible JSX

ACCESSIBLE MARKUP IN JSX

React renders whatever HTML you write, so accessibility starts with **semantic elements**: `<button>` for actions, `<a href>` for navigation, `<label>` for inputs, headings in order, and landmarks (`<header>`, `<nav>`, `<main>`). JSX supports every ARIA attribute with its HTML spelling (`aria-label`, `aria-expanded`). Use `htmlFor` and `useId` to link labels, and lint with **`eslint-plugin-jsx-a11y`**.

NOTES

- `aria-*` and `role` keep lowercase, hyphenated HTML names
- Boolean ARIA values can be booleans in JSX: `aria-expanded={open}`
- First rule of ARIA: use a native element before adding a role
- Fragments let components return valid list and table structures

♿ ACCESSIBILITY

- Every image needs `alt`; use `alt=""` for decorative ones
- Icon-only buttons need an accessible name (`aria-label` or visually hidden text)
- Never remove focus outlines without a visible replacement

VERSION NOTES

- React 19 adds no ARIA-specific APIs; `useId` (React 18) is the standard way to link labels
- WCAG 2.2 is the current W3C recommendation for conformance targets

CODE EXPLAINED

→ `role="search"` marks the form as a search landmark

→ `useId` links the label and hint to the input

→ `aria-label` names the icon button; the icon is hidden from screen readers

→ `aria-expanded` and `aria-controls` describe the disclosure state

→ A native `<button>` is focusable and responds to Enter and Space

SEMANTIC, LABELED, LINTED

```jsx
function SearchForm({ onSearch }) {
  const id = useId();
  return (
    <form role="search" onSubmit={onSearch}>
      <label htmlFor={id}>Search products</label>
      <input id={id} type="search" name="q"
        aria-describedby={`${id}-hint`} />
      <p id={`${id}-hint`}>Try “red shoes”</p>
      <button type="submit">Search</button>
    </form>
  );
}

function IconButton({ label, icon, ...props }) {
  return (
    <button type="button" aria-label={label} {...props}>
      <span aria-hidden="true">{icon}</span>
    </button>
  );
}

function Disclosure({ title, children }) {
  const [open, setOpen] = useState(false);
  const panelId = useId();
  return (
    <>
      <button aria-expanded={open} aria-controls={panelId}
        onClick={() => setOpen(!open)}>{title}</button>
      <div id={panelId} hidden={!open}>{children}</div>
    </>
  );
}

// ✗ Div button: no focus, no keyboard, no role
<div className="btn" onClick={save}>Save</div>
// ✓
<button className="btn" onClick={save}>Save</button>

// eslint.config.js
import jsxA11y from 'eslint-plugin-jsx-a11y';
export default [jsxA11y.flatConfigs.recommended];
```

JSX ACCESSIBILITY ATTRIBUTES

ATTRIBUTE	USE
`htmlFor`	link label → input
`aria-label`	name without visible text
`aria-labelledby`	name from another element
`aria-describedby`	hints and errors
`aria-expanded / aria-pressed`	toggle state
`aria-hidden`	hide decorative content

ELEMENT CHOICE

NEED	USE
Action	`<button>`
Navigation	`<a href>` / `<Link>`
Page regions	`<header>`, `<nav>`, `<main>`, `<footer>`
Lists	`<ul>`/`<ol>` + `<li>`
Data	`<table>` with `<th scope>`

⚠ COMMON MISTAKES

- Clickable `<div>` s and `<span>` s
- Placeholder text used as the only label
- `aria-label` on non-interactive elements where it's ignored

✓ TIPS & BEST PRACTICES

- Run axe DevTools or Lighthouse on every page
- Test with keyboard only: Tab, Shift+Tab, Enter, Space, Escape
- Use React Aria or Radix for complex widgets

QUICK SUMMARY

- Semantic elements first
- Label everything (`htmlFor`, `useId`)
- ARIA for state, not structure
- Lint with jsx-a11y

Focus Management & Live Regions

FOCUS AND ANNOUNCEMENTS

Single-page apps change content without page loads, so you must manage **focus** and **announcements** yourself. After route changes, move focus to the new page heading. When a dialog opens, move focus into it and **trap** it there; when it closes, **return** focus to the trigger. Announce async results (saved, 3 results, error) with an **`aria-live`** region. The native `<dialog>` element handles most dialog focus rules for you.

NOTES

- `<dialog>` opened with `showModal()` traps focus, closes on Escape and makes the page inert
- Headings that receive focus need `tabIndex={-1}`
- Live regions must exist in the DOM *before* their text changes
- `role="status"` is polite; `role="alert"` interrupts

♿ ACCESSIBILITY

- Provide a *Skip to main content* link as the first focusable element
- Keep focus visible: never set `outline: none` without a replacement

VERSION NOTES

- Native `<dialog>` and the `inert` attribute are supported in all current browsers
- React 19 passes `inert` through as a boolean attribute

CODE EXPLAINED

→ Focusing the `<h1>` on each route tells screen readers the page changed

→ `tabIndex={-1}` lets the heading take focus without entering the tab order

→ `showModal()` traps focus and returns it to the opener on close

→ `onClose` fires on Escape too, keeping React state in sync

→ The status paragraph is always rendered; only its text changes

ROUTE FOCUS, DIALOG, LIVE REGION

```jsx
// 1. Focus the heading after navigation
function Page({ title, children }) {
  const headingRef = useRef(null);
  const { pathname } = useLocation();
  useEffect(() => { headingRef.current?.focus(); }, [pathname]);
  return (
    <main id="main">
      <h1 ref={headingRef} tabIndex={-1}>{title}</h1>
      {children}
    </main>
  );
}

// 2. Native dialog with focus return
function ConfirmDialog({ open, onClose, children }) {
  const ref = useRef(null);
  useEffect(() => {
    const dlg = ref.current;
    if (open && !dlg.open) dlg.showModal();
    if (!open && dlg.open) dlg.close();
  }, [open]);
  return (
    <dialog ref={ref} onClose={onClose} aria-labelledby="dlg-title">
      <h2 id="dlg-title">Are you sure?</h2>
      {children}
      <button onClick={onClose}>Cancel</button>
    </dialog>
  );
}

// 3. Live region announcements
function SaveStatus({ status }) {
  return (
    <p role="status" aria-live="polite">
      {status === 'saved' ? 'Changes saved' : ''}
    </p>
  );
}

// 4. Skip link (first in body)
<a href="#main" className="skip-link">Skip to main content</a>
```

FOCUS RULES

EVENT	MOVE FOCUS TO
`Route change`	new <h1>
`Dialog opens`	first field or dialog heading
`Dialog closes`	the button that opened it
`Item deleted`	next item or list heading
`Form error`	error summary or first invalid field

LIVE REGIONS

MARKUP	BEHAVIOR
`role="status"`	polite, waits for pause
`role="alert"`	assertive, interrupts
`aria-live="polite"`	same as status
`aria-atomic="true"`	read whole region
`aria-busy="true"`	delay announcements

⚠ COMMON MISTAKES

- Mounting a live region together with its message (often not announced)
- Custom modals without focus trapping or Escape handling
- Losing focus to `<body>` after deleting the focused element

✓ TIPS & BEST PRACTICES

- Prefer `<dialog>` or a library (React Aria, Radix) over custom modals
- Keep one app-level live region and send messages to it
- Test with a real screen reader (VoiceOver, NVDA) regularly

QUICK SUMMARY

- Focus headings on route change
- Trap and return focus in dialogs
- `aria-live` for async results
- Skip link first

Testing Setup

Vitest · jsdom · @testing-library/react · jest-dom matchers · setupFiles · test scripts

VITEST + TESTING LIBRARY

Vitest is a fast, Vite-native test runner with a Jest-compatible API (`describe`, `it`, `expect`, `vi`). **React Testing Library** (RTL) renders components into a simulated DOM (**jsdom** or **happy-dom**) and queries them the way users find things: by role, label and text. **jest-dom** adds DOM matchers like `toBeInTheDocument()` and `toBeDisabled()`. A setup file wires them together.

NOTES

- Install: `vitest`, `jsdom`, `@testing-library/react`, `@testing-library/dom`, `@testing-library/jest-dom`, `@testing-library/user-event`
- `globals: true` enables `describe` / `it` / `expect` without imports and lets RTL auto-cleanup
- Test files: `*.test.tsx` next to the component
- Jest works the same way with `jest-environment-jsdom`

♿ ACCESSIBILITY

- Querying by role and label means a test fails when an element loses its accessible name: your tests double as accessibility checks

VERSION NOTES

- RTL 16 requires `@testing-library/dom` as a peer dependency and supports React 19
- Vitest browser mode can run tests in a real browser via Playwright

CODE EXPLAINED

- → `test` in `vite.config.ts` reuses the app's Vite plugins and aliases
- → `environment: 'jsdom'` provides `document` and `window`
- → The setup file registers jest-dom matchers with Vitest's `expect`
- → `render` mounts the component; `screen` queries the whole document
- → `getByRole('heading', { name })` finds the `<h1>` by its accessible name

CONFIG, SETUP, FIRST TEST

```ts
// vite.config.ts
/// <reference types="vitest/config" />
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',
    globals: true,
    setupFiles: './src/test/setup.ts',
    css: true,
  },
});

// src/test/setup.ts
import '@testing-library/jest-dom/vitest';

// package.json
// "test": "vitest", "test:run": "vitest run --coverage"

// src/components/Greeting.test.tsx
import { render, screen } from '@testing-library/react';
import Greeting from './Greeting';

describe('Greeting', () => {
  it('greets the user by name', () => {
    render(<Greeting name="Ada" />);
    expect(
      screen.getByRole('heading', { name: /hello, ada/i }),
    ).toBeInTheDocument();
  });

  it('shows a sign-in button when anonymous', () => {
    render(<Greeting />);
    expect(screen.getByRole('button', { name: 'Sign in' }))
      .toBeEnabled();
  });
});
```

PACKAGES

PACKAGE	ROLE
`vitest`	runner, assertions, mocks
`jsdom / happy-dom`	simulated DOM
`@testing-library/react`	`render`, `screen`, `renderHook`
`@testing-library/user-event`	realistic input
`@testing-library/jest-dom`	DOM matchers
`msw`	network mocking (sheet 94)

JEST-DOM MATCHERS

MATCHER	CHECKS
`toBeInTheDocument()`	present
`toBeVisible()`	visible
`toBeDisabled() / toBeEnabled()`	state
`toHaveValue(v)`	input value
`toHaveAccessibleName(n)`	accessible name
`toHaveAttribute(a, v)`	attribute

⚠ COMMON MISTAKES

- Forgetting the setup file, so `toBeInTheDocument` is undefined
- Testing implementation details (state values, class names)
- Snapshot tests of entire pages that nobody reviews

✓ TIPS & BEST PRACTICES

- Test behavior users see: text, roles, enabled/disabled
- Run `vitest` in watch mode while developing
- Use Playwright for full end-to-end flows

QUICK SUMMARY

- Vitest + jsdom + RTL
- Setup file loads jest-dom
- `render` → `screen.getByRole`
- Test what users see

Testing Library: Queries

render · screen · getByRole · query priority · getBy / queryBy / findBy · within · screen.debug()

FINDING ELEMENTS LIKE A USER

Testing Library queries come in three families. **`getBy*`** returns the element or throws, for things that should be there. **`queryBy*`** returns `null` instead of throwing, for asserting absence. **`findBy*`** returns a promise and waits, for things that appear asynchronously. Prefer queries in **priority order**: by role (with `name`), by label, by placeholder, by text, and only last by `data-testid`.

NOTES

- `getAllBy*` / `queryAllBy*` / `findAllBy*` return arrays
- `{ name: /save/i }` matches the accessible name with a regex
- `within(element)` scopes queries to part of the page
- `screen.debug()` prints the current DOM; `screen.logTestingPlaygroundURL()` suggests queries

♿ ACCESSIBILITY

- If `getByRole` can't find your button, assistive tech probably can't either: fix the markup, not the query

VERSION NOTES

- Query APIs are stable across RTL 14–16
- `*ByRole` supports `level` for headings and `checked`, `pressed`, `expanded`, `selected` filters

CODE EXPLAINED

- → `getByRole` with `name` and `level` pins down exactly one heading
- → `checked: false` filters checkboxes by state
- → `queryByRole` returns `null`, so `.not.toBeInTheDocument()` passes
- → `findByText` waits (default 1000 ms) for the text to appear
- → `within(reviews)` searches only inside the Reviews region

QUERIES IN PRIORITY ORDER

```
import { render, screen, within } from '@testing-library/react';

test('product card', async () => {
  render(<ProductPage id="42" />);

  // 1. Role + name (best)
  screen.getByRole('heading', { name: 'Blue Mug', level: 1 });
  screen.getByRole('button', { name: /add to cart/i });
  screen.getByRole('checkbox', { name: 'Gift wrap', checked: false });

  // 2. Label
  screen.getByLabelText('Quantity');

  // 3. Placeholder / text / display value
  screen.getByPlaceholderText('Promo code');
  screen.getByText(/free shipping/i);
  screen.getByDisplayValue('1');

  // 4. alt / title
  screen.getByAltText('Blue mug on a table');

  // 5. Test ID (last resort)
  screen.getByTestId('price-chart');

  // Absence
  expect(screen.queryByRole('alert')).not.toBeInTheDocument();

  // Async appearance
  expect(await screen.findByText('In stock')).toBeVisible();

  // Scope to a region
  const reviews = screen.getByRole('region', { name: 'Reviews' });
  expect(within(reviews).getAllByRole('article')).toHaveLength(3);

  screen.debug(reviews);
});
```

QUERY FAMILIES

PREFIX	NO MATCH	MANY	ASYNC
`getBy`	throws	throws	no
`queryBy`	`null`	throws	no
`findBy`	rejects	rejects	yes
`getAllBy`	throws	array	no
`queryAllBy`	`[]`	array	no
`findAllBy`	rejects	array	yes

QUERY PRIORITY

RANK	QUERY
1	`*ByRole`
2	`*ByLabelText`
3	`*ByPlaceholderText`
4	`*ByText`
5	`*ByDisplayValue`
6	`*ByAltText` / `*ByTitle`
7	`*ByTestId`

⚠ COMMON MISTAKES

- Using `getBy*` to assert something is absent (it throws)
- Wrapping `findBy*` in `waitFor` (it already waits)
- Reaching for `data-testid` first

✓ TIPS & BEST PRACTICES

- Use Testing Playground to find the best query
- Match names with case-insensitive regexes to survive copy changes
- Scope with `within` instead of adding test IDs

QUICK SUMMARY

- `getBy` present, `queryBy` absent, `findBy` async
- Role + name first
- `within()` to scope
- Test IDs last

Testing Library: User Interactions

userEvent.setup() · click · type · keyboard · fireEvent vs userEvent · act

SIMULATING REAL INPUT

user-event simulates complete interactions the way a browser would: a click fires pointer, mouse, focus and click events; typing fires key events and updates the value one character at a time. Create a session with `userEvent.setup()` at the start of each test and `await` every call. Use `fireEvent` only for events user-event doesn't cover. RTL wraps interactions in **`act`**, so you rarely call it yourself.

NOTES

- `user.type(el, 'text')` clicks the element, then types
- `{Enter}`, `{Escape}`, `{ArrowDown}`, `{Shift}A{/Shift}` describe special keys
- `user.keyboard('…')` types into whatever has focus
- `user.tab()` moves focus like the Tab key

♿ ACCESSIBILITY

- Test keyboard paths with `user.tab()` and `user.keyboard('{Enter}')` so keyboard support can't regress

VERSION NOTES

- user-event v14: all APIs are async; call `userEvent.setup()` before rendering
- React 19 `act` is imported from `react`; RTL re-exports it

CODE EXPLAINED

- → `userEvent.setup()` creates a session that tracks pointer and keyboard state
- → `{Enter}` submits the form after typing
- → `user.tab()` then `toHaveFocus()` checks keyboard order
- → `selectOptions` and `upload` cover selects and file inputs
- → `hover` then `findByRole('tooltip')` waits for the tooltip to appear

USER-EVENT API

CALL	SIMULATES
`click(el) / dblClick(el)`	pointer clicks
`type(el, text)`	click + typing
`clear(el)`	select all + delete
`keyboard('{Enter}')`	keys on focused element
`tab({ shift })`	focus navigation
`selectOptions(el, v) / upload(el, f)`	select, files
`hover(el) / unhover(el)`	pointer over

USER-EVENT IN PRACTICE

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';

test('adds a todo with the keyboard', async () => {
  const user = userEvent.setup();
  render(<TodoApp />);

  const input = screen.getByRole('textbox', { name: 'New todo' });
  await user.type(input, 'Buy milk{Enter}');

  expect(screen.getByRole('listitem')).toHaveTextContent('Buy milk');
  expect(input).toHaveValue('');
});

test('toggles and deletes', async () => {
  const user = userEvent.setup();
  render(<TodoApp initial={[{ id: '1', text: 'Walk dog' }]} />);

  await user.click(screen.getByRole('checkbox', { name: 'Walk dog' }));
  expect(screen.getByRole('checkbox')).toBeChecked();

  await user.tab();
  expect(screen.getByRole('button', { name: 'Delete' })).toHaveFocus();
  await user.keyboard('{Enter}');
  expect(screen.queryByRole('listitem')).not.toBeInTheDocument();
});

test('select and upload', async () => {
  const user = userEvent.setup();
  render(<ProfileForm />);
  await user.selectOptions(screen.getByLabelText('Plan'), 'pro');
  const file = new File(['img'], 'me.png', { type: 'image/png' });
  await user.upload(screen.getByLabelText('Avatar'), file);
  await user.hover(screen.getByRole('button', { name: 'Help' }));
  expect(await screen.findByRole('tooltip')).toBeVisible();
});
```

USER-EVENT VS FIREEVENT

USER-EVENT	FIREEVENT
Full event sequence	single event
Respects disabled, focus	doesn't check
Async (await)	sync
Default choice	edge cases only

⚠ COMMON MISTAKES

- Forgetting `await` on user-event calls
- Calling `userEvent.click` without `setup()` in v14
- Wrapping every call in `act` (RTL already does)

✓ TIPS & BEST PRACTICES

- Create `user` before `render` in each test
- Assert on visible results, not handler calls
- Write a `setup()` helper that renders and returns `user` plus key elements

QUICK SUMMARY

- `const user = userEvent.setup()`
- `await user.click/type/keyboard`
- `{Enter}` for special keys
- `fireEvent` only for edge cases

Testing Async Code, Hooks & Mocks

findBy · waitFor · renderHook · vi.mock · MSW · fake timers

WAITING, HOOKS AND FAKES

For async UI, prefer `findBy*` queries; use `waitFor(() => expect(…))` when you need to wait for an assertion that isn't a query. Test custom hooks with **`renderHook`**. Replace modules with `vi.mock`, spy on functions with `vi.fn()` and `vi.spyOn`, and control time with **fake timers**. For network calls, **MSW** (Mock Service Worker) intercepts requests at the network level, so components keep using real `fetch`.

NOTES

- `waitFor` retries its callback until it stops throwing or times out
- `renderHook` returns `{ result, rerender, unmount }`; read `result.current`
- Wrap hooks that need context with the `wrapper` option
- With fake timers, pass `advanceTimers: vi.advanceTimersByTime` to `userEvent.setup`

♿ ACCESSIBILITY

- Assert that error states render `role="alert"` so failures are announced

VERSION NOTES

- `renderHook` lives in `@testing-library/react`; `@testing-library/react-hooks` is deprecated
- MSW 2 uses `http` and `HttpResponse` (v1's `rest` and `res(ctx…)` are gone)

CODE EXPLAINED

→ MSW answers `/api/user` for every test; `server.use` overrides one test

→ `findByText` waits for the fetched name to render

→ `renderHook` runs the hook in a test component; `act` applies the update

→ `vi.mock` replaces the analytics module with a spy

→ Fake timers skip the 300 ms debounce instantly

ASYNC, HOOKS, MOCKS

```
import { act, fireEvent, render, renderHook, screen }
  from '@testing-library/react';
import { http, HttpResponse } from 'msw';
import { setupServer } from 'msw/node';

const server = setupServer(
  http.get('/api/user', () => HttpResponse.json({ name: 'Ada' })),
);
beforeAll(() => server.listen());
afterEach(() => server.resetHandlers());
afterAll(() => server.close());

test('loads the user', async () => {
  render(<Profile />);
  expect(await screen.findByText('Ada')).toBeInTheDocument();
});

test('shows an error', async () => {
  server.use(http.get('/api/user', () =>
    new HttpResponse(null, { status: 500 })));
  render(<Profile />);
  expect(await screen.findByRole('alert')).toHaveTextContent(/error/i);
});

test('useCounter', () => {
  const { result } = renderHook(() => useCounter(5));
  act(() => result.current.increment());
  expect(result.current.count).toBe(6);
});

vi.mock('./analytics', () => ({ track: vi.fn() }));

test('debounced search', async () => {
  vi.useFakeTimers();
  const onSearch = vi.fn();
  render(<Search onSearch={onSearch} />);
  fireEvent.change(screen.getByRole('searchbox'),
    { target: { value: 'mug' } });
  act(() => vi.advanceTimersByTime(300));
  expect(onSearch).toHaveBeenCalledWith('mug');
  vi.useRealTimers();
});
```

ASYNC HELPERS

HELPER	USE
`findBy*`	wait for an element
`waitFor(fn)`	wait for an assertion
`waitForElementToBeRemoved(el)`	wait for removal
`act(fn)`	flush manual updates
`vi.advanceTimersByTime(ms)`	move fake time

MOCKING TOOLS

TOOL	REPLACES
`vi.fn()`	a function (spy)
`vi.spyOn(obj, 'm')`	a method, keeps original
`vi.mock('mod')`	a whole module
`MSW`	network responses
`vi.useFakeTimers()`	`setTimeout`, `Date`

⚠ COMMON MISTAKES

- Side effects inside `waitFor` callbacks (they run many times)
- Mocking `fetch` by hand in every test instead of MSW
- Forgetting to restore real timers

✓ TIPS & BEST PRACTICES

- Share MSW handlers between tests, Storybook and local dev
- Test hooks through components when possible; use `renderHook` for reusable hooks
- Set `onUnhandledRequest: 'error'` in MSW to catch unmocked calls

QUICK SUMMARY

- `findBy` / `waitFor` for async
- `renderHook` for hooks
- MSW for the network
- `vi.fn`, `vi.mock`, fake timers

Environment Variables

import.meta.env · VITE_ prefix · .env files · modes · no secrets in the client · typing env

CONFIGURATION AT BUILD TIME

Vite loads variables from `.env` files and exposes those prefixed with **`VITE_`** on `import.meta.env`. Values are **inlined into the bundle at build time**, so anything you expose is public: never put secrets in client env vars. Vite picks files by **mode** (`development` for `dev`, `production` for `build`), and built-ins like `import.meta.env.MODE`, `DEV` and `PROD` are always available.

NOTES

- Load order: `.env` → `.env.local` → `.env.[mode]` → `.env.[mode].local` (later wins)
- `*.local` files are for machine-specific values and should be gitignored
- Changing `.env` requires restarting the dev server
- Type custom variables in `src/vite-env.d.ts` with `ImportMetaEnv`

♿ ACCESSIBILITY

- Gate dev-only tools (mock APIs, debug overlays) behind `import.meta.env.DEV` so they never ship to users

VERSION NOTES

- Next.js uses `NEXT_PUBLIC_` and React Router framework uses server-side `process.env`; prefixes differ by tool
- CRA's `REACT_APP_` prefix doesn't work in Vite

CODE EXPLAINED

→ Only `VITE_`-prefixed variables reach client code

→ `.env.production` overrides `.env` during `vite build`

→ `DB_PASSWORD` stays out of the bundle because it lacks the prefix

→ `import.meta.env.DEV` starts mock APIs in dev; the branch is removed from production builds

→ `ImportMetaEnv` types the variables for autocomplete

.ENV FILES AND IMPORT.META.ENV

```
# .env  (committed: safe defaults)
VITE_APP_NAME=Acme
VITE_API_URL=http://localhost:3000

# .env.production
VITE_API_URL=https://api.acme.com

# .env.local  (gitignored)
VITE_MAPS_KEY=pk_test_123          # public key, still visible!
DB_PASSWORD=secret                 # not exposed (no VITE_ prefix)

// src/api.ts
const base = import.meta.env.VITE_API_URL;
export const getUsers = () => fetch(`${base}/users`);

if (import.meta.env.DEV) {
  const { worker } = await import('./mocks/browser');
  await worker.start();              // MSW only in dev
}

console.log(import.meta.env.MODE);   // 'development' | 'production'

// src/vite-env.d.ts
/// <reference types="vite/client" />
interface ImportMetaEnv {
  readonly VITE_APP_NAME: string;
  readonly VITE_API_URL: string;
}
interface ImportMeta { readonly env: ImportMetaEnv; }

# custom mode
npx vite build --mode staging      # loads .env.staging
```

BUILT-IN VARIABLES

VARIABLE	VALUE
`import.meta.env.MODE`	current mode
`import.meta.env.DEV`	`true` in dev
`import.meta.env.PROD`	`true` in build
`import.meta.env.BASE_URL`	`base` config
`import.meta.env.SSR`	`true` on server

SECRETS

VALUE	WHERE
Public API URL	VITE_ var
Publishable keys (maps, Stripe pk)	VITE_ var
Secret API keys	server / serverless function
DB credentials	server only
Per-environment config	`.env.[mode]`

⚠ COMMON MISTAKES

- Putting secret keys in `VITE_` variables
- Using `process.env.X` in Vite client code
- Committing `.env.local`

✓ TIPS & BEST PRACTICES

- Validate env vars at startup with Zod
- Keep a `.env.example` with every key and dummy values
- Proxy API calls through your server when a key must stay secret

QUICK SUMMARY

- `VITE_` prefix → `import.meta.env`
- Inlined at build: public
- Mode picks `.env.[mode]`
- Type with `ImportMetaEnv`

Build & Deploy

SHIPPING A VITE APP

`npm run build` produces a static `dist/` folder: an `index.html` plus hashed JS and CSS files. Any static host can serve it. Single-page apps need one server rule: **rewrite unknown paths to `index.html`** so client-side routes work on refresh. Hashed assets can be cached forever; `index.html` should not be cached. If the app lives in a subfolder, set Vite's `base` option.

NOTES

- `npm run preview` serves `dist/` locally to test the real build
- Asset file names include content hashes, so new deploys bust caches automatically
- GitHub Pages serves from `/<repo>/`: set `base: '/<repo>/'`
- SSR or framework apps need a Node, edge or serverless runtime instead of static hosting

♿ ACCESSIBILITY

- Test the production build with Lighthouse and axe; minification can't break semantics, but missing assets can break icon labels

VERSION NOTES

- Vercel, Netlify, Cloudflare Pages and GitHub Pages all host Vite builds
- React Router framework mode and Next.js have their own adapters and build commands

CODE EXPLAINED

- → `vite build` bundles, minifies and hashes everything into `dist/`
- → `preview` catches problems that only appear in production builds
- → `base` prefixes asset URLs when the app isn't at the domain root
- → Each host's rewrite rule sends deep links like `/users/42` to `index.html`
- → Long-lived caching is safe for hashed files in `/assets/`

BUILD, PREVIEW, HOST

```
# build and check
npm run build          # -> dist/
npm run preview        # http://localhost:4173

dist/
├── index.html
└── assets/
    ├── index-Bx3kq9.js
    └── index-C8dPa1.css

// vite.config.ts: app served from a subfolder
export default defineConfig({
  base: '/my-app/',
  build: { sourcemap: true },
});

# Netlify: public/_redirects
/*    /index.html   200

// Vercel: vercel.json
{ "rewrites": [{ "source": "/(.*)", "destination": "/" }] }

# Nginx
location / {
  try_files $uri /index.html;
}
location /assets/ {
  add_header Cache-Control "public, max-age=31536000, immutable";
}

# GitHub Pages (Actions) uploads dist/ as the Pages artifact
# Cloudflare Pages: build command "npm run build", output "dist"
```

HOSTS

HOST	SPA FALLBACK
Netlify	`_redirects` `/* /index.html 200`
Vercel	`rewrites` in `vercel.json`
Cloudflare Pages	automatic for SPAs
GitHub Pages	`404.html` copy of `index.html`
Nginx	`try_files $uri /index.html`
S3 + CloudFront	error page → `/index.html`

CACHING

FILE	CACHE-CONTROL
`/assets/*-[hash].js`	`max-age=31536000, immutable`
`index.html`	`no-cache`
Images in `public/`	moderate (no hash)
Service worker	`no-cache`

⚠ COMMON MISTAKES

- 404s on refresh because the host has no SPA rewrite
- Caching `index.html`, so users keep loading old asset names
- Wrong `base` and blank pages on GitHub Pages

✓ TIPS & BEST PRACTICES

- Deploy previews per pull request (Vercel/Netlify/Cloudflare)
- Upload source maps to your error tracker, not the public site
- Check bundle size in CI and fail on large regressions

QUICK SUMMARY

- `build` → static `dist/`
- Rewrite routes to `index.html`
- Cache hashed assets forever
- Set `base` for subfolders

Upgrading React

18 → 19 changes · codemods · removed APIs · propTypes / defaultProps · ReactDOM.render removal · @types/react changes

MOVING TO REACT 19

Upgrade in two steps: first to **React 18.3**, which adds warnings for every API removed in 19, then to **19**. Run the official **codemods** to update most code automatically, and update `@types/react` with the types codemod. The biggest removals are `ReactDOM.render`, string refs, legacy context, `propTypes` checking and `defaultProps` on function components. After that, adopt new features gradually: Actions, `use`, ref as a prop.

NOTES

- `npx codemod@latest react/19/migration-recipe` runs the main code changes
- `npx types-react-codemod@latest preset-19 ./src` updates TypeScript types
- Libraries must support React 19 too; check peer dependencies
- Minor releases since 19.0 (19.1, 19.2, 19.3) add features without breaking changes

♿ ACCESSIBILITY

- Re-run accessibility tests after upgrading; changed error handling or Suspense timing can affect focus and announcements

VERSION NOTES

- React 19.0 (Dec 2024), 19.1 (Mar 2025), 19.2 (Oct 2025), 19.3 (Sep 2026)
- React 18 apps keep working on 18.3; security patches are backported to supported lines

CODE EXPLAINED

→ 18.3 warns about every API that 19 removes

→ The migration recipe codemod rewrites renders, refs and imports

→ The types codemod fixes `@types/react` 19 breakages

→ Default parameters replace `defaultProps`

→ `ref` as a prop replaces `forwardRef`

UPGRADE STEPS AND COMMON CHANGES

```
# 1. Step through 18.3 to see deprecation warnings
npm install react@18.3 react-dom@18.3

# 2. Upgrade
npm install react@^19 react-dom@^19
npm install -D @types/react@^19 @types/react-dom@^19

# 3. Codemods
npx codemod@latest react/19/migration-recipe
npx types-react-codemod@latest preset-19 ./src

// Removed: ReactDOM.render
ReactDOM.render(<App />, el);                    // ✗
createRoot(el).render(<App />);                  // ✓

// Removed: defaultProps on function components
Button.defaultProps = { size: 'md' };            // ✗
function Button({ size = 'md' }) {}              // ✓

// Removed: propTypes checking (silently ignored)
Button.propTypes = { size: PropTypes.string };   // ✗ use TS

// Removed: string refs
<input ref="name" />                             // ✗
<input ref={nameRef} />                          // ✓

// Deprecated: forwardRef, Context.Provider
const In = forwardRef((p, ref) => <input ref={ref} />);
function In2({ ref }) { return <input ref={ref} />; }  // ✓

// Types: useRef needs an argument; global JSX removed
useRef<number>();                                // ✗
useRef<number | null>(null);                     // ✓
```

REMOVED IN 19

API	REPLACEMENT
`ReactDOM.render / hydrate`	`createRoot / hydrateRoot`
`unmountComponentAtNode`	`root.unmount()`
`findDOMNode`	refs
`String refs`	callback or object refs
`Legacy context (contextTypes)`	`createContext`
`propTypes, function defaultProps`	TypeScript, default params
`react-test-renderer/shallow`	RTL

ADOPT GRADUALLY

NEW FEATURE	SHEET
`Actions / useActionState`	28, 29
`use()`	47
`useOptimistic`	50
`Ref as a prop`	33
`Metadata tags`	53
`<Activity>, useEffectEvent (19.2)`	51, 35

⚠ COMMON MISTAKES

- Jumping straight from 17 or 18.0 to 19 without the 18.3 step
- Upgrading React before checking that key libraries support 19
- Rewriting everything to new APIs at once

✓ TIPS & BEST PRACTICES

- Upgrade on a branch with full tests and StrictMode enabled
- Read the official React 19 Upgrade Guide for edge cases
- Adopt the React Compiler after the upgrade settles

QUICK SUMMARY

- 18.3 first, then 19
- Run code + types codemods
- Removed: render, string refs, propTypes
- Adopt new APIs gradually

Quick Reference

REACT AT A GLANCE

A one-page index of the APIs covered in this set. **Hooks** must be called at the top level of components and custom hooks; `use` is the exception. **Components** from `react` handle loading, errors and visibility; **react-dom** handles rendering, portals, forms and resource loading. Directives (`'use client'`, `'use server'`) mark module boundaries in frameworks that support Server Components.

NOTES

- State: `useState`, `useReducer`; context: `useContext`, `use`
- Refs: `useRef`, `useImperativeHandle`; effects: `useEffect`, `useLayoutEffect`, `useInsertionEffect`, `useEffectEvent`
- Performance: `useMemo`, `useCallback`, `useTransition`, `useDeferredValue`
- Other: `useId`, `useSyncExternalStore`, `useActionState`, `useOptimistic`, `useDebugValue`

♿ ACCESSIBILITY

- `useId` for label/ARIA links · semantic elements first · manage focus on route changes · `aria-live` for async results

VERSION NOTES

- Covers React 19.3 (September 2026)
- See sheet 97 for removed and deprecated APIs

CODE EXPLAINED

- → Hooks return values and setters; they never take `this`
- → `use(Ctx)` and `use(promise)` may run conditionally
- → Form hooks pair with React 19 Actions
- → `Suspense`, `Activity` and `ViewTransition` are components, not hooks
- → `createPortal` renders children into another DOM node

CHEAT SHEET

```
// State & refs
const [v, setV] = useState(init);
const [s, dispatch] = useReducer(reducer, init);
const ref = useRef(null);

// Effects
useEffect(() => { /* sync */ return () => {}; }, [deps]);
const onEvt = useEffectEvent(() => {/* latest values */});

// Context
const Ctx = createContext(def);
<Ctx value={x}>…</Ctx>;   const x2 = use(Ctx);

// Performance & concurrency
const memo1 = useMemo(() => calc(a), [a]);
const [isPending, startTransition] = useTransition();
const deferred = useDeferredValue(value);

// Forms & actions
<form action={async (fd) => {}}>…</form>
const [st, act, pending] = useActionState(fn, init);
const [opt, addOpt] = useOptimistic(state, update);
const { pending: p } = useFormStatus();   // react-dom

// Components
<Suspense fallback={<Spin />}>…</Suspense>
<Activity mode="hidden">…</Activity>
<ViewTransition name="x">…</ViewTransition>
const Lazy = lazy(() => import('./X'));
const Fast = memo(Comp);

// react-dom
createRoot(el).render(<App />);
createPortal(<Modal />, document.body);
flushSync(() => setV(1));
preload('/font.woff2', { as: 'font' });

// Directives
'use client';   'use server';
```

BUILT-IN COMPONENTS

COMPONENT	PURPOSE
`<Fragment> / <>`	group without DOM
`<StrictMode>`	dev checks
`<Suspense>`	loading fallback
`<Activity>`	hide, keep state (19.2)
`<ViewTransition>`	animate transitions (19.3)
`<Profiler>`	measure renders

JSX & EVENT RULES

RULE	EXAMPLE
`className, htmlFor`	not `class, for`
`camelCase events`	`onClick, onKeyDown`
`style object`	`style={{ gap: 8 }}`
`List keys`	stable IDs
`One root`	`<>…</>`
`{cond && <X />}`	boolean `cond`

⚠ COMMON MISTAKES

- Conditional hooks
- Mutating state
- Effects for derived data

✓ TIPS & BEST PRACTICES

- Data down, events up
- Derive, don't sync
- Measure before memoizing

QUICK SUMMARY

- Hooks at the top level
- Pure render, immutable state
- Suspense + transitions for async
- Actions for forms